Q1. **BREAKCHECK AI - TELEVISION ADVERTISING CLEARANCE AND SPOT COMPLIANCE PLATFORM**

**PROBLEM STATEMENT**

Harbourlight Media runs four commercial television channels in the UK — **Harbourlight One**, **Harbourlight Sport**, **Harbourlight Drama** and **Harbourlight Life**. It built the group by buying three smaller broadcasters, and each channel is still scheduled on the system it came with. Harbourlight Sport's scheduler, **TXLine**, writes the day the way broadcast engineers do: a broadcast day that runs from **06:00** to **05:59** the next morning, with the hours after midnight written **24:00** to **29:59**. The other three write the ordinary clock on the calendar day, so a spot at ten past one on Sunday morning is **25:10** on Saturday in one system and **01:10** on Sunday in the others.

Advertising agencies book thousands of spots a week. Five product categories are restricted — **Alcohol**, **Betting and gaming**, **Less healthy food**, **Consumer credit** and **Medicines and health** — and before a campaign in one of them goes to air, Harbourlight's clearance desk must clear it: the copy must be acceptable, and every booked spot must sit where that category's code allows it. Most campaigns are fine. A few are not — a betting spot in the break straight after a football match that young people watch in large numbers, a spot at **05:40** on Sunday morning booked as part of "Saturday night", a campaign booked into one channel's broadcast day more often than its code allows. Each raises a question the desk has to answer exactly: which of these spots may air, and why not the others?

The desk has two problems. The queue is bigger than the desk, so the campaign that would put a betting spot in front of two hundred thousand young viewers waits behind twenty that are fine. And whether a spot may air depends on three things held in three places: the category's own code, written in one of five documents that share a single template and differ in their figures, in whether they restrict hours at all, in the reach of the children's rule and in one exception only the betting code has; the programme around the break, with its forecast under-18 audience index, whether it is live sport and whether it was made for children; and the broadcast day the spot belongs to, which is not always the day the scheduler wrote. Read a spot against the wrong code, or on the wrong day, and a spot that breaks the code is cleared, or a spot the advertiser paid for is withdrawn.

Harbourlight requires a system that ranks the clearance queue by the under-18 audience the offending spots would reach, assembles the assessment for the campaign an executive opens, checks every booked spot against its category's own code read from that code's own document, drafts a recommendation, and stops there. No spot is cleared and nothing is sent to an agency on the word of a model.

**OBJECTIVE**

Build **BreakCheck AI**, a working advertising clearance platform that a clearance desk can use end to end. The finished system must:

- Clean five raw extracts that arrive with real data-quality defects, and produce the evidence that each one needed cleaning.
- Train two machine learning models on tabular data — one that predicts whether a campaign's copy will need changes before it can be cleared, and one that forecasts how many viewers under 18 a programme will have — and one NLP model that reads free-text clearance notes and names the copy issue they describe.
- Build a searchable knowledge base from sixteen company documents, so that every statement the system makes can be traced to the document it came from.
- Read each category's five rules — its restricted hours, the live sport exception where it has one, its under-18 index threshold, the reach of its children's rule, its daily limit and its separation — out of that category's own code. No hour, threshold, limit or separation is written into the solution or the configuration.
- Place every booked spot on the broadcast day and the broadcast clock, decide which programme its break is in or next to, and test it against all five rules of its own category's code.
- Run a multi-agent workflow in which each agent is a single language model call that interprets facts the models, the documents and the tools established, and decides no spot of its own.
- Stop that workflow before any spot is cleared or anything is sent to an agency, wait for a human decision by a Clearance Controller other than the person who opened the assessment, and survive the machine being switched off while it waits.
- Record every run, every assessment, every decision, every language model call a run makes and every chat question in a database that can be audited afterwards.

- Serve all of it through a REST service and a dashboard a Clearance Controller could use without being told how.
- Put every fact about a campaign that the dashboard shows behind that REST service, so the dashboard holds no business rule and reads no file of its own.
- Require everyone to sign in, and show each of the three roles only the pages their role covers, with the service refusing the matching request on its own account rather than trusting the dashboard to hide it.
- Answer questions from the documents the chat may read and from nothing else, in exactly one language model call per question, refusing rather than guessing when the documents do not cover what was asked, and telling an Agency Liaison about the campaigns of their own agencies only.

**PROJECT STRUCTURE**

```
Project/
|-- .env                              <- pre-loaded, your key goes in it
|-- .streamlit/                       <- pre-loaded
|   '-- config.toml
|-- access/
|   |-- password_security.py
|   '-- permissions.py
|-- agentic/
|   |-- agents.py
|   |-- graph.py
|   |-- nodes.py
|   '-- state.py
|-- api/
|   |-- routes.py
|   '-- schemas.py
|-- chat/
|   |-- campaign_status_answer.py
|   |-- clearance_assistant.py
|   '-- conversation_memory.py
|-- config/
|   '-- config.py                     <- pre-loaded, complete, do not edit
|-- dashboard/
|   |-- api_client.py
|   |-- audit_trail.py
|   |-- campaign_detail.py
|   |-- clearance_chat.py
|   |-- clearance_queue.py
|   '-- decision_queue.py
|-- data/                             <- pre-loaded
|   |-- docs/                         <- the sixteen PDF documents
|   |-- input/
|   |   |-- booked_spots.csv
|   |   |-- open_campaign_notes.csv
|   |   |-- open_campaigns.csv
|   |   '-- programme_schedule.csv
|   |-- campaign_submissions_eval.csv
|   |-- campaign_submissions_train.csv
|   |-- clearance_notes.csv
|   |-- programme_audiences_eval.csv
|   '-- programme_audiences_train.csv
|-- data_pipeline/
|   |-- audience_cleaning.py
|   |-- common_cleaning.py
|   |-- dataset_loader.py
|   |-- feature_builder.py
|   |-- note_cleaning.py
|   |-- quality_profiler.py
|   '-- submission_cleaning.py
|-- database/
|   |-- breakcheck.db                 <- created by run_database_setup.py
|   |-- checkpoint.db                 <- created by run_api_service.py
|   '-- chroma_store/                 <- created by run_knowledge_indexing.py
|-- knowledge/
|   |-- chunk_splitting.py
|   |-- code_reading.py
|   |-- pdf_extraction.py
|   '-- vector_index.py
|-- modeling/
|   |-- amendment_model.py
|   |-- exposure_model.py
|   |-- issue_model.py
|   |-- model_evaluation.py
|   '-- model_registry.py
|-- output/                           <- created by run_data_pipeline.py
|   |-- models/                       <- created by run_model_training.py
|   |-- data_quality_report.json
|   '-- model_metrics.json
|-- persistence/
|   |-- database.py
|   |-- models.py
|   '-- repository.py
|-- processed/                        <- created by run_data_pipeline.py
|-- tools/
|   |-- campaign_data_tools.py
|   |-- prediction_tools.py
|   |-- retrieval_tools.py
|   '-- schedule_tools.py
|-- requirements.txt                  <- pre-loaded
|-- run_api_service.py
|-- run_dashboard.py
|-- run_data_pipeline.py
|-- run_database_setup.py
|-- run_knowledge_indexing.py
'-- run_model_training.py
```

## DATASET DESCRIPTION

### campaign_submissions_train.csv —

| Column | Type once cleaned | What it holds |
|---|---|---|
| submission_id | **str** | The clearance system's own identifier for the submission |
| campaign_reference | **str** | The campaign, in the form **CLR-AAA-NNNNNNN**, the middle part being the agency's code. **config.CAMPAIGN_REFERENCE_COLUMN** names it |
| clock_number | **str** | The copy's identity, in the form **AAA/BBBB123/030**: the agency, the advertiser and the copy, then the copy's length in seconds in its last three digits. **config.CLOCK_NUMBER_COLUMN** names it and **config.CLOCK_NUMBER_PATTERN** reads it |
| agency_code | **str** | The agency that submitted the copy and booked the spots. **config.AGENCY_CODE_COLUMN** names it |
| advertiser_code | **str** | The advertiser whose product the copy sells |
| submitted_via | **str** | How the copy was submitted: Clearance portal, Email or Agency API |
| product_category | **str** | The restricted category, and so the code the spots answer to: one of the five categories. **config.PRODUCT_CATEGORY_COLUMN** names it |
| sub_category | **str** | The kind of product within the category |
| copy_length_seconds | **int** | The copy's length in seconds. **config.COPY_LENGTH_COLUMN** names it. Written several ways, and on some rows it disagrees with the clock number |
| supers_count | **int** | On-screen text supers in the copy |
| legal_text_seconds | **int** | Seconds the legal small print is on screen. **config.LEGAL_TEXT_SECONDS_COLUMN** names it |
| voiceover_flag | **str** | The copy has a voiceover |
| lead_performer_age_band | **str** | The lead performer's age band, or No performer |
| celebrity_flag | **str** | A celebrity appears |
| sports_personality_flag | **str** | A sports personality appears |
| child_appeal_elements_count | **int** | Cartoon characters, mascots, toys or games-style graphics in the copy |
| humour_flag | **str** | The copy relies on humour |
| price_claim_flag | **str** | The copy states a price, a saving, odds or an offer |
| health_claim_flag | **str** | The copy claims a health or nutrition benefit |
| environmental_claim_flag | **str** | The copy claims an environmental benefit |
| comparative_claim_flag | **str** | The copy compares the product with competitors |
| offer_terms_shown_flag | **str** | The offer's significant terms are on screen; empty when the copy makes no offer |
| bonus_or_free_bet_flag | **str** | The copy offers a bonus or a free bet |
| representative_apr_shown_flag | **str** | The representative APR is on screen; empty outside consumer credit |
| claims_count | **int** | Objective claims the copy makes |
| spots_booked | **int** | Spots the campaign booked across the booking week. Written with a unit on some rows |
| channels_booked | **int** | Channels the spots were booked on, at most four. Written with a unit on some rows |
| share_before_21_pct | **int** | Per cent of the spots booked between **06:00** and **21:00** of the broadcast day. Written with a sign on some rows |
| share_in_live_sport_pct | **int** | Per cent of the spots booked into centre breaks of live sport coverage |
| days_to_first_transmission | **int** | Days from submission to the first booked spot |
| advertiser_campaigns_past_year | **int** | The advertiser's campaigns cleared in the past year. **config.ADVERTISER_CAMPAIGNS_COLUMN** names it |
| advertiser_amendments_past_year | **int** | How many of those needed changes to the copy. **config.ADVERTISER_AMENDMENTS_COLUMN** names it |
| agency_amendment_rate_pct | **float** | Per cent of the agency's copy needing changes over the previous twelve months |
| new_advertiser_flag | **str** | The advertiser had no campaigns in the past year |
| desk_code | **str** | The clearance desk that logged the submission. Carries no signal |
| weekday_submitted | **str** | The weekday the copy was submitted. Carries no signal |
| batch_code | **str** | The intake batch. Carries no signal |
| amendment_required | **str** | **The answer.** The copy needed changes before it could be cleared. **config.SUBMISSION_TARGET_COLUMN** names it |

### campaign_submissions_train.csv: SAMPLE RECORDS

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| submission_id | SUB431988 | SUB152986 | SUB710750 |
| campaign_reference | CLR-OKB-0007882 | CLR-OKB-0013881 | CLR-ASH-0029310 |
| clock_number | OKB/OSIV035/040 | OKB/BLUF078/060 | ASH/QUIO059/030 |
| agency_code | OKB | OKB | ASH |
| advertiser_code | OSIV | BLUF | QUIO |
| submitted_via | Clearance portal | Email | Not recorded |
| product_category | Medicines & Health | B&G | alcohol |
| sub_category | Vitamins and supplements | Sports betting | Cider |
| copy_length_seconds | 40s | 00:01:30 | 0:30 |
| supers_count | 4 | 3 | 7 |
| legal_text_seconds | 14 | 13 | 2 |
| voiceover_flag | Y | No | yes |
| lead_performer_age_band | No performer shown | 25 to 34 | 25 to 34 |
| celebrity_flag | No | Yes | 0 |
| sports_personality_flag | No | Yes | no |
| child_appeal_elements_count | 0 | 0 | 0 |
| humour_flag | No | Yes | false |
| price_claim_flag | false | No | Yes |
| health_claim_flag | Yes | No | No |
| environmental_claim_flag | false | No | No |
| comparative_claim_flag | FALSE | No | No |
| offer_terms_shown_flag | ? | TRUE | Yes |
| bonus_or_free_bet_flag | No | true | N |
| representative_apr_shown_flag | | | N/A |
| claims_count | 1 | 1 | 1 |
| spots_booked | 14 | 21 | 36 |
| channels_booked | 1 | 1 | 2 |
| share_before_21_pct | 35% | 19 % | 43% |
| share_in_live_sport_pct | 1 | 62% | 6 |
| days_to_first_transmission | 4 | 6 | 8 |
| advertiser_campaigns_past_year | 67 | 40 | 25 |
| advertiser_amendments_past_year | 3 | 11 | 4 |
| agency_amendment_rate_pct | 46.9 | 45.9 | 31.7 |
| new_advertiser_flag | false | No | No |
| desk_code | CD-01 | CD-01 | - |
| weekday_submitted | Tuesday | Wednesday | Thursday |
| batch_code | BCH-011 | BCH-058 | BCH-056 |
| amendment_required | false | Yes | yes |

**campaign_submissions_eval.csv —**
The same columns, held out for scoring the amendment model.

**programme_audiences_train.csv —**

| Column | Type once cleaned | What it holds |
|---|---|---|
| airing_id | **str** | The audience system's identifier for one airing of one programme |
| programme_title | **str** | The programme. **config.PROGRAMME_TITLE_COLUMN** names it |
| channel | **str** | The channel it aired on, arriving in several spellings and as channel codes. **config.CHANNEL_COLUMN** names it |
| scheduling_system | **str** | The channel's scheduling system. It decides which clock the raw times are written on. **config.SCHEDULING_SYSTEM_COLUMN** names it |
| transmission_day | **str** | The day as the scheduler wrote it, in several notations: the broadcast day from TXLine, the calendar day from the other three. Once cleaned it is the weekday of the broadcast day the programme belongs to. **config.TRANSMISSION_DAY_COLUMN** names it |
| start_time | **str** | The start, on the system's own clock as it arrives; once cleaned, on the broadcast clock, written **HH:MM** from **06:00** to **29:59**. **config.START_TIME_COLUMN** names it |
| end_time | **str** | The end, on the same terms; once cleaned, after the start and at most **30:00**. **config.END_TIME_COLUMN** names it |
| genre | **str** | The programme's genre |
| live_flag | **str** | Broadcast live |
| live_sport_flag | **str** | Live sport coverage. **config.LIVE_SPORT_FLAG_COLUMN** names it |
| made_for_children_flag | **str** | Made for children. **config.MADE_FOR_CHILDREN_FLAG_COLUMN** names it |
| premiere_flag | **str** | The first showing of this episode |
| episode_number | **int** | The episode number through the programme's run; empty for programmes that are not episodes of a series |
| school_holiday_flag | **str** | The week falls in school holidays |
| day_type | **str** | Weekday, Weekend or Bank holiday |
| lead_in_genre | **str** | The genre of the programme before it on the same channel |
| forecast_under18_index | **int** | The forecast under-18 audience index published before the programme aired, **100** being the average. **config.FORECAST_INDEX_COLUMN** names it |
| audience_all | **float** | All viewers, in thousands, as measured after the programme aired. Known only afterwards |
| region_feed_code | **str** | The playout region feed. Carries no signal |
| playout_code | **str** | The playout server. Carries no signal |
| under18_viewers | **float** | **The answer.** Viewers under 18, in thousands, as measured. **config.AUDIENCE_TARGET_COLUMN** names it |

**programme_audiences_train.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| airing_id | AIR618158 | AIR554452 | AIR788495 |
| programme_title | Night Film | Airport Live | Drama Shorts |
| channel | H-One | HLL | H-Drama |
| scheduling_system | Airtime Planner | Planit | SlotMaster |
| transmission_day | Fr. | FRIDAY | We. |
| start_time | 02:00 | 23:00 | 05:00 |
| end_time | 04:30 | 00:00 | 05:30 |
| genre | Film | Documentary | Drama |
| live_flag | no | false | 0 |
| live_sport_flag | No | No | No |
| made_for_children_flag | No | No | No |
| premiere_flag | No | Yes | No |
| episode_number | Not recorded | 54 | 7 |
| school_holiday_flag | No | no | Y |
| day_type | Weekday | Weekday | Weekday |
| lead_in_genre | Film | Doc | Crime drama |
| forecast_under18_index | 34 | 37 | 36 |
| audience_all | 0.0533m | 95.4 | 0.015m |
| region_feed_code | RF-01 | RF-02 | RF-06 |
| playout_code | PLY-1031 | PLY-2871 | PLY-7924 |
| under18_viewers | 3.6 thousand | 7.3 thousand | 1,000 |

**programme_audiences_eval.csv —**
The same columns from ten other weeks, held out for scoring the exposure model.

**clearance_notes.csv —**

| Column | Type once cleaned | What it holds |
|---|---|---|
| note_id | **str** | The note's own identifier |
| campaign_reference | **str** | The campaign the note is about |
| agency_code | **str** | The agency that submitted the copy |
| clock_number | **str** | The copy's identity |
| author_id | **str** | Who wrote it: **EX-** an executive, **LG-** a legal adviser, **AC-** an agency contact, **CC-** a controller |
| author_role | **str** | The author's role |
| note_source | **str** | The kind of note: Executive first read, Legal review, Agency reply, Executive update or Controller comment. **config.NOTE_SOURCE_COLUMN** names it |
| note_sequence | **int** | The note's position among that campaign's notes, oldest first. **config.NOTE_SEQUENCE_COLUMN** names it |
| note_written_at | **str** | The weekday and time the note was written, the weekday arriving in several notations. **config.NOTE_TIMESTAMP_COLUMN** names it |
| product_category | **str** | The campaign's restricted category |
| note_text | **str** | The note itself, written at speed. **config.NOTE_TEXT_COLUMN** names it |
| note_length_chars | **int** | Characters in the note. **config.NOTE_LENGTH_COLUMN** names it, and it is measured on the text as it arrived, before any repair |
| issue_category | **str** | **The answer.** The copy issue the note describes, one of the eight keys of **config.GUIDE_DOCUMENT_BY_ISSUE**. **config.NOTE_TARGET_COLUMN** names it |

**clearance_notes.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| note_id | CN3066586 | CN6774819 | CN1628825 |
| campaign_reference | CLR-MSL-0002297 | CLR-OKB-0039396 | CLR-HWK-0027483 |
| agency_code | MSL | OKB | HWK |
| clock_number | MSL/PNBR025/030 | OKB/WENL092/030 | HWK/FIZW050/030 |
| author_id | EX-2213 | EX-2363 | EX-2272 |
| author_role | Clearance Executive | Clearance Executive | clearance executive |
| note_source | executive first read | Exec first read | Executive first read |
| note_sequence | 1 | 1 | 1 |
| note_written_at | MONDAY 08:36 | Mo. 09:42 | Thu 08:52 |
| product_category | Consumer Credit | Alcohol | less healthy food |
| note_text | FIRST READ OF 'NEW WHEELS': THE COST OF BORROWIN... | The 30-second copy uses animated characters, which ... | <p>Fizzwell Soda: Food copy states the fizzy drink is 'pac... |
| note_length_chars | 177 | 100 | 145 |
| issue_category | Credit promotion | Under-18 appeal | HEALTH CLAIM |

**open_campaigns.csv —**
The live queue's campaigns, named by **config.OPEN_CAMPAIGNS_FILE**, with **no answer column**. None of the **240** campaigns appears in the history.
Its first **37** columns are the submissions extract's columns other than **amendment_required**, in the same order; the **three** below follow them, in this order. The live queue is read as text, and an empty cell is a missing value.

| Column | Type as read | What it holds |
|---|---|---|
| advertiser_name | **str** | The advertiser's trading name. **config.ADVERTISER_NAME_COLUMN** names it |
| agency_name | **str** | The agency's name, to which a restriction notice is addressed. **config.AGENCY_NAME_COLUMN** names it |
| copy_title | **str** | The copy's working title. **config.COPY_TITLE_COLUMN** names it |

It arrives clean: canonical categories, plain numbers, flags **Yes** or **No**, and the offer-terms and APR flags empty where the copy makes no offer or the product is not consumer credit.

**booked_spots.csv —**
Every spot the live campaigns booked, named by **config.BOOKED_SPOTS_FILE**, exactly as each channel's scheduler wrote it.

| Column | Type as read | What it holds |
|---|---|---|
| spot_reference | **str** | The booked spot, in the form **SPT-NNNNNNN-NN**; two spots of one campaign in one break go out in the order of their references. **config.SPOT_REFERENCE_COLUMN** names it |
| campaign_reference | **str** | The campaign that booked it |
| clock_number | **str** | The copy the spot carries |
| channel | **str** | The channel the spot is booked on |
| scheduling_system | **str** | The channel's scheduling system, which decides the clock and the notation the spot is written in |
| transmission_day | **str** | The day as the scheduler wrote it: **SAT** from TXLine, **Saturday** from Airtime Planner, **Sat** from SlotMaster, **Sa.** from Planit |
| break_time | **str** | The start of the break the spot is booked into, on the scheduler's own clock. **config.BREAK_TIME_COLUMN** names it |
| spot_length_seconds | **str** | The spot's length in the scheduler's own notation: **30"**, **0:30**, **30s** or **00:00:30**. **config.SPOT_LENGTH_COLUMN** names it |

**config.BOOKED_SPOT_REQUIRED_COLUMNS** lists the details every booked spot must carry, and **config.BOOKED_SPOT_VALUE_MAPPINGS** the spellings its day, scheduling system and channel are folded from.

**programme_schedule.csv —**
The booking week's programmes, named by **config.PROGRAMME_SCHEDULE_FILE**, clean and on the broadcast clock: each channel's schedule runs without a gap from **06:00** to **30:00** on each of the seven broadcast days. It carries no audience.

| Column | Type as read | What it holds |
|---|---|---|
| schedule_entry_id | **str** | The scheduling identifier for one programme in the booking week. **config.SCHEDULE_ENTRY_COLUMN** names it |
| channel, scheduling_system, transmission_day | **str** | As in the audiences extract, already in canonical spelling; **transmission_day** is the broadcast day |
| start_time, end_time | **str** | The programme's start and end on the broadcast clock, written **HH:MM** |
| programme_title, genre, live_flag, live_sport_flag, made_for_children_flag, premiere_flag, episode_number, school_holiday_flag, day_type, lead_in_genre | **str** | As in the audiences extract |
| forecast_under18_index | **str** | The forecast under-18 audience index published before the booking week, a whole number |
| region_feed_code, playout_code | **str** | As in the audiences extract. Carry no signal |

**open_campaign_notes.csv —**
The live campaigns' notes, named by **config.OPEN_CAMPAIGN_NOTES_FILE**, read as text. Its columns are the notes extract's first twelve, in the same order, without **issue_category**. Every live campaign has at least one note.

### open_campaigns.csv: SAMPLE RECORDS

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| submission_id | SUB642540 | SUB280699 | SUB776705 |
| campaign_reference | CLR-QSC-0042117 | CLR-FWL-0042743 | CLR-MSL-0042822 |
| clock_number | QSC/SPWB017/030 | FWL/GOAU092/010 | MSL/RAVE272/010 |
| agency_code | QSC | FWL | MSL |
| advertiser_code | SPWB | GOAU | RAVE |
| submitted_via | Clearance portal | Email | Clearance portal |
| product_category | Betting and gaming | Betting and gaming | Alcohol |
| sub_category | Sports betting | Online casino | Beer and lager |
| copy_length_seconds | 30 | 10 | 10 |
| supers_count | 3 | 1 | 1 |
| legal_text_seconds | 6 | 3 | 2 |
| voiceover_flag | Yes | Yes | Yes |
| lead_performer_age_band | 25 to 34 | 35 to 49 | 50 and over |
| celebrity_flag | No | Yes | No |
| sports_personality_flag | No | Yes | No |
| child_appeal_elements_count | 0 | 0 | 0 |
| humour_flag | Yes | Yes | No |
| price_claim_flag | Yes | No | No |
| health_claim_flag | No | No | No |
| environmental_claim_flag | No | No | No |
| comparative_claim_flag | No | No | No |
| offer_terms_shown_flag | No | | |
| bonus_or_free_bet_flag | No | No | No |
| representative_apr_shown_flag | | | |
| claims_count | 1 | 2 | 0 |
| spots_booked | 10 | 10 | 33 |
| channels_booked | 2 | 2 | 2 |
| share_before_21_pct | 50 | 90 | 64 |
| share_in_live_sport_pct | 40 | 0 | 0 |
| days_to_first_transmission | 4 | 16 | 12 |
| advertiser_campaigns_past_year | 2 | 16 | 49 |
| advertiser_amendments_past_year | 2 | 2 | 14 |
| agency_amendment_rate_pct | 31.0 | 30.1 | 43.0 |
| new_advertiser_flag | No | No | No |
| desk_code | CD-03 | CD-02 | CD-05 |
| weekday_submitted | Tuesday | Sunday | Wednesday |
| batch_code | BCH-017 | BCH-006 | BCH-040 |
| advertiser_name | Spinwell Bet | Goalrush Slots | Ravensbrook Lager |
| agency_name | Quayside Creative | Fenwick Lane | Marlow Street |
| copy_title | Acca Boost | Lucky Streak | First Round |

**booked_spots.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 | Record 4 |
|---|---|---|---|---|
| spot_reference | SPT-0042117-08 | SPT-0042117-10 | SPT-0041903-02 | SPT-0042244-01 |
| campaign_reference | CLR-QSC-0042117 | CLR-QSC-0042117 | CLR-SWR-0041903 | CLR-CRM-0042244 |
| clock_number | QSC/SPWB017/030 | QSC/SPWB017/030 | SWR/BRAC196/020 | CRM/MUNO274/010 |
| channel | Harbourlight Sport | Harbourlight One | Harbourlight Drama | Harbourlight Life |
| scheduling_system | TXLine | Airtime Planner | SlotMaster | Planit |
| transmission_day | SAT | Sunday | Sun | Mo. |
| break_time | 25:10 | 05:40 | 01:10 | 16:30 |
| spot_length_seconds | 30" | 0:30 | 20s | 00:00:10 |

**programme_schedule.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| schedule_entry_id | SCH61430 | SCH23746 | SCH64978 |
| channel | Harbourlight Sport | Harbourlight Sport | Harbourlight One |
| scheduling_system | TXLine | TXLine | Airtime Planner |
| transmission_day | Saturday | Saturday | Saturday |
| start_time | 19:00 | 21:15 | 27:30 |
| end_time | 21:15 | 22:00 | 30:00 |
| programme_title | Premier Football Live | Football Tonight | Night Film |
| genre | Football | Football | Film |
| live_flag | Yes | No | No |
| live_sport_flag | Yes | No | No |
| made_for_children_flag | No | No | No |
| premiere_flag | Yes | Yes | No |
| episode_number | | | |
| school_holiday_flag | No | No | No |
| day_type | Weekend | Weekend | Weekend |
| lead_in_genre | Sport | Football | Talk and magazine |
| forecast_under18_index | 104 | 116 | 40 |
| region_feed_code | RF-04 | RF-05 | RF-05 |
| playout_code | PLY-7172 | PLY-1218 | PLY-2759 |

**open_campaign_notes.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| note_id | CN7702018 | CN2388821 | CN9491582 |
| campaign_reference | CLR-QSC-0042117 | CLR-QSC-0042117 | CLR-QSC-0042117 |
| agency_code | QSC | QSC | QSC |
| clock_number | QSC/SPWB017/030 | QSC/SPWB017/030 | QSC/SPWB017/030 |
| author_id | EX-2432 | LG-3123 | AC-5108 |
| author_role | Senior Clearance Executive | Legal Adviser | Agency Contact |
| note_source | Executive first read | Legal review | Agency reply |
| note_sequence | 1 | 2 | 3 |
| note_written_at | Tuesday 10:12 | Tuesday 15:40 | Wednesday 09:25 |
| product_category | Betting and gaming | Betting and gaming | Betting and gaming |
| note_text | Acca Boost: the boosted odds headline has no minimum stake or elig... | Legal: the boosted price must be shown with its significant terms,... | Agency says the 'bet now before kick-off' line is standard for mat... |
| note_length_chars | 113 | 94 | 104 |

**THE DEFECTS IN THE RAW EXTRACTS**

Every defect below is present in the raw extracts or the booked spots. The last column names the constants that repair it; a dash means none is needed.

| Defect | How it looks | Where the repair values live |
|---|---|---|
| Padding and case | **" Fast food ", "Weekday ", "CONSUMER CREDIT", "executive first read", "clearance executive"** | **config.SUBMISSION_VALUE_MAPPINGS**, **config.AUDIENCE_VALUE_MAPPINGS**, **config.NOTE_VALUE_MAPPINGS** |
| Spelling variants and codes | A category in shorthand (**"B&G"**, **"Gambling"**, **"HFSS"**, **"LHF"**, **"Alc"**, **"OTC medicines"**), a channel as its code (**"HL1"**, **"H-One"**, **"HARBOURLIGHT 1"**), a performer band (**"U18"**, **"50+"**), a genre (**"Doc"**, **"Sitcom"**), a note source, a role or an issue in the author's words | **config.PRODUCT_CATEGORY_MAPPING**, **config.SUB_CATEGORY_MAPPING**, **config.SUBMISSION_CHANNEL_MAPPING**, **config.PERFORMER_AGE_BAND_MAPPING**, **config.CHANNEL_MAPPING**, **config.SCHEDULING_SYSTEM_MAPPING**, **config.GENRE_MAPPING**, **config.DAY_TYPE_MAPPING**, **config.AUTHOR_ROLE_MAPPING**, **config.NOTE_SOURCE_MAPPING**, **config.ISSUE_CATEGORY_MAPPING** |
| Weekdays written several ways | **"Sat"**, **"SAT"**, **"Sa."**, **"saturday"**, **"Mo."**, **"Thurs"** | **config.WEEKDAY_MAPPING**, **config.WEEKDAYS** |
| Yes and no written many ways | **"Y"**, **"yes"**, **"TRUE"**, **"1"**, **"N"**, **"no"**, **"FALSE"**, **"0"** | **config.BOOLEAN_MAPPING**, **config.BOOLEAN_YES**, **config.BOOLEAN_NO** |
| Missing written many ways | **"-"**, **"?"**, **"N/A"**, **"NULL"**, **"Not recorded"**, blank | **config.NULL_TOKENS** |
| Duplicated rows | The same record twice, identical in every column | — |
| Two clocks | TXLine writes the broadcast clock on the broadcast day (**"25:10"** on Saturday); Airtime Planner, SlotMaster and Planit write the calendar clock on the calendar day (**"01:10"** on Sunday) | **config.BROADCAST_CLOCK_SYSTEMS**, **config.CALENDAR_CLOCK_SYSTEMS**, **config.CLOCK_TIME_PATTERN** |
| The small hours on the next calendar day | A calendar-clock start before **06:00**, which belongs to the previous weekday's broadcast day, **24** hours later on its clock | **config.PREVIOUS_WEEKDAY**, **config.BROADCAST_DAY_START_MINUTE**, **config.MINUTES_PER_DAY**, **config.MINUTES_PER_HOUR** |
| An end written before its start | A calendar-clock end earlier than its start, such as **23:30** to **00:30**; every calendar-clock end at or before **06:00** is **24** hours later on the broadcast clock, here **24:30** | **config.BROADCAST_DAY_START_MINUTE**, **config.MINUTES_PER_DAY** |
| Spot lengths written several ways | **30"**, **"30s"**, **"30 sec"**, **"0:30"**, **"00:00:30"** | **config.SECONDS_NOTATION_PATTERN**, **config.MINUTES_AND_SECONDS_NOTATION_PATTERN**, **config.HOURS_MINUTES_AND_SECONDS_NOTATION_PATTERN**, **config.SECONDS_PER_MINUTE**, **config.SECONDS_PER_HOUR**, **config.STANDARD_SPOT_LENGTHS** |
| A copy length that disagrees with its clock number | **copy_length_seconds** of **90** on a clock number ending **060** | **config.CLOCK_NUMBER_PATTERN**, **config.CLOCK_NUMBER_COLUMN**, **config.COPY_LENGTH_COLUMN** |
| Audiences written several ways | **"0.2272m"**, **"62.3k"**, **"54.8 thousand"**, **"220,500"** viewers | **config.MILLIONS_NOTATION_PATTERN**, **config.THOUSANDS_NOTATION_PATTERN**, **config.VIEWERS_NOTATION_PATTERN**, **config.DIGIT_GROUPING_SEPARATOR**, **config.THOUSANDS_PER_MILLION**, **config.VIEWERS_PER_THOUSAND**, **config.AUDIENCE_DECIMALS**, **config.AUDIENCE_COLUMNS_IN_THOUSANDS** |
| Numbers written with a unit or a sign | **"24 spots"**, **"2 channels"**, **"45%"**, **"31.0 %"** | **config.SUBMISSION_WHOLE_NUMBER_PATTERNS**, **config.SUBMISSION_DECIMAL_NUMBER_PATTERNS**, **config.AUDIENCE_WHOLE_NUMBER_PATTERNS**, **config.NOTE_WHOLE_NUMBER_PATTERNS**, built from **config.WHOLE_NUMBER_PATTERN**, **config.SPOTS_COUNT_PATTERN**, **config.CHANNELS_COUNT_PATTERN**, **config.WHOLE_PERCENTAGE_PATTERN** and **config.DECIMAL_PERCENTAGE_PATTERN**; **config.PLAIN_NUMBER_PATTERN** is a number written plainly |
| Impossible readings | An index of **0** or over **999**, a negative count or audience, a percentage over **100**, more than four channels, a programme ending after **30:00**, small print on screen longer than the copy, more amendments than campaigns | **config.SUBMISSION_RANGE_LIMITS**, **config.AUDIENCE_RANGE_LIMITS**, **config.LEGAL_TEXT_SECONDS_COLUMN**, **config.ADVERTISER_AMENDMENTS_COLUMN**, **config.ADVERTISER_CAMPAIGNS_COLUMN** |
| Missing answer column | A row with no value in the column the model must learn | — |
| Note times in several notations | **"MONDAY 08:36"**, **"Mo. 09:42"**, **"Thu 08:52"** | **config.NOTE_TIMESTAMP_PATTERN**, **config.NOTE_WEEKDAY_PART**, **config.NOTE_TIME_PART**, **config.WEEKDAY_MAPPING** |
| Markup in free text | ****, **<p>** and **<br/>** around and inside a note | **config.MARKUP_PATTERN** |
| Escaped characters in free text | ** **, **'** and no-break spaces where plain characters belong | **config.ENCODING_ARTIFACT_PATTERN**, **config.NO_BREAK_SPACE**, **config.WHITESPACE_RUN_PATTERN** |
| Notes written in capitals | A whole note shouted, which can be restored only in lower case | — |
| Notes too short to learn from | **"Noted, thanks."** | **config.MINIMUM_NOTE_WORDS** |
| A stale derived column | **note_length_chars** measured on the raw text | **config.NOTE_LENGTH_COLUMN** |

Some gaps are facts, not defects: offer terms on copy that makes no offer, an APR outside consumer credit, an episode number for a film. They stay empty in the cleaned tables.

## THE SIXTEEN DOCUMENTS

Sixteen PDFs under **data/docs/**, the only source of any rule, figure or advice the system states.

| Document | What it settles |
|---|---|
| **Code-Alcohol.pdf** | The alcohol code: its restricted hours or the sentence allowing spots at any time, its under-18 index threshold, its children's rule, its daily limit, its separation, its copy requirements, its standing restrictions and how a withdrawn spot is re-booked |
| **Code-Betting-And-Gaming.pdf** | The same for betting and gaming, the only code with a live sport exception |
| **Code-Less-Healthy-Food.pdf** | The same for less healthy food |
| **Code-Consumer-Credit.pdf** | The same for consumer credit |
| **Code-Medicines-And-Health.pdf** | The same for medicines and health |
| **Advertising-Clearance-Policy.pdf** | The roles at the desk, the separation of review and clearance, what is checked, the three decisions and when each is open, what a restriction notice and a clearance certificate must show, what happens to a rejected campaign, the order campaigns are worked in, and what an Agency Liaison may be told |
| **Scheduling-And-Audience-Rules.pdf** | The broadcast day and the booking week, the broadcast clock, how each channel writes the time, restricted hours on the broadcast day, where a break sits, live sport coverage, programmes made for children, the forecast under-18 index, counting the daily limit and measuring the separation |
| **Past-Clearances-Casebook.pdf** | Eight closed campaigns, one per copy issue, and how each was decided. It quotes figures from the codes in sentences of its own, and it is the one document the chat may not read |
| **Guide-Health-Claims.pdf** | How the desk handles a health claim: how it shows up, what the executive checks, the evidence to ask for, when the copy can be cleared and what the agency should do |
| **Guide-Price-And-Offer-Claims.pdf** | The same, for a price or offer claim |
| **Guide-Environmental-Claims.pdf** | The same, for an environmental claim |
| **Guide-Comparative-Claims.pdf** | The same, for a comparative claim |
| **Guide-Appeal-To-Under-18s.pdf** | The same, for copy that appeals to under-18s |
| **Guide-Gambling-Inducements.pdf** | The same, for a gambling inducement |
| **Guide-Alcohol-And-Social-Success.pdf** | The same, for alcohol linked with social success |
| **Guide-Financial-Promotions.pdf** | The same, for a financial promotion |

Every document opens with **HARBOURLIGHT MEDIA**, its title and one header line matched by **config.DOCUMENT_HEADER_PATTERN**, and is divided into sections numbered from **1**, matched by **config.SECTION_HEADING_PATTERN**.
The five codes share one template and differ in their figures, in whether they restrict hours, in the reach of the children's rule and in the live sport exception only betting has. **config.CATEGORY_CODE_DOCUMENTS** names each category's code, **config.GUIDE_DOCUMENT_BY_ISSUE** the guide for each copy issue, and **config.CHAT_EXCLUDED_DOCUMENTS** the casebook. Each rule is one sentence of a fixed form, read by **config.RESTRICTED_HOURS_PATTERN**, **config.ANY_TIME_PATTERN**, **config.LIVE_SPORT_EXCEPTION_PATTERN**, **config.CHILDREN_RULE_PATTERNS** and **config.CODE_FIGURE_PATTERNS**. The configuration holds no restricted hour, exception, threshold, limit or separation; each is read from its own code at run time.

**DOMAIN KNOWLEDGE CONTEXT**
Every value named here is in **config/config.py**, except the rules of the five codes, which exist only in the codes.

**WHAT ADVERTISING CLEARANCE IS**
Clearance is the broadcaster's check, before a campaign airs, that its copy is acceptable and that every booked spot sits where its category's code allows. Each spot either may air or breaks a named rule.

**THE BROADCAST DAY AND THE BROADCAST CLOCK**
The broadcast day runs from **06:00** to **05:59** the next morning, and every schedule, count and comparison is made on it. The **broadcast clock** writes **06:00** to **23:59** as usual, then **24:00** to **29:59** after midnight; a programme ends at **30:00** at the latest. **config.BROADCAST_DAY_START_MINUTE** and **config.BROADCAST_DAY_END_MINUTE** give the two edges in minutes, **360** and **1800**.
The schedulers in **config.BROADCAST_CLOCK_SYSTEMS** write the broadcast clock on the broadcast day. Those in **config.CALENDAR_CLOCK_SYSTEMS** write the ordinary clock on the calendar day, and their times move onto the broadcast clock by one rule: a start from **06:00** keeps its weekday and clock, as does an end after **06:00**; a start before **06:00** belongs to the previous weekday's broadcast day, **24** hours later, and an end at or before **06:00** is **24** hours later too. **Sunday 05:40** is **Saturday 29:40**, **Saturday 23:10** is unchanged, **23:30** to **00:30** is **23:30** to **24:30**, and **Monday 01:10** is **Sunday 25:10**, the booking week's last broadcast day. **config.PREVIOUS_WEEKDAY** gives each weekday the one before it.
A broadcast-clock time before **06:00**, or a calendar-clock time of **24:00** or later, cannot be trusted. Every cleaned time is **HH:MM** on the broadcast clock beside the weekday of its broadcast day. Nothing is measured from the current date.

### FIVE CATEGORIES, FIVE CODES

| Category | Code | What sets the code apart |
|---|---|---|
| Alcohol | **Code-Alcohol.pdf** | The template's rules, with its own figures |
| Betting and gaming | **Code-Betting-And-Gaming.pdf** | The only code with a live sport exception to its restricted hours |
| Less healthy food | **Code-Less-Healthy-Food.pdf** | The template's rules, with its own figures |
| Consumer credit | **Code-Consumer-Credit.pdf** | The template's rules, with its own figures |
| Medicines and health | **Code-Medicines-And-Health.pdf** | The template's rules, with its own figures |

**config.CATEGORY_CODE_DOCUMENTS** maps each category to its code. A code either names its restricted hours, a span its spots may not be transmitted in, or says its spots may be transmitted at any time of the broadcast day.

### THE RULES ARE NOT IN THE SOLUTION

A code's restricted hours are written on the ordinary clock, the exception in minutes, and the index threshold, daily limit and separation as whole numbers. They are read at run time from the category's own code; no figure is written into the solution or the configuration, and no category's figure is used for another.

A code is read whole, and each rule must have exactly one value: the same sentence met in two passages is one statement. A code is refused when it states two different values for a rule, both restricted hours and the any-time sentence or neither, an exception without restricted hours, restricted hours that begin and end at the same minute, no children's rule or both, or a daily limit of nought.

### WHERE A BREAK SITS: IN AND ADJACENT

Every spot is booked into a break, named by the broadcast time it starts.

- A break strictly inside a programme is a **centre break**; the spot is **in** that programme: **config.BREAK_IN**.
- A break where one programme ends and the next begins is an **end break**; the spot is **adjacent** to both: **config.BREAK_ADJACENT**.
- A break at the very start or end of the broadcast day, or at or inside a gap in the schedule, is refused.

### THE FIVE RULES

Each spot is tested against all five rules of its category's code; a spot that breaks none may air. A spot carries the code of every rule it breaks, in the order of **config.BREACH_CODES**.

| Code | The spot breaks the rule when |
|---|---|
| **config.BREACH_WINDOW** | Its break falls inside the code's restricted hours, start included and end excluded, unless the live sport exception applies. Hours whose start is later than their end run through midnight, so **22:00** to **02:00** covers **22:00** to **26:00**; on the broadcast clock **05:30** to **21:00** covers **06:00** to **21:00** and **29:30** to **30:00**; a break at **21:00** is clear, one at **05:30** is not |
| The live sport exception | Restricted hours do not apply to a centre break inside **live sport coverage** that begins **more than** the code's minutes after the coverage starts and **more than** its minutes before it ends. Each live sport programme is its own coverage; an end break never qualifies |
| **config.BREACH_INDEX** | Any programme it is in or adjacent to has a forecast under-18 audience index **at or above** the code's threshold |
| **config.BREACH_CHILDREN** | A programme made for children is one it is in or adjacent to, under the code's rule **config.CHILDREN_IN_OR_ADJACENT**, or one it is in, under **config.CHILDREN_IN_ONLY** |
| **config.BREACH_FREQUENCY** | It is beyond the code's daily limit: the campaign's spots on one channel in one broadcast day are counted in transmission order, earliest first, and every booked spot counts, whether or not it breaks another rule |
| **config.BREACH_SEPARATION** | It starts **less than** the code's minutes after the campaign's previous spot on the same channel in the same broadcast day, whatever that previous spot's own result |

Transmission order runs through the booking week, Monday first, then by the break's broadcast time. Two spots of one campaign in one break are zero minutes apart and go out in the order of their spot references.

### THE WORKED EXAMPLE

**CLR-QSC-0042117**: Spinwell Bet's 30-second copy **Acca Boost**, booked by Quayside Creative on Harbourlight Sport and Harbourlight One for the booking week's Saturday. Its category is **Betting and gaming**, and the betting code states restricted hours of **05:30** to **21:00**, an exception for centre breaks of live sport coverage more than **10** minutes after its start and more than **10** minutes before its end, an index threshold of **110**, a children's rule that reaches programmes in or adjacent, a daily limit of **7** and a separation of **30** minutes.

| Spot | Channel | As the scheduler wrote it | Broadcast day and time | Break | Result |
|---|---|---|---|---|---|
| **01** | Harbourlight Sport | SAT 18:20 | Saturday 18:20 | in Saturday Scoreline, index 96 | **WINDOW** — not live sport |
| **02** | Harbourlight Sport | SAT 19:05 | Saturday 19:05 | in Premier Football Live, live sport, index 104 | **WINDOW** — 5 minutes into the coverage |
| **03** | Harbourlight Sport | SAT 19:40 | Saturday 19:40 | in Premier Football Live | compliant — exception applies |
| **04** | Harbourlight Sport | SAT 20:20 | Saturday 20:20 | in Premier Football Live | compliant |
| **05** | Harbourlight Sport | SAT 20:35 | Saturday 20:35 | in Premier Football Live | **SEPARATION** — 15 minutes after 04 |
| **06** | Harbourlight Sport | SAT 21:15 | Saturday 21:15 | between Premier Football Live and Football Tonight, index 116 | **INDEX** |
| **07** | Harbourlight Sport | SAT 22:30 | Saturday 22:30 | in Fight Night Preview, index 72 | compliant — seventh spot of the day |
| **08** | Harbourlight Sport | SAT 25:10 | Saturday 25:10 | in Harbour Poker Club, index 58 | **FREQUENCY** — eighth spot of the day |
| **09** | Harbourlight One | Saturday 22:45 | Saturday 22:45 | in Harbour Crime, index 62 | compliant |
| **10** | Harbourlight One | Sunday 05:40 | Saturday 29:40 | in Night Film, index 40 | **WINDOW** |

Four spots comply and six breach, so the decisions open are **CLEAR WITH RESTRICTIONS** and **REJECT**. The breaches by rule are **WINDOW 3**, **INDEX 1**, **CHILDREN 0**, **FREQUENCY 1** and **SEPARATION 1**, and the first transmission is **Saturday 18:20**. On **CLEAR WITH RESTRICTIONS**, spots **03**, **04**, **07** and **09** are cleared and the other six withdrawn.

**THREE DECISIONS, AND THE DECISIONS OPEN**

The Clearance Controller chooses one of the three decisions in **config.DECISIONS** that the spots leave open.

| Decision | Open when | What happens | Status afterwards | Further model calls |
|---|---|---|---|---|
| **config.DECISION_CLEAR** | No spot breaches | Every booked spot is cleared, and the agency is sent a clearance certificate | **config.CAMPAIGN_STATUS_CLEARED** | One:the certificate |
| **config.DECISION_CLEAR_WITH_RESTRICTIONS** | At least one spot breaches and at least one complies | The compliant spots are cleared and the breaching ones withdrawn; the agency is sent a restriction notice, then the certificate | **config.CAMPAIGN_STATUS_CLEARED_WITH_RESTRICTIONS** | Two: the notice, then the certificate |
| **config.DECISION_REJECT** | Always | No spot is cleared and every spot is withdrawn; the agency is sent neither document and may submit the copy again as a new campaign | **config.CAMPAIGN_STATUS_REJECTED** | None |

While it waits, a campaign stands at **config.CAMPAIGN_STATUS_PENDING**; **config.CAMPAIGN_STATUS_BY_DECISION** maps each decision to its status. A decision not open is refused with **config.DECISION_NOT_OPEN_MESSAGE**. **config.LLM_CALLS_BEFORE_DECISION** is the number of language model calls before the gate, and **config.LLM_CALLS_BY_DECISION** the total for each decision. A campaign is assessed once.

**THE EXPOSURE, AND THE ORDER OF THE QUEUE**

The exposure adds up, over a campaign's breaching spots, the forecast under-18 viewers of the programme each spot is in, or, for an end break, the larger forecast of its two programmes. It is in thousands, rounded once at the end to **config.EXPOSURE_DECIMALS** place. A campaign with a breaching spot has an exposure above nought; one with none, nought. The queue puts the largest exposure first, then the earlier first transmission, then the campaign reference in alphabetical order. Positions count from **1**.

**THE EXPOSURE FORECAST**

The exposure model forecasts a programme's viewers under 18, in thousands, from what the schedule knows before it airs: channel, broadcast day, genre, slot, live, live sport, made for children, premiere, episode number, school holidays, day type, lead-in genre and forecast index. The slot is the start in minutes on the broadcast clock, **config.START_MINUTE_COLUMN**, and the length in minutes, **config.DURATION_MINUTES_COLUMN**. **audience_all** is never an input. Every programme of the booking week is forecast once.

**AMENDMENT RISK AND ITS BAND**

The amendment model gives the probability that a campaign's copy will need changes, rounded to **config.AMENDMENT_SCORE_DECIMALS** places and banded on the rounded figure: **config.AMENDMENT_BAND_HIGH** at or above **config.AMENDMENT_BAND_HIGH_THRESHOLD**, **config.AMENDMENT_BAND_MEDIUM** at or above **config.AMENDMENT_BAND_MEDIUM_THRESHOLD**, otherwise **config.AMENDMENT_BAND_LOW**. The score does not say whether the copy breaks a code. A working model's scores for the live queue fall in all three bands, with a standard deviation above **0.05**.

**READING A CLEARANCE NOTE**

The issue model names which of the eight copy issues, the keys of **config.GUIDE_DOCUMENT_BY_ISSUE**, a note describes. A note shorter than **config.MINIMUM_NOTE_WORDS** words, a word being any run of characters between whitespace, is set aside, as in training.
Each readable note is read as the issue with the highest probability and votes for it with the weight **config.NOTE_SOURCE_WEIGHTS** gives its source: a legal review counts twice, every other source once, and an unlisted source is refused. The campaign's issue has the most weighted votes; a tie goes to the tied issue whose probabilities, weighted the same way and summed across the readable notes, are greatest. The **weighted agreement** is that issue's share of the weighted votes, rounded to **config.NOTE_AGREEMENT_DECIMALS** places.

**THE THREE ROLES**

| Role | Reaches | Does not reach |
| --- | --- | --- |
| **config.ROLE_CONTROLLER** | Everything, and is the only role that may record a decision, never on an assessment they opened themselves | — |
| **config.ROLE_EXECUTIVE** | **config.AREA_INTAKE**, **config.AREA_ASSESSMENT**, **config.AREA_AUDIT** and **config.AREA_CHAT** | **config.AREA_DECISION** |
| **config.ROLE_LIAISON** | **config.AREA_CHAT** only | Everything else |

The three roles are **Clearance Controller**, **Clearance Executive** and **Agency Liaison**. The areas are named by **config.ROLE_AREA_ACCESS**, the pages each role is offered by **config.ROLE_PAGE_ACCESS**, and the area a request belongs to is decided from its path by **config.AREA_BY_PATH_START**, except that a request to record a decision, whose path ends in **config.DECISION_PATH_ENDING**, belongs to **config.AREA_DECISION**. Each Agency Liaison is assigned four agencies, listed against their account in **config.SEED_USER_ACCOUNTS**.
The person who opened an assessment may not decide it; another Clearance Controller does, and the refusal is worded **config.OWN_ASSESSMENT_REFUSED_MESSAGE**.

**THE OWNERSHIP LINE**
A campaign belongs to its agency. An Agency Liaison is told only about their own agencies' campaigns and must not learn that another agency's campaign exists.

- An assessed campaign of their own agencies: its status and when it last changed, worded by **config.CHAT_STATUS_TEMPLATE** with the phrase from **config.CHAT_STATUS_PHRASES** and the time in **config.STATUS_CHANGE_TIME_FORMAT**; once decided, also the decision and how many spots it cleared and withdrew, worded by **config.CHAT_DECIDED_TEMPLATE**, and the documents sent, worded by **config.CHAT_DOCUMENTS_SENT**. The status last changed when the decision was recorded or, while waiting, when the assessment was recorded.
- An unassessed campaign of their own agencies: **config.CHAT_NOT_ASSESSED_TEMPLATE** filled with the reference.
- Another agency's campaign, or a reference not in the queue: **config.CHAT_CAMPAIGN_NOT_FOUND_MESSAGE**, identical in both cases.

The roles in **config.CHAT_RECORD_ROLES** hear any campaign's full record: the assessment, its scores, breaching spots, decisions open and recommendation, and once decided the decision, who took it and why, and the spots cleared and withdrawn. An Agency Liaison never hears a score, a note, a breaching spot, an index or a staff member's name; every document the chat may read is answered for anyone.

**A SHORT GLOSSARY**

| Term | Meaning |
|---|---|
| Clearance | The broadcaster's check that a campaign's copy is acceptable and its spots sit where its code allows |
| Campaign reference | The desk's identifier for a campaign, **CLR-AAA-NNNNNNN** |
| Clock number | The copy's identity, whose last three digits are its length in seconds |
| Copy | The advertisement itself |
| Spot | One transmission of the copy, booked into one break |
| Break | A pause in the schedule where spots go out, named by the broadcast time it starts |
| Centre break | A break inside a programme; its spots are in that programme |
| End break | A break between two programmes; its spots are adjacent to both |
| Broadcast day | **06:00** to **05:59** the next morning, the day every count and comparison is made on |
| Broadcast clock | Times on the broadcast day, **06:00** to **29:59** |
| Booking week | The seven broadcast days, Monday to Sunday, the live queue is booked for |
| Restricted hours | The span of the day a code's spots may not be transmitted in |
| Live sport coverage | A programme the schedule marks as live sport, from its start to its end |
| Forecast under-18 index | The audience planning team's forecast of a programme's young audience, **100** being average |
| Daily limit | The most spots one campaign may have on one channel in one broadcast day |
| Separation | The least time between two spots of one campaign on one channel in one broadcast day |
| Breach | A rule a spot breaks, recorded by its code |
| Decisions open | The decisions a campaign's spots allow |
| Exposure | The forecast under-18 viewers each of a campaign's breaching spots would reach, added up spot by spot |
| Amendment score | The probability that a campaign's copy will need changes before it can be cleared |
| Weighted agreement | The share of a campaign's weighted note votes that name its issue |
| Assessment | The write-up and recommendation the workflow produces for the Clearance Controller |
| Restriction notice | The document that tells the agency which spots are withdrawn, under which rule, and what to do next |
| Clearance certificate | The document that lists every cleared spot and the standing restrictions on it |

**TASKS**

For each module the responsibility is stated, and then every function within it is specified by its signature, its parameters, its purpose, the requirements its result must satisfy, and what it returns.
Names matter. Every function name, parameter name, module path and dictionary key given below is read by other parts of the system and by the marking suite, and must be reproduced exactly as written, including case.
Every path, spelling correction, validation limit, threshold, vocabulary, pattern and business rule already exists in **config/config.py**, which is supplied complete.

**========== DATA PREPARATION ==========**

**MODULE: data_pipeline/dataset_loader.py**
The only data preparation module that reads or writes CSV files. Raw files are read without inferring any column's type; cleaned files are read with each column's type inferred.
**FUNCTION SIGNATURE**
`load_raw_dataset(file_path)`
**PARAMETERS**
**file_path** — **str**. The path of one raw extract under **data/**, or of a live-queue file under **data/input/**, passed by a cleaning function, by **run_data_pipeline.py** or by the workflow's nodes module from config, for example **config.RAW_SUBMISSIONS_TRAIN_FILE** or **config.BOOKED_SPOTS_FILE**.
**PURPOSE**
Read one raw file as text, guessing nothing.
**REQUIREMENTS**

- Every column holds text, and no value carries leading or trailing whitespace.
- A value that matches one of **config.NULL_TOKENS**, ignoring case, is a missing value (NaN); a value that merely contains such a token is kept.
- The row count and the column count match the file on disk.

**RETURNS**
**pandas.DataFrame**. Every row of the named file, every column held as text and every missing value as NaN.

**FUNCTION SIGNATURE**
`load_processed_dataset(file_path)`

**PARAMETERS**
**file_path** — **str**. The path of a cleaned table under **processed/**, passed by **run_model_training.py** from config, for example **config.CLEAN_SUBMISSIONS_TRAIN_FILE**.
**PURPOSE**
Read a table the pipeline has already cleaned.
**REQUIREMENTS**
- Column types follow the file's contents, and only an empty cell is read as missing.
- Every row and every column of the file is returned, unfiltered and unsorted.

**RETURNS**
**pandas.DataFrame**. The file as it stands on disk.

**FUNCTION SIGNATURE**
`save_processed_dataset(dataframe, file_path)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The finished table, passed by the cleaning function that produced it.
**file_path** — **str**. The destination under **processed/**, passed on by that cleaning function from its **output_file_path**, for example **config.CLEAN_AUDIENCES_TRAIN_FILE**.
**PURPOSE**
Save a cleaned table.
**REQUIREMENTS**
- The folder the file belongs in exists afterwards, whether or not it existed before.
- The file holds the table's own columns and no added index column.

**RETURNS**
**None**.

**MODULE: data_pipeline/common_cleaning.py**
Repairs shared by the cleaning runs, the profiler and the live-booking tools. Every function takes a table and returns a repaired copy, leaving its input untouched; none reads or writes a file or removes a column. A value in an unrecognised notation is refused with a **ValueError** naming the column and the values.

**FUNCTION SIGNATURE**
`remove_duplicate_records(dataframe)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The table a cleaning run has reached, passed by the calling cleaning function.
**PURPOSE**
Drop records the extract wrote twice.
**REQUIREMENTS**
- Rows identical in every column appear once, the first of them kept; rows that differ anywhere are all kept.
- The result is numbered from nought upwards with no gaps.

**RETURNS**
**pandas.DataFrame**. The table without exact duplicates.

**FUNCTION SIGNATURE**
`normalize_category_values(dataframe, value_mappings)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The table to fold, passed by a cleaning function, by **fetch_campaign_spots** or by **profile_audience_details**.
**value_mappings** — **dict[str, dict]**. Each column against its lower-case mapping, passed from config, for example **config.SUBMISSION_VALUE_MAPPINGS** or **config.BOOKED_SPOT_VALUE_MAPPINGS**.
**PURPOSE**
Fold every spelling, case and code of a category value onto its one canonical form.
**REQUIREMENTS**
- Every value in a mapped column is its mapping's canonical value, whatever case it arrived in; a missing value stays missing.
- A value the mapping does not recognise is refused, naming the column and the values.

**RETURNS**
**pandas.DataFrame**. The table with every mapped column folded.

**FUNCTION SIGNATURE**
`remove_rows_missing_target(dataframe, target_column)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The table reached so far, passed by a cleaning function.
**target_column** — **str**. The answer column, passed from config, for example **config.SUBMISSION_TARGET_COLUMN**.

**PURPOSE**
Remove the rows no model can learn from.
**REQUIREMENTS**
- Exactly the rows with no answer are removed, and the result is numbered from nought upwards with no gaps.

**RETURNS**
**pandas.DataFrame**. The rows that carry an answer.

**FUNCTION SIGNATURE**
```
convert_whole_numbers(dataframe, number_patterns)
```
**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The table to convert, passed by a cleaning function, a profiler or a campaign data tool.
**number_patterns** — **dict[str, str]**. Each column against the pattern that captures its number, passed from config, for example **config.SUBMISSION_WHOLE_NUMBER_PATTERNS**.
**PURPOSE**
Read counts and whole percentages written with a unit or a sign as whole numbers.
**REQUIREMENTS**
- Each named column holds whole numbers, read through its pattern, with gaps kept as gaps.
- A value its pattern does not read, a fraction included, is refused, naming the column and the values.

**RETURNS**
**pandas.DataFrame**. The table with each named column as whole numbers.

**FUNCTION SIGNATURE**
```
convert_decimal_numbers(dataframe)
```
**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The submissions table, passed by **clean_submission_dataset** or **profile_submission_details**.
**PURPOSE**
Read the decimal percentages of the submissions extract as numbers.
**REQUIREMENTS**
- Each column of **config.SUBMISSION_DECIMAL_NUMBER_PATTERNS** holds a decimal number read through its pattern, with gaps kept as gaps.
- A value the pattern does not read is refused, naming the column and the values.

**RETURNS**
**pandas.DataFrame**. The table with those columns as decimals.

**FUNCTION SIGNATURE**
```
convert_spot_lengths_to_seconds(dataframe, length_columns)
```
**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The table to convert, passed by **clean_submission_dataset**, **profile_submission_details** or **fetch_campaign_spots**.
**length_columns** — **list[str]**. The columns that hold a length, passed from config, for example **[config.COPY_LENGTH_COLUMN]** or **[config.SPOT_LENGTH_COLUMN]**.
**PURPOSE**
Turn a length written in seconds, in minutes and seconds, or in hours, minutes and seconds into whole seconds.
**REQUIREMENTS**
- A length matched by **config.SECONDS_NOTATION_PATTERN**, **config.MINUTES_AND_SECONDS_NOTATION_PATTERN** or **config.HOURS_MINUTES_AND_SECONDS_NOTATION_PATTERN** becomes its whole number of seconds; a gap stays a gap.
- A length in none of those notations is refused, naming the column and the values.

**RETURNS**
**pandas.DataFrame**. The table with each named column in whole seconds.

**FUNCTION SIGNATURE**
```
convert_audiences_to_thousands(dataframe)
```
**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The audiences table, passed by **clean_audience_dataset** or **profile_audience_details**.
**PURPOSE**
Put every audience on one scale, thousands of viewers.
**REQUIREMENTS**
- Each column of **config.AUDIENCE_COLUMNS_IN_THOUSANDS** is in thousands, rounded to **config.AUDIENCE_DECIMALS** place: a value matched by **config.MILLIONS_NOTATION_PATTERN**, **config.THOUSANDS_NOTATION_PATTERN** or **config.VIEWERS_NOTATION_PATTERN** is converted from millions, thousands or viewers.
- An audience none of those patterns matches is refused, naming the column and the values.

**RETURNS**
**pandas.DataFrame**. The table with both audience columns in thousands.

**FUNCTION SIGNATURE**
```
convert_clock_times_to_minutes(dataframe, time_columns)
```

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. A table holding times written **HH:MM**, passed by **convert_to_broadcast_clock**, **add_slot_minutes**, **profile_audience_details**, **fetch_campaign_schedules** or **check_campaign_spots**.
**time_columns** — **list[str]**. The columns that hold a time, passed by the same caller, for example **[config.START_TIME_COLUMN, config.END_TIME_COLUMN]**.
**PURPOSE**
Read a clock time as minutes after midnight.
**REQUIREMENTS**

- A time matched by **config.CLOCK_TIME_PATTERN** becomes its minutes, hours past **23** included, so **25:10** is **1510**; a gap stays a gap.
- A time the pattern does not read is refused, naming the column and the values.

**RETURNS**
**pandas.DataFrame**. The table with each named column in minutes.

**FUNCTION SIGNATURE**
`convert_to_broadcast_clock(dataframe, start_column, end_columns)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. A table whose day and scheduling system are already canonical, passed by **clean_audience_dataset**, **profile_audience_details** or **fetch_campaign_spots**.
**start_column** — **str**. The column of times that decide the broadcast day, passed from config: **config.START_TIME_COLUMN** for a programme, **config.BREAK_TIME_COLUMN** for a booked spot.
**end_columns** — **list[str]**. The columns of times that end what the start begins, passed from config: **[config.END_TIME_COLUMN]** for a programme, an empty list for a spot.
**PURPOSE**
Place every time on the broadcast day and the broadcast clock, whichever clock its scheduler wrote.
**REQUIREMENTS**

- **transmission_day** becomes the weekday of the broadcast day, and every named time becomes its minutes on the broadcast clock, under the rules in **THE BROADCAST DAY AND THE BROADCAST CLOCK**.
- A scheduling system in neither **config.BROADCAST_CLOCK_SYSTEMS** nor **config.CALENDAR_CLOCK_SYSTEMS**, a missing system, a missing start time, a day that is not a weekday, a calendar-clock time of **24:00** or later, a broadcast-clock start outside the broadcast day, or an end that is not after its start, is refused.

**RETURNS**
**pandas.DataFrame**. The table with the broadcast day and every named time in minutes on the broadcast clock.

**FUNCTION SIGNATURE**
`format_broadcast_clock(dataframe, time_columns)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. A table whose times are minutes on the broadcast clock, passed by **clean_audience_dataset** or **fetch_campaign_spots**.
**time_columns** — **list[str]**. The columns to write out, passed from config, for example **[config.START_TIME_COLUMN, config.END_TIME_COLUMN]**.
**PURPOSE**
Write minutes on the broadcast clock back as broadcast-clock times.
**REQUIREMENTS**

- Each value is written **HH:MM**, hours and minutes each in **config.CLOCK_TIME_DIGITS** digits, so **1510** is **25:10** and **1800** is **30:00**; a gap stays a gap.

**RETURNS**
**pandas.DataFrame**. The table with each named column written **HH:MM**.

**FUNCTION SIGNATURE**
`convert_markup_to_plain_text(dataframe)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The notes table, passed by **clean_note_dataset** or **profile_note_details**.
**PURPOSE**
Turn note text into plain text.
**REQUIREMENTS**

- **config.NOTE_TEXT_COLUMN** holds no markup matched by **config.MARKUP_PATTERN**; escaped characters are the characters they stand for and no-break spaces are ordinary spaces; words a tag separated stay separate, whitespace runs are single spaces, and nothing leads or trails.

**RETURNS**
**pandas.DataFrame**. The table with plain note text.

**FUNCTION SIGNATURE**
`treat_out_of_range_as_missing(dataframe, range_limits)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. A table whose named columns are already numbers, passed by a cleaning function.
**range_limits** — **dict[str, tuple]**. Each column against its lowest and highest possible value, both included, passed from config, for example **config.AUDIENCE_RANGE_LIMITS**.

**PURPOSE**
Blank readings that cannot be true.
**REQUIREMENTS**
- A value outside its column's limits becomes missing; every other value, and every row, is kept.

**RETURNS**
**pandas.DataFrame**. The table with impossible readings blanked.

**MODULE: data_pipeline/submission_cleaning.py**
The cleaning run for the campaign submission extracts, including the two contradictions only a submission can hold.

**FUNCTION SIGNATURE**
`clean_submission_dataset(input_file_path, output_file_path)`

**PARAMETERS**
**input_file_path** — **str**. A raw submissions extract, passed by **run_data_pipeline.py** from **config.RAW_SUBMISSIONS_TRAIN_FILE** or **config.RAW_SUBMISSIONS_EVAL_FILE**.
**output_file_path** — **str**. Where the cleaned table is written, passed by the same caller from **config.CLEAN_SUBMISSIONS_TRAIN_FILE** or **config.CLEAN_SUBMISSIONS_EVAL_FILE**.
**PURPOSE**
Produce a submissions table a model can be fitted on.
**REQUIREMENTS**
- Duplicates and rows with no answer are removed, every category column holds its canonical vocabulary, every number is read under the patterns of config, and a reading outside **config.SUBMISSION_RANGE_LIMITS** becomes missing.
- **copy_length_seconds** is whole seconds; where it disagrees with the length in the clock number's last three digits, the clock number's length wins. A clock number that does not end in one of **config.STANDARD_SPOT_LENGTHS** is refused.
- **legal_text_seconds** longer than the copy, and **advertiser_amendments_past_year** greater than **advertiser_campaigns_past_year**, become missing.
- The table is saved to **output_file_path** with the raw file's columns, in the raw file's order.

**RETURNS**
**pandas.DataFrame**. The cleaned submissions table, as saved.

**MODULE: data_pipeline/audience_cleaning.py**
The cleaning run for the programme audience extracts, in which every time is moved onto the broadcast day and the broadcast clock.

**FUNCTION SIGNATURE**
`clean_audience_dataset(input_file_path, output_file_path)`

**PARAMETERS**
**input_file_path** — **str**. A raw audiences extract, passed by **run_data_pipeline.py** from **config.RAW_AUDIENCES_TRAIN_FILE** or **config.RAW_AUDIENCES_EVAL_FILE**.
**output_file_path** — **str**. Where the cleaned table is written, passed by the same caller from **config.CLEAN_AUDIENCES_TRAIN_FILE** or **config.CLEAN_AUDIENCES_EVAL_FILE**.
**PURPOSE**
Produce an audiences table a model can be fitted on, with every programme on its broadcast day.
**REQUIREMENTS**
- Duplicates are removed and every category column holds its canonical vocabulary.
- **transmission_day** is the broadcast day and **start_time** and **end_time** are written **HH:MM** on the broadcast clock; a time that cannot be placed for certain is refused.
- Both audiences are in thousands and the whole numbers are read; a reading outside **config.AUDIENCE_RANGE_LIMITS** becomes missing, and a row left without an answer is removed.
- The table is saved to **output_file_path** with the raw file's columns, in the raw file's order.

**RETURNS**
**pandas.DataFrame**. The cleaned audiences table, as saved.

**MODULE: data_pipeline/note_cleaning.py**
The cleaning run for the free-text clearance notes.

**FUNCTION SIGNATURE**
`clean_note_dataset(input_file_path, output_file_path)`

**PARAMETERS**
**input_file_path** — **str**. The raw notes extract, passed by **run_data_pipeline.py** from **config.RAW_CLEARANCE_NOTES_FILE**.
**output_file_path** — **str**. Where the cleaned table is written, passed by the same caller from **config.CLEAN_CLEARANCE_NOTES_FILE**.
**PURPOSE**
Produce a notes table the issue model can learn from.
**REQUIREMENTS**

- Duplicates and rows with no answer are removed, every category column holds its canonical vocabulary, and **note_sequence** is a whole number.
- **note_written_at** is the weekday in full, a space and the time, read through **config.NOTE_TIMESTAMP_PATTERN**; a timestamp it cannot read is refused.
- **note_text** is plain text, and a note whose plain text is wholly in capitals is restored in lower case. A note of fewer than **config.MINIMUM_NOTE_WORDS** words is removed, and **note_length_chars** is the length of the text as cleaned.
- The table is saved to **output_file_path** with the raw file's columns, in the raw file's order.

**RETURNS**
**pandas.DataFrame**. The cleaned notes table, as saved.

**MODULE: data_pipeline/feature_builder.py**
Turns a cleaned table into model inputs, and builds the preparation saved with a model and applied alike in fitting and scoring.

**FUNCTION SIGNATURE**
`add_slot_minutes(dataframe)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. A table of programmes whose **start_time** and **end_time** are on the broadcast clock, passed by **run_model_training.py** with a cleaned audiences table or by **forecast_programme_audiences** with the booking week's schedule.
**PURPOSE**
Add the programme slot as numbers.
**REQUIREMENTS**

- **config.START_MINUTE_COLUMN** holds each programme's start in minutes on the broadcast clock, and **config.DURATION_MINUTES_COLUMN** its end less its start.
- Every other column is kept as it was.

**RETURNS**
**pandas.DataFrame**. The table with the two slot columns added.

**FUNCTION SIGNATURE**
`build_feature_frame(dataframe, numeric_columns, categorical_columns)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. A cleaned table, the live campaigns or the booking week's schedule, passed by **run_model_training.py**, **build_amendment_facts** or **forecast_programme_audiences**.
**numeric_columns** — **list[str]**. The numeric inputs, passed from config, for example **config.AMENDMENT_NUMERIC_FEATURES**.
**categorical_columns** — **list[str]**. The category inputs, passed from config, for example **config.AMENDMENT_CATEGORICAL_FEATURES**.
**PURPOSE**
Select and type a model's inputs.
**REQUIREMENTS**

- The result holds the numeric columns, then the categorical columns, in the order given, and no other column.
- Numeric columns are floating-point, whether they arrived as text or as numbers, and a gap is NaN; categorical columns hold text, and a gap is a missing value.

**RETURNS**
**pandas.DataFrame**. The model's inputs, one row per row of the table.

**FUNCTION SIGNATURE**
`build_model_pipeline(estimator, numeric_columns, categorical_columns)`

**PARAMETERS**
**estimator** — **sklearn.base.BaseEstimator**. An unfitted estimator, passed by **train_amendment_model** or **train_exposure_model**.
**numeric_columns** — **list[str]**. The numeric inputs, passed on by the same caller from config.
**categorical_columns** — **list[str]**. The category inputs, passed on by the same caller from config.
**PURPOSE**
Join the input preparation and the estimator, so the preparation is saved with the model.
**REQUIREMENTS**

- Every preparation the inputs need is learned inside the pipeline, from the rows it is fitted on.
- A row with gaps, or with a category value never seen in fitting, is scored without error.

**RETURNS**
**sklearn.pipeline.Pipeline**. The preparation followed by the estimator, unfitted.

**MODULE: data_pipeline/quality_profiler.py**
Measures how dirty each raw extract was and how clean its cleaned table is, and writes that evidence to **output/data_quality_report.json**.

**FUNCTION SIGNATURE**
`profile_dataset(raw_frame, clean_frame, target_column)`

**PARAMETERS**
**raw_frame** — **pandas.DataFrame**. The extract as **load_raw_dataset** read it, passed by **run_data_pipeline.py**.

**clean_frame** — **pandas.DataFrame**. The same extract after cleaning, passed by the same caller.
**target_column** — **str**. The extract's answer column, passed from config, for example **config.NOTE_TARGET_COLUMN**.
**PURPOSE**
Account for every row an extract arrived with.
**REQUIREMENTS**

- **rows_received**, **rows_kept** and **rows_removed** count the raw rows, the cleaned rows and the difference.
- **duplicate_rows** counts the repeated copies of earlier rows, and **rows_missing_target** the first copies that carry no answer.
- **columns** is the cleaned table's column count, and **missing_cells_as_received** and **missing_cells_after_cleaning** give, for every column with at least one gap, its count of gaps.

**RETURNS**
**dict**. The eight keys above.

**FUNCTION SIGNATURE**
`profile_submission_details(raw_frame, clean_frame)`

**PARAMETERS**
**raw_frame** — **pandas.DataFrame**. A submissions extract as read, passed by **run_data_pipeline.py**.
**clean_frame** — **pandas.DataFrame**. The same extract after cleaning, passed by the same caller.
**PURPOSE**
Record the defects peculiar to the submissions.
**REQUIREMENTS**

- **copy_lengths_not_written_in_seconds** counts the lengths that **config.WHOLE_NUMBER_PATTERN** does not match in full, and **copy_lengths_disagreeing_with_clock_number** the lengths that disagree with their clock number, as received.
- **numbers_written_with_a_unit_or_sign** and **out_of_range_by_column** give, for every whole and decimal number column of config, the values **config.PLAIN_NUMBER_PATTERN** does not match in full and the readings outside its limits.
- **legal_text_longer_than_copy_as_received**, **legal_text_longer_than_copy_after_cleaning**, **amendments_exceeding_campaigns_as_received** and **amendments_exceeding_campaigns_after_cleaning** count the two contradictions before and after cleaning, a copy's length as received being its clock number's.
- **target_shares** gives each answer's share of the cleaned rows to **config.TARGET_SHARE_DECIMALS** places.

**RETURNS**
**dict**. The nine keys above.

**FUNCTION SIGNATURE**
`profile_audience_details(raw_frame, clean_frame)`

**PARAMETERS**
**raw_frame** — **pandas.DataFrame**. An audiences extract as read, passed by **run_data_pipeline.py**.
**clean_frame** — **pandas.DataFrame**. The same extract after cleaning, passed by the same caller.
**PURPOSE**
Record the defects peculiar to the audiences.
**REQUIREMENTS**

- **times_written_on_the_calendar_clock**, **small_hours_starts_moved_to_the_previous_weekday** and **ends_written_before_their_starts** count those calendar-clock rows as received, an end before its start being one earlier than the start as written; **weekdays_not_written_in_full** counts the days not written exactly as in **config.WEEKDAYS**.
- **audiences_not_written_in_thousands** and **out_of_range_by_column** give each audience column's values **config.PLAIN_NUMBER_PATTERN** does not match in full and each limited column's impossible readings; **rows_with_impossible_target** counts the first copies whose answer is impossible.
- **median_under18_viewers** is the cleaned answer's median.

**RETURNS**
**dict**. The eight keys above.

**FUNCTION SIGNATURE**
`profile_note_details(raw_frame, clean_frame)`

**PARAMETERS**
**raw_frame** — **pandas.DataFrame**. The notes extract as read, passed by **run_data_pipeline.py**.
**clean_frame** — **pandas.DataFrame**. The same extract after cleaning, passed by the same caller.
**PURPOSE**
Record the defects peculiar to free text.
**REQUIREMENTS**

- **weekdays_not_written_in_full** counts the timestamps whose weekday is not written exactly as in **config.WEEKDAYS**, **notes_with_markup** and **notes_with_encoding_artifacts** the notes carrying them as received, and **notes_in_capitals** the notes whose plain text is wholly in capitals.
- **note_lengths_disagreeing_with_plain_text_as_received** and **note_lengths_disagreeing_with_plain_text_after_cleaning** count the stale lengths before and after cleaning, and **rows_with_too_few_words** the first copies with an answer that are too short to learn from.
- **shortest_note_words** is the fewest words a cleaned note holds, and **target_shares** gives each issue's share of the cleaned rows to **config.TARGET_SHARE_DECIMALS** places.

**RETURNS**
**dict**. The nine keys above.

**FUNCTION SIGNATURE**
`write_quality_report(profiles)`
**PARAMETERS**
**profiles** — **dict[str, dict]**. Each extract's name, the file name without its extension, against its general and detail profiles merged, passed by **run_data_pipeline.py**.
**PURPOSE**
Write the quality report.
**REQUIREMENTS**
- **config.DATA_QUALITY_REPORT_FILE** holds the profiles as indented JSON with sorted keys, its folder created if needed.

**RETURNS**
**None**.

**========== MODELLING ==========**
Three models are trained, scored on held-out data and saved with everything needed to score one row. The estimator and its tuning are the team's choice; the interface, the scores reported and each model's minimum are fixed.

**MODULE: modeling/model_registry.py**
Saves and loads the models, their feature schemas and their scores. Nothing else in the project writes to **output/models/**.
**FUNCTION SIGNATURE**
`save_model(model, file_path)`
**PARAMETERS**
**model** — **sklearn.pipeline.Pipeline**. A fitted model, passed by **run_model_training.py**.
**file_path** — **str**. Where it is written, passed from config: **config.AMENDMENT_MODEL_FILE**, **config.EXPOSURE_MODEL_FILE** or **config.ISSUE_MODEL_FILE**.
**PURPOSE**
Save a fitted model, preparation included.
**REQUIREMENTS**
- The folder exists afterwards, and **load_model** on the same path gives back a model that scores exactly as the one saved.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`save_feature_schema(feature_names, file_path)`
**PARAMETERS**
**feature_names** — **list[str]**. The columns of the inputs the model was fitted on, in order, passed by **run_model_training.py**.
**file_path** — **str**. Where they are written, passed from config, for example **config.EXPOSURE_SCHEMA_FILE**.
**PURPOSE**
Record the exact inputs a model expects.
**REQUIREMENTS**
- The file is a JSON list of the column names in the order given, its folder created if needed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`save_metrics(metrics)`
**PARAMETERS**
**metrics** — **dict[str, dict]**. Each model's name against its record, as described under **run_model_training.py**, passed by **run_model_training.py**.
**PURPOSE**
Save every model's scores.
**REQUIREMENTS**
- **config.MODEL_METRICS_FILE** holds the metrics as indented JSON with sorted keys, its folder created if needed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`load_model(file_path)`
**PARAMETERS**
**file_path** — **str**. A saved model, passed by the workflow's nodes module from config, for example **config.ISSUE_MODEL_FILE**.
**PURPOSE**
Bring a saved model back into memory.
**REQUIREMENTS**
- The model is returned exactly as saved, ready to score.

**RETURNS**
**sklearn.pipeline.Pipeline**. The fitted model.

**FUNCTION SIGNATURE**

```
load_feature_schema(file_path)
```

**PARAMETERS**

**file_path** — **str**. A saved schema, passed by the workflow's nodes module from config, for example **config.AMENDMENT_SCHEMA_FILE**.

**PURPOSE**

Bring a model's input columns back into memory, in order.

**REQUIREMENTS**

- The names are returned in the order they were saved.

**RETURNS**

**list[str]**. The column names.

**MODULE: modeling/model_evaluation.py**

Scores a fitted model on held-out rows and on its training rows. No function here fits anything, and every score is a **float** rounded to **config.METRIC_DECIMALS** places.

**FUNCTION SIGNATURE**

```
evaluate_classifier(model, train_features, train_target, held_out_features, held_out_target)
```

**PARAMETERS**

**model** — **sklearn.pipeline.Pipeline**. The fitted amendment pipeline, whose classes are **0** and **1**, passed by **train_amendment_model** once it has fitted it.

**train_features** — **pandas.DataFrame**. The inputs the model was fitted on, passed on by **train_amendment_model**.

**train_target** — **pandas.Series**. The answers for those rows as integers, encoded by **config.SUBMISSION_TARGET_ENCODING**, passed on by **train_amendment_model**.

**held_out_features** — **pandas.DataFrame**. The held-out inputs the model never saw, passed on by **train_amendment_model**.

**held_out_target** — **pandas.Series**. The answers for those rows, encoded the same way, passed on by **train_amendment_model**.

**PURPOSE**

Score the amendment model.

**REQUIREMENTS**

- **held_out_roc_auc** and **train_roc_auc** are measured from the probability of the positive class, and **generalisation_gap** is the training score less the held-out score, both taken before rounding.
- **accuracy**, **precision**, **recall** and **f1** describe the held-out predictions, and **mean_predicted_probability** and **positive_share** the held-out rows.

**RETURNS**

**dict**. The nine scores above.

**FUNCTION SIGNATURE**

```
evaluate_regressor(model, train_features, train_target, held_out_features, held_out_target)
```

**PARAMETERS**

**model** — **sklearn.pipeline.Pipeline**. The fitted exposure pipeline, passed by **train_exposure_model** once it has fitted it.

**train_features** — **pandas.DataFrame**. The inputs the model was fitted on, passed on by **train_exposure_model**.

**train_target** — **pandas.Series**. The under-18 viewers for those rows, in thousands, passed on by **train_exposure_model**.

**held_out_features** — **pandas.DataFrame**. The held-out inputs the model never saw, passed on by **train_exposure_model**.

**held_out_target** — **pandas.Series**. The under-18 viewers for those rows, passed on by **train_exposure_model**.

**PURPOSE**

Score the exposure model and the always-the-mean baseline.

**REQUIREMENTS**

- **held_out_r2** and **train_r2** are the R squared on each set of rows, and **generalisation_gap** is the training score less the held-out score, both taken before rounding.
- **mean_absolute_error** is the model's average miss on the held-out rows, and **average_estimate_mean_absolute_error** the average miss there of always answering the mean of **train_target**; **lowest_prediction** is its lowest forecast there.

**RETURNS**

**dict**. The six scores above.

**FUNCTION SIGNATURE**

```
evaluate_text_classifier(model, train_text, train_target, held_out_text, held_out_target)
```

**PARAMETERS**

**model** — **sklearn.pipeline.Pipeline**. The fitted issue pipeline, passed by **train_issue_model** once it has fitted it.

**train_text** — **pandas.Series**. The note texts it was fitted on, passed on by **train_issue_model**.

**train_target** — **pandas.Series**. Their issues as text, passed on by **train_issue_model**.

**held_out_text** — **pandas.Series**. Note texts it never saw, passed on by **train_issue_model**.

**held_out_target** — **pandas.Series**. Their issues, passed on by **train_issue_model**.

**PURPOSE**

Score the issue model across every issue.

**REQUIREMENTS**

- **held_out_macro_f1** and **train_macro_f1** are the macro-averaged F1 on each set of notes, and **generalisation_gap** is the training score less the held-out score, both taken before rounding.

- **accuracy** and **weighted_f1** describe the held-out predictions, and **per_class_f1** is a **dict[str, float]** giving each of the model's classes its own held-out score.

**RETURNS**
**dict**. The six entries above.

**MODULE: modeling/amendment_model.py**
The model that predicts whether a campaign's copy will need changes before it can be cleared.

**FUNCTION SIGNATURE**
`train_amendment_model(train_features, train_target, held_out_features, held_out_target)`

**PARAMETERS**
**train_features** — **pandas.DataFrame**. The cleaned training submissions as **build_feature_frame** built them from **config.AMENDMENT_NUMERIC_FEATURES** and **config.AMENDMENT_CATEGORICAL_FEATURES**, passed by **run_model_training.py**.
**train_target** — **pandas.Series**. **amendment_required** for those submissions as integers, encoded by **config.SUBMISSION_TARGET_ENCODING**, passed by **run_model_training.py**.
**held_out_features** — **pandas.DataFrame**. The cleaned held-out submissions, built the same way, passed by **run_model_training.py**.
**held_out_target** — **pandas.Series**. **amendment_required** for those submissions, encoded the same way, passed by **run_model_training.py**.
**PURPOSE**
Predict the probability that a campaign's copy needs changes.
**REQUIREMENTS**
- The model is a pipeline from **build_model_pipeline**, fitted on the training rows only; it records the names of the columns it was fitted on, gives probabilities, and its classes are **0** and **1** in that order.
- Its held-out ROC AUC reaches **config.MINIMUM_AMENDMENT_ROC_AUC**; across the held-out copy its probabilities spread with a standard deviation above **0.05**, and their mean is within **0.05** of the share of that copy that needed changes.
- On average it scores copy that makes a health claim above the same copy without one, and an offer whose terms are left off screen above the same offer with its terms shown.
- Its scores come from **evaluate_classifier**.

**RETURNS**
**tuple[sklearn.pipeline.Pipeline, dict]**. The fitted model and its scores.

**MODULE: modeling/exposure_model.py**
The model that forecasts how many viewers under 18 a programme will have, in thousands.

**FUNCTION SIGNATURE**
`train_exposure_model(train_features, train_target, held_out_features, held_out_target)`

**PARAMETERS**
**train_features** — **pandas.DataFrame**. The cleaned training audiences, with their slot minutes from **add_slot_minutes**, as **build_feature_frame** built them from **config.EXPOSURE_NUMERIC_FEATURES** and **config.EXPOSURE_CATEGORICAL_FEATURES**, passed by **run_model_training.py**.
**train_target** — **pandas.Series**. **under18_viewers** for those programmes, in thousands, passed by **run_model_training.py**.
**held_out_features** — **pandas.DataFrame**. The cleaned held-out audiences, built the same way, passed by **run_model_training.py**.
**held_out_target** — **pandas.Series**. **under18_viewers** for those programmes, passed by **run_model_training.py**.
**PURPOSE**
Forecast a programme's under-18 audience.
**REQUIREMENTS**
- The model is a pipeline from **build_model_pipeline**, fitted on the training rows only, and it records the names of the columns it was fitted on.
- Its held-out R squared reaches **config.MINIMUM_EXPOSURE_R2**, its average miss is smaller than that of always answering the training average, and its forecasts spread across the held-out programmes with a standard deviation above **5** thousand, and every forecast it gives is above nought.
- On average it forecasts more young viewers for the same programmes in school holidays than in term time, at **19:00** to **20:00** than at **25:00** to **26:00** on the broadcast clock, and with a forecast index fifty points higher.
- Its scores come from **evaluate_regressor**.

**RETURNS**
**tuple[sklearn.pipeline.Pipeline, dict]**. The fitted model and its scores.

**MODULE: modeling/issue_model.py**
The NLP model that reads a clearance note and names the copy issue it describes.

**FUNCTION SIGNATURE**
`train_issue_model(train_text, train_target, held_out_text, held_out_target)`

**PARAMETERS**

**train_text** — **pandas.Series**. The training share of the cleaned notes' **note_text**, from the split **run_model_training.py** makes.
**train_target** — **pandas.Series**. Their **issue_category**, from the same split.
**held_out_text** — **pandas.Series**. The held-out share of the cleaned notes' **note_text**, from the same split.
**held_out_target** — **pandas.Series**. Their **issue_category**, from the same split.
**PURPOSE**
Name the copy issue a note describes.
**REQUIREMENTS**

- The model is one pipeline that takes raw note texts, as a list or a Series of strings, fitted on the training notes only, and it gives a probability for every issue.
- Its classes are the eight issues of **config.GUIDE_DOCUMENT_BY_ISSUE**, no more and no fewer, and any text, however strange, is read as one of them.
- Its held-out macro F1 reaches **config.MINIMUM_ISSUE_MACRO_F1**, every issue is predicted for some held-out note, and the notes about each issue are most often read as that issue.
- Its scores come from **evaluate_text_classifier**.

**RETURNS**
**tuple[sklearn.pipeline.Pipeline, dict]**. The fitted model and its scores.

**============== THE KNOWLEDGE BASE ==============**
The sixteen documents are read once, cut into passages and stored for search by meaning. Every later lookup goes through that store, and a category's rules are read from its own code alone.

**MODULE: knowledge/pdf_extraction.py**
Gets the text out of the documents.
**FUNCTION SIGNATURE**
`extract_text_from_directory(docs_directory)`

**PARAMETERS**
**docs_directory** — **str**. The folder that holds the documents, passed by **run_knowledge_indexing.py** from **config.DOCS_DIRECTORY**.
**PURPOSE**
Read every PDF in the folder.
**REQUIREMENTS**

- Every PDF in the folder gives one entry, in file-name order, holding **document_name**, the file name, and **document_text**, the text of all its pages joined by line breaks; a file that is not a PDF is skipped.
- A document with a page from which no text can be extracted is refused, naming the file.

**RETURNS**
**list[dict]**. One entry per document.

**MODULE: knowledge/chunk_splitting.py**
Cuts the documents into numbered sections, and the sections into passages, never cutting a sentence in two.

**FUNCTION SIGNATURE**
`split_documents_into_sections(document_entries)`

**PARAMETERS**
**document_entries** — **list[dict]**. The documents, each holding **document_name** and **document_text**, as **extract_text_from_directory** returned them, passed by **run_knowledge_indexing.py**.
**PURPOSE**
Divide each document at its section headings and drop its header line.
**REQUIREMENTS**

- Each document carries exactly one header line matched by **config.DOCUMENT_HEADER_PATTERN**, which is removed, and sections matched by **config.SECTION_HEADING_PATTERN** numbered **1**, **2**, **3** and on without a gap; a document that does not is refused, naming it.
- Each section becomes one entry holding **document_name** and **section_text**, the text before the second heading belonging to the first section; control characters matched by **config.CONTROL_CHARACTER_PATTERN** give way to spaces, and every run of whitespace becomes one space.

**RETURNS**
**list[dict]**. One entry per section, documents and sections in their own order.

**FUNCTION SIGNATURE**
`build_chunk_records(section_entries)`

**PARAMETERS**
**section_entries** — **list[dict]**. The sections, as **split_documents_into_sections** returned them, passed by **run_knowledge_indexing.py**.
**PURPOSE**
Cut each section into passages of about **config.CHUNK_SIZE_CHARACTERS** characters that overlap their neighbours.
**REQUIREMENTS**

- A section is cut only between sentences, at the boundaries **config.SENTENCE_BOUNDARY_PATTERN** marks, so every sentence lies whole within a passage; a passage never crosses from one section into the next.
- A new passage starts once adding the next sentence would take the current one past **config.CHUNK_SIZE_CHARACTERS**, and it begins with as many of the previous passage's last whole sentences as fit within **config.CHUNK_OVERLAP_CHARACTERS**.
- Each passage holds **chunk_id**, written as the document name, a hyphen and the passage's number within that document counted from **0**, **chunk_text** and **document_name**.

**RETURNS**
**list[dict]**. Every passage of every section, in order.

**MODULE: knowledge/vector_index.py**
Turns passages into vectors, stores them in a persistent Chroma collection, reads a whole document back, and searches by meaning.

**FUNCTION SIGNATURE**
`build_vector_index(chunk_records, store_path)`

**PARAMETERS**
**chunk_records** — **list[dict]**. The passages, as **build_chunk_records** returned them, passed by **run_knowledge_indexing.py**.
**store_path** — **str**. The folder the store lives in, passed by **run_knowledge_indexing.py** from **config.VECTOR_STORE_DIRECTORY**.
**PURPOSE**
Build the searchable knowledge base from scratch.
**REQUIREMENTS**

- Any store already at **store_path** is replaced, so building twice never holds a passage twice.
- The collection is named **config.VECTOR_COLLECTION_NAME**; passages are embedded with **config.EMBEDDING_MODEL_NAME** on **config.EMBEDDING_DEVICE**, and closeness is measured as cosine distance.
- Each passage is stored under its **chunk_id**, with its text, and with its **document_name** as metadata.

**RETURNS**
**int**. The number of passages stored.

**FUNCTION SIGNATURE**
`open_vector_index(store_path)`

**PARAMETERS**
**store_path** — **str**. The folder the store lives in, passed by the workflow's nodes module and by **run_knowledge_indexing.py** from **config.VECTOR_STORE_DIRECTORY**.
**PURPOSE**
Open the store that indexing built, without building anything.
**REQUIREMENTS**

- The collection **config.VECTOR_COLLECTION_NAME** is opened with the same embedding model it was built with.
- A folder that does not exist is refused, naming it.

**RETURNS**
**chromadb.api.models.Collection.Collection**. The opened collection.

**FUNCTION SIGNATURE**
`read_document_chunks(collection, document_name)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index**, passed by **read_category_code** or **answer_from_documents**.
**document_name** — **str**. One document's file name, passed by the same caller from **config.CATEGORY_CODE_DOCUMENTS**, for example **Code-Alcohol.pdf**.
**PURPOSE**
Read one document whole.
**REQUIREMENTS**

- Every passage stored for **document_name** is returned, and no passage of any other document.
- A document with no passages in the store is refused.

**RETURNS**
**list[str]**. The document's passage texts.

**FUNCTION SIGNATURE**
`find_nearest_passage(collection, question, document_name)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index**, passed by the retrieval tools.
**question** — **str**. What the passage should answer, passed by the retrieval tools from config, for example **config.REBOOKING_QUESTION**.
**document_name** — **str**. The one document to search, passed by the retrieval tools from **config.CATEGORY_CODE_DOCUMENTS** or **config.GUIDE_DOCUMENT_BY_ISSUE**.
**PURPOSE**

Find the one passage of one document that best answers one question.
**REQUIREMENTS**
- The passage returned is the one of **document_name** closest in meaning to **question**, and never another document's.
- A document with no passages in the store is refused.

**RETURNS**
**str**. The passage text.

**FUNCTION SIGNATURE**
`search_documents(collection, query_texts)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index**, passed by **answer_from_documents**.
**query_texts** — **list[str]**. One or more questions searched together, passed by **answer_from_documents** as the question and the conversation's **search_text**.
**PURPOSE**
Search by meaning, across every document the chat may read.
**REQUIREMENTS**
- There is one result list per question, in the order the questions were given, each holding the **config.CHAT_RETRIEVAL_RESULT_COUNT** passages closest to it, closest first.
- Each passage carries **chunk_id**, **chunk_text**, **document_name** and **distance**, the cosine distance as a **float**.
- No passage comes from a document in **config.CHAT_EXCLUDED_DOCUMENTS**.

**RETURNS**
**list[list[dict]]**. The passages found for each question.

**MODULE: knowledge/code_reading.py**
Reads one category's five rules out of its own code.

**FUNCTION SIGNATURE**
`read_category_code(collection, product_category)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index**, passed by **run_knowledge_indexing.py**, by the service at start-up or by **check_spot_schedule_node**.
**product_category** — **str**. One of the five categories, from a campaign's **product_category** or from the keys of **config.CATEGORY_CODE_DOCUMENTS**, passed by the same caller.
**PURPOSE**
Read the rules a campaign's spots answer to.
**REQUIREMENTS**
- The rules are read from every passage of the category's own code, named by **config.CATEGORY_CODE_DOCUMENTS**, and from no other document; a category with no configured code is refused.
- **restricted_hours** is a **dict** of **start** and **end**, written **HH:MM** as **config.RESTRICTED_HOURS_PATTERN** reads them, or **None** where **config.ANY_TIME_PATTERN** is stated; **live_sport_exception** is a **dict** of **minutes_after_start** and **minutes_before_end**, as integers, where **config.LIVE_SPORT_EXCEPTION_PATTERN** is stated, otherwise **None**.
- **children_rule** is the key of **config.CHILDREN_RULE_PATTERNS** whose sentence is stated, and **index_threshold**, **daily_cap** and **separation_minutes** are the integers **config.CODE_FIGURE_PATTERNS** reads.
- **rule_sentences** maps each breach code of **config.BREACH_CODES** to the category code's own sentence for that rule, the window's being the restricted-hours or any-time sentence followed by the exception where one is stated.
- A code that gives a rule no value or two different values, or fails in any other way set out in **THE RULES ARE NOT IN THE SOLUTION**, is refused, naming the document.

**RETURNS**
**dict**. **document_name**, the code's file name, **restricted_hours**, **live_sport_exception**, **children_rule**, **index_threshold**, **daily_cap**, **separation_minutes** and **rule_sentences**.

## ========== AGENT TOOLS ==========
Every fact an agent is given is assembled here; no agent works one out for itself. Every function works on data its caller has loaded: no tool reads a file or changes the data it is given.

**MODULE: tools/campaign_data_tools.py**
Serves single campaigns out of the live queue: the campaign's own facts, its spots placed on the broadcast day, the schedules those spots fall in, and its notes.

**FUNCTION SIGNATURE**
`fetch_campaign_profile(open_campaigns, campaign_reference)`

**PARAMETERS**
**open_campaigns** — **pandas.DataFrame**. The live campaigns, read as text once at start-up by **load_raw_dataset** from **config.OPEN_CAMPAIGNS_FILE**, passed by **load_campaign_context_node** or by the service at start-up.

**campaign_reference** — **str**. The campaign being asked about, for example **CLR-QSC-0042117**, taken by the same caller from the workflow state or from the queue.

**PURPOSE**

Return one campaign's own facts.

**REQUIREMENTS**

- The record is the campaign's one row, with every column the queue holds.
- A reference the queue does not hold is refused with **config.CAMPAIGN_NOT_IN_QUEUE_MESSAGE** filled with the reference, and a reference the queue holds more than once is refused.

**RETURNS**

**dict**. The campaign's row, keyed by column name, each value the **str** the queue holds, or NaN where it holds none.

**FUNCTION SIGNATURE**

```
fetch_campaign_spots(booked_spots, campaign_reference)
```

**PARAMETERS**

**booked_spots** — **pandas.DataFrame**. The live booking, read as text once at start-up by **load_raw_dataset** from **config.BOOKED_SPOTS_FILE**, each spot exactly as its scheduler wrote it, passed by **load_campaign_context_node** or by the service at start-up.

**campaign_reference** — **str**. The campaign whose spots are wanted, taken by the same caller from the workflow state or from the queue.

**PURPOSE**

Place one campaign's spots on the broadcast day and clock.

**REQUIREMENTS**

- Each spot's **channel** and **broadcast_day** are canonical, as **config.BOOKED_SPOT_VALUE_MAPPINGS** folds them, and its break sits on its broadcast day and the broadcast clock as **THE BROADCAST DAY AND THE BROADCAST CLOCK** sets out; spots come in spot-reference order.
- **break_time** is written **HH:MM** on the broadcast clock, and **break_minute** is the same time in minutes.
- A spot reference repeated anywhere in the booking, a campaign with no spots, a spot leaving any of **config.BOOKED_SPOT_REQUIRED_COLUMNS** blank, a spot whose time or day cannot be placed for certain, and a length that, read in seconds, is not one of **config.STANDARD_SPOT_LENGTHS**, are refused.

**RETURNS**

**list[dict]**. One entry per spot, holding **spot_reference**, **channel**, **broadcast_day**, **break_time** and **break_minute**.

**FUNCTION SIGNATURE**

```
fetch_campaign_schedules(programme_schedule, campaign_spots)
```

**PARAMETERS**

**programme_schedule** — **pandas.DataFrame**. The booking week's schedule, read as text once at start-up by **load_raw_dataset** from **config.PROGRAMME_SCHEDULE_FILE**, passed by **check_spot_schedule_node** or by the service at start-up.

**campaign_spots** — **list[dict]**. Spots as **fetch_campaign_spots** returned them, one campaign's or every campaign's, passed by the same caller.

**PURPOSE**

Return the schedules of the channels and broadcast days the spots are booked on.

**REQUIREMENTS**

- There is one entry for each channel and broadcast day the spots touch, in booking-week order and then by channel name, holding **channel**, **broadcast_day** and its **programmes** in start order.
- Each programme holds **schedule_entry_id**, **programme_title**, **start_time**, **end_time**, **start_minute** and **end_minute** on the broadcast clock, **forecast_under18_index** as an integer, and **live_sport** and **made_for_children** as **bool**.
- A channel and day the schedule does not carry, a programme that does not end after it starts, runs outside the broadcast day or leaves a time or the index blank, programmes that overlap, and a flag that is not **Yes** or **No**, are refused.

**RETURNS**

**list[dict]**. One schedule per channel and broadcast day booked.

**FUNCTION SIGNATURE**

```
fetch_campaign_notes(open_campaign_notes, campaign_reference)
```

**PARAMETERS**

**open_campaign_notes** — **pandas.DataFrame**. The live campaigns' notes, read as text once at start-up by **load_raw_dataset** from **config.OPEN_CAMPAIGN_NOTES_FILE**, passed by **load_campaign_context_node** or **get_campaign_detail**.

**campaign_reference** — **str**. The campaign whose notes are wanted, taken by the same caller from the workflow state or from the request path.

**PURPOSE**

Hand the issue model one campaign's notes, oldest first.

**REQUIREMENTS**

- Only that campaign's notes are returned, ordered by **note_sequence** read as a number, whatever order the table is in.
- A campaign with no notes, notes that do not each carry their own sequence number, and a note with its source or text blank, are refused.

**RETURNS**

**list[dict]**. One entry per note, oldest first, holding **note_sequence** as an integer, **note_source** and **note_text**.

**MODULE: tools/prediction_tools.py**
Turns a model's raw output into the facts the business acts on.

**FUNCTION SIGNATURE**
`build_amendment_facts(amendment_model, amendment_feature_names, campaigns)`

**PARAMETERS**
**amendment_model** — **sklearn.pipeline.Pipeline**. The saved amendment model, loaded once at start-up by the workflow's nodes module, passed by **load_campaign_context_node** or by the service at start-up.
**amendment_feature_names** — **list[str]**. Its saved feature schema, loaded with it, passed by the same caller.
**campaigns** — **pandas.DataFrame**. One row per campaign to score, each value as the queue holds it: the one campaign's profile as a single row, or every live campaign, passed by the same caller.
**PURPOSE**
Score and band each campaign's amendment risk.
**REQUIREMENTS**
- Each campaign is scored on the inputs **build_feature_frame** builds from its row, in the saved schema's order.
- **amendment_score** is the probability of the positive class rounded to **config.AMENDMENT_SCORE_DECIMALS** places, and **amendment_band** bands the score shown, as **AMENDMENT RISK AND ITS BAND** sets out.

**RETURNS**
**list[dict]**. One entry per campaign, in the table's order, holding **amendment_score** and **amendment_band**.

**FUNCTION SIGNATURE**
`build_issue_facts(issue_model, campaign_notes)`

**PARAMETERS**
**issue_model** — **sklearn.pipeline.Pipeline**. The saved issue model, loaded once at start-up by the workflow's nodes module, passed by **load_campaign_context_node** or **get_campaign_detail**.
**campaign_notes** — **list[dict]**. One campaign's notes, as **fetch_campaign_notes** returned them, passed by the same caller.
**PURPOSE**
Name the issue a campaign's notes describe, and say how far they agree.
**REQUIREMENTS**
- The notes are read, weighed and resolved as **READING A CLEARANCE NOTE** sets out; a note from a source **config.NOTE_SOURCE_WEIGHTS** does not list is refused, and so is a campaign with no readable note.
- **weighted_votes** gives each issue named its weighted votes, most first; **weighted_agreement** is the issue's share of all the weighted votes; **notes_read** and **notes_too_short_to_read** count the notes read and set aside.
- **note_readings** gives each readable note's **note_sequence** and the **issue_category** it was read as, in the order written.

**RETURNS**
**dict**. **issue_category**, **weighted_votes**, **weighted_agreement**, **notes_read**, **notes_too_short_to_read** and **note_readings**.

**FUNCTION SIGNATURE**
`forecast_programme_audiences(exposure_model, exposure_feature_names, programme_schedule)`

**PARAMETERS**
**exposure_model** — **sklearn.pipeline.Pipeline**. The saved exposure model, loaded once at start-up by the workflow's nodes module, which passes it.
**exposure_feature_names** — **list[str]**. Its saved feature schema, loaded with it, passed by the same caller.
**programme_schedule** — **pandas.DataFrame**. The booking week's schedule, as read at start-up, passed by the same caller.
**PURPOSE**
Forecast every booking-week programme's under-18 audience.
**REQUIREMENTS**
- Every programme is forecast once, on the inputs **add_slot_minutes** and **build_feature_frame** give it, in the saved schema's order.
- A schedule entry that appears more than once is refused.

**RETURNS**
**dict[str, float]**. Each **schedule_entry_id** against its forecast under-18 viewers, in thousands.

**FUNCTION SIGNATURE**
`compute_campaign_exposure(spot_results, programme_forecasts)`

**PARAMETERS**
**spot_results** — **list[dict]**. One campaign's checked spots, as **check_campaign_spots** returned them, passed by **compose_clearance_assessment_node** or by the service at start-up.
**programme_forecasts** — **dict[str, float]**. Every programme's forecast, as **forecast_programme_audiences** returned it at start-up, passed by the same caller.
**PURPOSE**
Compute a campaign's exposure.
**REQUIREMENTS**
- The exposure is worked out as **THE EXPOSURE, AND THE ORDER OF THE QUEUE** sets out, rounded once to **config.EXPOSURE_DECIMALS** place; a campaign with no breaching spot has an exposure of nought.
- A breaching spot's programme with no forecast is refused.

**RETURNS**
**float**. The exposure, in thousands.

**MODULE: tools/schedule_tools.py**
Places each spot's break in the schedule, tests the five rules, summarises a campaign's results, works out what a decision clears, and ranks the queue.

**FUNCTION SIGNATURE**
`place_spot(campaign_spot, campaign_schedules)`

**PARAMETERS**
**campaign_spot** — **dict**. One spot, as **fetch_campaign_spots** returned it, passed by **check_spot_schedule_node** or by the service at start-up.
**campaign_schedules** — **list[dict]**. The schedules, as **fetch_campaign_schedules** returned them, passed by the same caller.
**PURPOSE**
Say which programme a spot's break is in, or which two it is next to.
**REQUIREMENTS**

- The spot is placed in the one schedule of its channel and broadcast day, as **WHERE A BREAK SITS: IN AND ADJACENT** sets out: **break_position** is **config.BREAK_IN** with the one programme around the break, or **config.BREAK_ADJACENT** with the programme ending at the break followed by the one starting there.
- A channel and day carried by no schedule or by more than one, and a break at the edge of the broadcast day or in a gap, are refused.

**RETURNS**
**dict**. **spot_reference**, **channel**, **broadcast_day**, **break_time**, **break_minute**, **break_position** and **programmes**, each programme as the schedule holds it.

**FUNCTION SIGNATURE**
`check_campaign_spots(spot_placements, code_rules)`

**PARAMETERS**
**spot_placements** — **list[dict]**. Every placed spot of one campaign, as **place_spot** returned them, passed by **check_spot_schedule_node** or by the service at start-up.
**code_rules** — **dict**. The rules of the campaign's own category, as **read_category_code** returned them, passed by the same caller.
**PURPOSE**
Test every spot of one campaign against the five rules of its code.
**REQUIREMENTS**

- Every spot is tested as **THE FIVE RULES** sets out, in transmission order, and **breach_codes** lists every rule it breaks in the order of **config.BREACH_CODES**, empty for a compliant spot.
- The restricted hours and the exception come from **code_rules**; a code with no restricted hours breaks no spot on **config.BREACH_WINDOW**.
- A children's rule that is not a key of **config.CHILDREN_RULE_PATTERNS** is refused.

**RETURNS**
**list[dict]**. Each placement with its **breach_codes** added, in transmission order.

**FUNCTION SIGNATURE**
`summarise_spot_checks(spot_results)`

**PARAMETERS**
**spot_results** — **list[dict]**. One campaign's checked spots, as **check_campaign_spots** returned them, passed by **check_spot_schedule_node**, **compute_clearance_outcome** or the service at start-up.
**PURPOSE**
Summarise a campaign's spot results and the decisions open.
**REQUIREMENTS**

- **spots_booked**, **spots_compliant** and **spots_breaching** count the spots, and **breaches_by_rule** gives every code of **config.BREACH_CODES**, in that order, the number of spots that break it, nought included.
- **first_transmission** holds the **broadcast_day**, **break_time** and **break_minute** of the earliest spot in transmission order, and **decisions_open** lists the open decisions in the order of **config.DECISIONS**, as **THREE DECISIONS, AND THE DECISIONS OPEN** sets out.
- An empty list of results is refused.

**RETURNS**
**dict**. **spots_booked**, **spots_compliant**, **spots_breaching**, **breaches_by_rule**, **first_transmission** and **decisions_open**.

**FUNCTION SIGNATURE**
`compute_clearance_outcome(decision, spot_results)`

**PARAMETERS**
**decision** — **str**. The Clearance Controller's decision, from the decision the workflow resumed with, passed by **request_controller_decision_node**.
**spot_results** — **list[dict]**. The campaign's checked spots, from the workflow state, passed by the same caller.
**PURPOSE**
Work out which spots a decision clears and withdraws.
**REQUIREMENTS**

- **config.DECISION_CLEAR** clears every spot, **config.DECISION_CLEAR_WITH_RESTRICTIONS** clears the compliant spots and withdraws the breaching ones, and **config.DECISION_REJECT** withdraws every spot; both lists keep the order of **spot_results**.
- A value that is not one of **config.DECISIONS**, and a decision the spots do not leave open, are refused, the second in the words of **config.DECISION_NOT_OPEN_MESSAGE**.

**RETURNS**
**dict**. **decision**, **spots_cleared** and **spots_withdrawn** as lists of spot references, and **spots_cleared_count** and **spots_withdrawn_count**.

**FUNCTION SIGNATURE**
`rank_clearance_queue(queue_entries)`

**PARAMETERS**
**queue_entries** — **list[dict]**. One entry per live campaign, each holding at least **campaign_reference**, **exposure** and **first_transmission**, built by the service at start-up, which passes them.
**PURPOSE**
Order the clearance queue.
**REQUIREMENTS**

- The entries are ordered as **THE EXPOSURE, AND THE ORDER OF THE QUEUE** sets out, each returned unchanged.
- A campaign reference that appears more than once is refused.

**RETURNS**
**list[dict]**. The same entries, in queue order.

**MODULE: tools/retrieval_tools.py**
Reads the passage an agent quotes from the one document that states it.

**FUNCTION SIGNATURE**
`read_executive_checks(collection, issue_category)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index**, passed by **identify_issues_node**.
**issue_category** — **str**. The issue the notes name, from the campaign's issue facts, passed by the same caller.
**PURPOSE**
Read the executive's checks from the issue's guide.
**REQUIREMENTS**

- The passage is the one of the issue's guide, named by **config.GUIDE_DOCUMENT_BY_ISSUE**, nearest to **config.EXECUTIVE_CHECKS_QUESTION**; an issue with no configured guide is refused.

**RETURNS**
**str**. The passage text.

**FUNCTION SIGNATURE**
`read_rebooking_steps(collection, product_category)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, passed by **draft_restriction_notice_node**.
**product_category** — **str**. The campaign's category, from its profile, passed by the same caller.
**PURPOSE**
Read the code's re-booking steps.
**REQUIREMENTS**

- The passage is the one of the category's code, named by **config.CATEGORY_CODE_DOCUMENTS**, nearest to **config.REBOOKING_QUESTION**; a category with no configured code is refused.

**RETURNS**
**str**. The passage text.

**FUNCTION SIGNATURE**
`read_standing_restrictions(collection, product_category)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, passed by **draft_clearance_certificate_node**.
**product_category** — **str**. The campaign's category, from its profile, passed by the same caller.
**PURPOSE**
Read the code's standing restrictions.
**REQUIREMENTS**

- The passage is the one of the category's code nearest to **config.STANDING_RESTRICTIONS_QUESTION**; a category with no configured code is refused.

**RETURNS**
**str**. The passage text.

**========== PERSISTENCE ==========**

Six tables hold the accounts, runs, assessments, decisions, language model calls and chat questions. Apart from the workflow's checkpoint, nothing else is written.

**MODULE: persistence/database.py**
The connection and the session factory: two module-level objects built on first import and shared by the whole project. It defines no function.
**MODULE-LEVEL OBJECTS**

- **database_engine** — **sqlalchemy.engine.Engine**. The connection to the database at **config.APPLICATION_DATABASE_URL**, whose connections the web framework may use from any thread.
- **session_factory** — **sqlalchemy.orm.sessionmaker**. Bound to that engine, and the only way the rest of the project opens a session.

**REQUIREMENTS**

- Both are created when the module is imported, and no other module creates an engine.
- Importing this module neither creates tables nor writes anything; creating the tables is the database setup entry point's work.

**MODULE: persistence/models.py**
Declares the six tables. Reproduce table and column names exactly, in lower case.

**CLASS**
`Base`

The declarative base every table below inherits from, and the object the setup entry point uses to create them all.

**TABLE**
**user_accounts**
**CLASS**
`UserAccount`

**COLUMNS**

| Column | Type | Notes |
|---|---|---|
| username | **str** | The key |
| display_name | **str** | The name a page shows |
| role | **str** | One of the three roles |
| assigned_agencies | **list** | The agency codes a liaison is assigned, stored as JSON; empty for the desk's roles |
| password_hash | **str** | Never the password itself |
| password_salt | **str** | This account's own salt |
| session_token | **str \| None** | The current session, or nothing when signed out |
| created_at | **datetime.datetime** | Set by the database when the row is written |

**TABLE**
**agent_runs**
**CLASS**
`AgentRun`

**COLUMNS**

| Column | Type | Notes |
|---|---|---|
| run_id | **str** | The key, **config.RUN_ID_PREFIX** followed by **config.IDENTIFIER_HEX_LENGTH** upper-case hexadecimal characters |
| thread_id | **str** | The identifier the paused workflow is stored under |
| campaign_reference | **str** | The campaign being assessed |
| executive_id | **str** | The username of whoever opened the assessment |
| executive_request | **str** | What they asked to be looked at |
| status | **str** | One of **config.AGENT_RUN_STATUS_RUNNING**, **config.AGENT_RUN_STATUS_AWAITING_DECISION**, **config.AGENT_RUN_STATUS_COMPLETE** and **config.AGENT_RUN_STATUS_FAILED** |
| llm_call_count | **int** | The calls recorded against the run, written when it reaches the gate, completes or fails, and nought before then |
| started_at | **datetime.datetime** | Set by the database when the row is written |
| finished_at | **datetime.datetime \| None** | Set when the run completes or fails, and nothing while it runs or waits |

TABLE
clearance_assessments
CLASS
`ClearanceAssessment`

COLUMNS

| Column | Type | Notes |
|---|---|---|
| assessment_id | **str** | The key, **config.ASSESSMENT_ID_PREFIX** followed by **config.IDENTIFIER_HEX_LENGTH** upper-case hexadecimal characters |
| run_id | **str** | The run that produced it |
| campaign_reference | **str** | The campaign |
| agency_code, agency_name, advertiser_name, copy_title, clock_number, product_category | **str** | The campaign's own facts, from its profile |
| code_document | **str** | The code its spots were checked against |
| executive_request | **str** | What the executive asked |
| amendment_score | **float** | The score, as **build_amendment_facts** gave it |
| amendment_band | **str** | Its band |
| issue_category | **str** | The issue the notes name |
| issue_agreement | **float** | Their weighted agreement |
| spots_booked, spots_compliant, spots_breaching | **int** | The spot counts, as **summarise_spot_checks** gave them |
| breaches_by_rule | **dict** | Each breach code against its count, stored as JSON |
| breaching_spots | **list** | Each breaching spot's **spot_reference**, **channel**, **broadcast_day**, **break_time**, **programmes** as **programme_title** and **forecast_under18_index**, and **breach_codes**, stored as JSON |
| first_transmission | **dict** | As **summarise_spot_checks** gave it, stored as JSON |
| exposure_thousands | **float** | The campaign's exposure |
| decisions_open | **list** | The decisions the spots leave open, stored as JSON |
| assessment_plan_text | **str** | The planner's three fields, each written as its label, a colon, a space and its text, joined by blank lines |
| assessment_text | **str** | The writer's assessment |
| recommended_decision | **str** | The writer's recommendation, one of the decisions open |
| status | **str** | **config.CAMPAIGN_STATUS_PENDING** until decided, then the status the decision leaves |
| spots_cleared, spots_withdrawn | **list \| None** | The spot references the decision cleared and withdrew, stored as JSON; nothing until decided |
| spots_cleared_count, spots_withdrawn_count | **int \| None** | How many of each; nothing until decided |
| restriction_notice_text | **str \| None** | The notice's fields written the same way as the plan, or nothing when no notice was drafted |
| clearance_certificate_text | **str \| None** | The certificate's fields written the same way, or nothing when none was drafted |
| created_at | **datetime.datetime** | Set by the database when the row is written |
| status_changed_at | **datetime.datetime** | Set when the row is written, and set again when the decision is recorded |

TABLE
controller_decisions
CLASS
`ControllerDecision`

COLUMNS

| Column | Type | Notes |
|---|---|---|
| decision_id | **int** | The key, numbered by the database |
| assessment_id | **str** | The assessment decided |
| run_id | **str** | Its run |
| decision | **str** | One of **config.DECISIONS** |
| decided_by | **str** | The display name of the Clearance Controller who took it |
| decided_by_role | **str** | Their role |
| decision_reason | **str** | Their reason, in their own words |
| decided_at | **datetime.datetime** | Set by the database when the row is written |

**TABLE**
**llm_calls**
**CLASS**
`LlmCall`
**COLUMNS**

| Column | Type | Notes |
|---|---|---|
| call_id | **int** | The key, numbered by the database |
| run_id | **str** | The run the call belongs to |
| agent_name | **str** | The agent that made it, one of the names in config |
| model_name | **str** | The model that answered |
| prompt_characters | **int** | The prompt's length |
| response_characters | **int** | The reply's length |
| latency_milliseconds | **int** | How long the call took |
| called_at | **datetime.datetime** | Set by the database when the row is written |

**TABLE**
**chat_turns**
**CLASS**
`ChatTurn`
**COLUMNS**

| Column | Type | Notes |
|---|---|---|
| turn_id | **int** | The key, numbered by the database |
| username | **str** | Who asked |
| user_role | **str** | Their role |
| question_text | **str** | What they asked |
| question_type | **str** | One of **config.CHAT_TYPE_OPTIONS** |
| campaign_reference | **str \| None** | The campaign the question named, or nothing |
| answer_text | **str** | The answer given |
| source_documents | **str** | The documents the answer was given from, joined by a comma and a space; empty when none |
| closest_distance | **float** | The distance of the nearest passage found |
| was_refused | **bool** | Whether the question was refused |
| asked_at | **datetime.datetime** | Set by the database when the row is written |

**MODULE: persistence/repository.py**
Every read and write against the six tables; nothing else builds a query, and every row is returned as a plain record, not a database object. Every function takes an open **session** from **session_factory** and writes and flushes but never commits; the caller commits.

**FUNCTION SIGNATURE**
`insert_user_account(session, account_record)`
**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.

**account_record** — **dict**. One account's columns other than **session_token** and **created_at**, built by **run_database_setup.py** from **config.SEED_USER_ACCOUNTS**, with the password already in its stored form.
**PURPOSE**
Create one sign-in account.
**REQUIREMENTS**
- The row is written and flushed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`fetch_user_account(session, condition)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.
**condition** — **sqlalchemy.sql.elements.ColumnElement**. A condition on the **user_accounts** columns, written by the caller: on **username** by **sign_in_user** and **run_database_setup.py**, on **session_token** by **authorize_request** and **sign_out_user**.
**PURPOSE**
Find one account.
**REQUIREMENTS**
- The record carries every column of the account, and nothing is returned when no account matches.

**RETURNS**
**dict | None**. The account's row, keyed by column name, or **None**.

**FUNCTION SIGNATURE**
`update_session_token(session, username, session_token)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.
**username** — **str**. The account whose session changes, passed by **sign_in_user** or **sign_out_user**.
**session_token** — **str | None**. The new token, or **None** to end the session, passed by the same caller.
**PURPOSE**
Start or end an account's session.
**REQUIREMENTS**
- The account holds the token given, replacing any earlier one, so signing in again ends the session before.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`insert_agent_run(session, run_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **load_campaign_context_node** from **session_factory**.
**run_record** — **dict**. **run_id**, **thread_id**, **campaign_reference**, **executive_id**, **executive_request** and **status**, built by **load_campaign_context_node**.
**PURPOSE**
Record that a run has started.
**REQUIREMENTS**
- The row is written and flushed, with a call count of nought.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`update_agent_run(session, run_id, status_value, llm_call_count)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.
**run_id** — **str**. The run, from the workflow state or the paused workflow's stored state, passed by **compose_clearance_assessment_node**, **record_clearance_outcome_node** or **report_failed_request**.
**status_value** — **str**. The run's new status, one of the run statuses in config, passed by the same caller.
**llm_call_count** — **int**. The calls the run has recorded, passed by the same caller.
**PURPOSE**
Move a run on to its next status.
**REQUIREMENTS**
- The run holds the status and the call count given, and **finished_at** is set when the status is one of **config.AGENT_RUN_FINISHED_STATUSES**.
- A run that does not exist is refused.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`insert_llm_call(session, run_id, call_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the node that made the call from **session_factory**.
**run_id** — **str**. The run the call belongs to, from the workflow state, passed by that node.
**call_record** — **dict**. **agent_name**, **model_name**, **prompt_characters**, **response_characters** and **latency_milliseconds**, as **call_language_model** returned them, passed by the same node.
**PURPOSE**
Log what one language model call cost.
**REQUIREMENTS**
- One row is written and flushed for the call.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`fetch_llm_calls(session, run_id)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **get_call_budget** or **report_failed_request** from **session_factory**.
**run_id** — **str**. The run whose calls are wanted, from the assessment's record or the paused workflow's stored state, passed by the same caller.
**PURPOSE**
Read back every call a run made.
**REQUIREMENTS**
- Every call of the run is returned, in the order the calls were made, each with every column of **llm_calls**.

**RETURNS**
**list[dict]**. The run's calls.

**FUNCTION SIGNATURE**
`insert_clearance_assessment(session, assessment_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **compose_clearance_assessment_node** from **session_factory**.
**assessment_record** — **dict**. Every column of **clearance_assessments** that is known at the gate, built by **compose_clearance_assessment_node**.
**PURPOSE**
Record the finished assessment, waiting for its decision.
**REQUIREMENTS**
- The row is written and flushed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`record_clearance_outcome(session, assessment_id, outcome_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **record_clearance_outcome_node** from **session_factory**.
**assessment_id** — **str**. The assessment decided, from the workflow state, passed by **record_clearance_outcome_node**.
**outcome_record** — **dict**. **status**, **spots_cleared**, **spots_withdrawn**, **spots_cleared_count**, **spots_withdrawn_count**, **restriction_notice_text** and **clearance_certificate_text**, built by **record_clearance_outcome_node**.
**PURPOSE**
Close an assessment with what its decision cleared, withdrew and sent.
**REQUIREMENTS**
- Exactly the one assessment holds the values given, and its **status_changed_at** is set again.
- An assessment that does not exist is refused.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`fetch_assessment_records(session, condition)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.
**condition** — **sqlalchemy.sql.elements.ColumnElement**. A condition on the **clearance_assessments** columns, written by the caller: on **assessment_id**, **campaign_reference** or **status**, by the service's endpoints and by **answer_campaign_status**.
**PURPOSE**
Read assessments back with the run and the decision behind each one.
**REQUIREMENTS**
- Each record holds every column of the assessment, the run's **thread_id**, **executive_id** and **llm_call_count**, and the decision's **decision**, **decided_by**, **decided_by_role**, **decision_reason** and **decided_at**, those five being **None** until it is decided.
- Records are returned newest first.

**RETURNS**
**list[dict]**. The matching records.

**FUNCTION SIGNATURE**
`insert_controller_decision(session, decision_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **record_clearance_outcome_node** from **session_factory**.
**decision_record** — **dict**. **assessment_id**, **run_id**, **decision**, **decided_by**, **decided_by_role** and **decision_reason**, built by **record_clearance_outcome_node**.
**PURPOSE**
Record a Clearance Controller's decision, who took it and why.
**REQUIREMENTS**
- The row is written and flushed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`insert_chat_turn(session, turn_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **ask_clearance_assistant** from **session_factory**.
**turn_record** — **dict**. Every column of **chat_turns** but **turn_id** and **asked_at**, built by **ask_clearance_assistant**.
**PURPOSE**
Record one chat question and its answer.
**REQUIREMENTS**
- The row is written and flushed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`fetch_chat_turns(session, condition, turn_limit)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.
**condition** — **sqlalchemy.sql.elements.ColumnElement**. A condition on the **chat_turns** columns, written by the caller: on **username** by **recall_conversation** and **list_own_chat_turns**, or one that every row meets by **list_recent_chat_turns**.
**turn_limit** — **int**. The most turns to return, passed by the same caller.
**PURPOSE**
Read back the latest chat turns.
**REQUIREMENTS**
- At most **turn_limit** matching turns are returned, newest first, each with every column of **chat_turns**.

**RETURNS**
**list[dict]**. The turns.

## ========== AGENTIC WORKFLOW ==========

Five agents assess a campaign, and one or two more write the documents the decision calls for. Each agent is one language model call and decides nothing about a spot; every agent that writes facts is told in its prompt to use only the facts it is given. The workflow stops before any spot is cleared or anything is sent, and the pause is checkpointed to disk.

### MODULE: agentic/state.py

Declares what the workflow carries from one node to the next. Every key a node reads must be present before that node runs, and no key may exist that nothing reads.

**CLASS**
`ClearanceAssessmentState`

**STATE KEYS**

| Key | Type | Written by | What it carries |
|---|---|---|---|
| campaign_reference | **str** | the caller | The campaign being assessed |
| executive_request | **str** | the caller | What the person who opened the assessment asked to be looked at |
| executive_id | **str** | the caller | The username of whoever opened the assessment |
| thread_id | **str** | the caller | The identifier the pause is stored under |
| run_id | **str** | load_campaign_context_node | The run, as recorded in **agent_runs** |
| assessment_id | **str** | load_campaign_context_node | The assessment this run will write |
| campaign_profile | **dict** | load_campaign_context_node | The campaign's own facts |
| campaign_spots | **list** | load_campaign_context_node | Its spots on the broadcast day |
| campaign_notes | **list** | load_campaign_context_node | Its notes, in the order written |
| amendment_facts | **dict** | load_campaign_context_node | What the amendment model and the bands established |
| issue_facts | **dict** | load_campaign_context_node | What the issue model established |
| schedule_facts | **dict** | check_spot_schedule_node | **code_rules**, **spot_results** and **summary**: the code read, every spot placed and tested, and the summary |
| assessment_plan | **dict** | plan_assessment_node | The planning agent's reply |
| copy_assessment | **dict** | assess_copy_risk_node | The copy risk agent's reply |
| issue_analysis | **dict** | identify_issues_node | The issue agent's reply |
| schedule_advice | **dict** | check_spot_schedule_node | The schedule agent's reply |
| controller_decision | **dict** | the gate | The decision, who took it and why: **decision**, one of **config.DECISIONS**; **decided_by**, the person's display name; **decided_by_role**, their role; and **decision_reason**, their own words |
| clearance_outcome | **dict** | the gate | What the decision clears and withdraws |
| restriction_notice | **dict** | the gate and draft_restriction_notice_node | The notice, empty until one is written |
| clearance_certificate | **dict** | the gate and draft_clearance_certificate_node | The certificate, empty until one is written |
| llm_call_count | **int** | every agent node | The calls the run has made, added up across nodes rather than replaced |

**REQUIREMENTS**

- Every key above is declared, and nothing else is.
- **llm_call_count** accumulates across nodes, each agent node adding the one call it made. Every other key is replaced by whichever node writes it.

**MODULE: agentic/agents.py**

The language model client, the reply parser and the seven prompts. Each prompt gives an agent established facts and asks for a reply in labelled fields. Every prompt speaks for **config.ORGANISATION_NAME**, and a yes-or-no fact is written in the words **config.BOOLEAN_LABELS** gives it.

The key and the model name are read from the environment through **config.GEMINI_API_KEY_VARIABLE** and **config.GEMINI_MODEL_VARIABLE**, loaded from **.env** on import; neither is written into the code.

**MODULE-LEVEL OBJECTS**

- **generation_settings** — **google.genai.types.GenerateContentConfig**. The one generation configuration every call uses, holding **config.GEMINI_MAX_OUTPUT_TOKENS** and **config.GEMINI_TEMPERATURE**, with the client's own function calling turned off.

**FUNCTION SIGNATURE**

```
call_language_model(prompt_text, agent_name)
```

**PARAMETERS**

**prompt_text** — **str**. The whole prompt, assembled by the agent function calling this, or by **answer_from_documents**.

**agent_name** — **str**. Which agent is calling, passed by that function from its name constant in config, for example **config.AGENT_NAME_PLANNER**.

**PURPOSE**

Make one language model call and return the reply with what the call cost.

**REQUIREMENTS**

- The key and the model name are read at the moment of the call; when either is missing the call is refused, naming both settings. The reply is generated under **generation_settings**.

- An empty reply, or a reply that did not finish normally, is refused, naming the agent and the model.
- **call_record** carries **agent_name**, **model_name**, **prompt_characters**, **response_characters** and **latency_milliseconds**, the last measured around the model call itself.

**RETURNS**
**dict**. **response_text**, the reply with surrounding space removed, and **call_record**.

**FUNCTION SIGNATURE**
`parse_labelled_response(response_text, field_labels, agent_name)`

**PARAMETERS**
**response_text** — **str**. What the model replied, taken by the calling agent function, or by **answer_from_documents**, from the **response_text** that **call_language_model** returned.
**field_labels** — **list[str]**. The labels the prompt asked for, in order, written by that same caller.
**agent_name** — **str**. Which agent replied, passed by that same caller from its name constant, and used in the refusal when a label is missing.
**PURPOSE**
Read a labelled reply back into its fields.
**REQUIREMENTS**
- A line that starts with one of **field_labels** and a colon starts that field, whatever case the model used and whatever decoration from **config.RESPONSE_DECORATION_CHARACTERS**, bold markers included, it put around the label.
- The lines after a label, up to the next label, belong to that field; text before the first label is ignored; a label written again adds its text to the field rather than replacing it. Bold markers are removed wherever they appear, text on a label's own line carries no decoration at either end, and no field has surrounding whitespace.
- A label that is missing, or answered with nothing, is refused, naming the agent and the labels; the result holds exactly the labels asked for, in the order asked.

**RETURNS**
**dict**. One entry per label.

**FUNCTION SIGNATURE**
`plan_assessment(campaign_profile, executive_request)`

**PARAMETERS**
**campaign_profile** — **dict**. The campaign's queue row, as **fetch_campaign_profile** returned it, passed by **plan_assessment_node** from the workflow state.
**executive_request** — **str**. What the person who opened the assessment asked, in their own words, passed by **plan_assessment_node** from the workflow state.
**PURPOSE**
Plan what the assessment must establish.
**REQUIREMENTS**
- The prompt states the campaign's own facts — the copy, its claims and offers, the booking and the advertiser's and agency's record, the advertiser's share of amended campaigns given ready-made — and the request, and asks for **PRIORITY**, **CHECKS** and **RATIONALE**.
- The agent is told that it is planning the assessment, and to name no rule, code or figure it was not given. The call is made as **config.AGENT_NAME_PLANNER**.

**RETURNS**
**dict**. **assessment_plan**, holding the three fields, and **call_record**.

**FUNCTION SIGNATURE**
`assess_copy_risk(amendment_facts, campaign_profile, assessment_plan)`

**PARAMETERS**
**amendment_facts** — **dict**. The campaign's entry from **build_amendment_facts**, passed by **assess_copy_risk_node** from the workflow state.
**campaign_profile** — **dict**. The campaign's queue row, passed by the same node.
**assessment_plan** — **dict**. The planning agent's three fields, carried in the workflow state, passed by the same node.
**PURPOSE**
Explain what drives the amendment score.
**REQUIREMENTS**
- The prompt hands the agent the score, the band, the band thresholds, the facts the model read and the plan's **CHECKS**, and tells it not to recalculate or invent any figure, and that the score is a prediction that does not say whether the copy breaks a code.
- The reply carries **DRIVERS**, **HISTORY** and **COPY RISK SUMMARY**, and the call is made as **config.AGENT_NAME_COPY_RISK**.

**RETURNS**
**dict**. **copy_assessment** and **call_record**.

**FUNCTION SIGNATURE**
`identify_issues(issue_facts, campaign_notes, executive_checks, assessment_plan, copy_assessment`

**PARAMETERS**
**issue_facts** — **dict**. What **build_issue_facts** established for this campaign, passed by **identify_issues_node** from the workflow state.

**campaign_notes** — **list[dict]**. The campaign's notes, as **fetch_campaign_notes** returned them, passed by the same node.
**executive_checks** — **str**. The passage **read_executive_checks** returned to the same node from the issue's guide.
**assessment_plan** — **dict**. The planning agent's three fields, passed by the same node.
**copy_assessment** — **dict**. The copy risk agent's three fields, passed by the same node.
**PURPOSE**
Weigh the issue the notes name against its guide.
**REQUIREMENTS**

- The prompt hands the agent every note with how it was read, the notes read and set aside, the issue, its weighted agreement and votes, the guide's checks, the plan's **CHECKS**, the copy risk agent's **COPY RISK SUMMARY** and the eight issues.
- The agent is told that agreement below about half means the notes disagree, and to use only the notes, their readings, the votes and the guide's passage it is given.
- The reply carries **ISSUE**, **NOTES** and **CHECKS TO MAKE**, and the call is made as **config.AGENT_NAME_ISSUE**.

**RETURNS**
**dict**. **issue_analysis** and **call_record**.

**FUNCTION SIGNATURE**
```
advise_on_spot_schedule(schedule_facts, campaign_profile, assessment_plan)
```
**PARAMETERS**
**schedule_facts** — **dict**. **code_rules**, **spot_results** and **summary**, established by **check_spot_schedule_node** just before it calls this.
**campaign_profile** — **dict**. The campaign's queue row, passed by the same node.
**assessment_plan** — **dict**. The planning agent's three fields, passed by the same node.
**PURPOSE**
State which spots may air and which may not, and the rule each breaks.
**REQUIREMENTS**

- The prompt states the code's own rule sentences and the document they came from, every spot in transmission order with its broadcast day and time, the programmes it is in or between with their times, index and flags, and its result, the summary, and the plan's **CHECKS**.
- The agent is told to quote each result exactly as given and never to test a spot or move it from one list to the other.
- The reply carries **MAY AIR**, **MAY NOT AIR** and **SCHEDULE SUMMARY**, and the call is made as **config.AGENT_NAME_SCHEDULE**.

**RETURNS**
**dict**. **schedule_advice** and **call_record**.

**FUNCTION SIGNATURE**
```
write_clearance_assessment(campaign_profile, executive_request, amendment_facts, issue_facts, s
```
**PARAMETERS**
**campaign_profile** — **dict**. The campaign's queue row, passed by **compose_clearance_assessment_node** from the workflow state.
**executive_request** — **str**. What the assessment was opened to look at, passed by the same node from the workflow state.
**amendment_facts** — **dict**. The campaign's amendment facts, passed by the same node from the workflow state.
**issue_facts** — **dict**. The campaign's issue facts, passed by the same node from the workflow state.
**schedule_facts** — **dict**. The schedule facts, passed by the same node from the workflow state.
**exposure_thousands** — **float**. What **compute_campaign_exposure** returned, worked out by the same node just before it calls this.
**copy_assessment** — **dict**. The copy risk agent's fields, passed by the same node from the workflow state.
**issue_analysis** — **dict**. The issue agent's fields, passed by the same node from the workflow state.
**schedule_advice** — **dict**. The schedule agent's fields, passed by the same node from the workflow state.
**PURPOSE**
Write the assessment the Clearance Controller decides from, and recommend one decision.
**REQUIREMENTS**

- The prompt states the campaign, the request, what the copy, issue and schedule agents found, the figures the tools established and the exposure, and explains the three decisions and when each is open, in the Advertising Clearance Policy's terms.
- The agent is told that it recommends and does not decide, and to quote every figure exactly.
- The reply carries **ASSESSMENT** and **RECOMMENDED DECISION**, which is returned as one of the decisions open, whatever its case, decoration, surrounding quotation marks or closing full stop; a recommendation over more than one line, or one that is not open, is refused.
- The call is made as **config.AGENT_NAME_WRITER**.

**RETURNS**
**dict**. **clearance_assessment**, holding the two fields, and **call_record**.

**FUNCTION SIGNATURE**
```
draft_restriction_notice(campaign_profile, schedule_facts, clearance_outcome, rebooking_steps)
```
**PARAMETERS**
**campaign_profile** — **dict**. The campaign's queue row, passed by **draft_restriction_notice_node** from the workflow state.
**schedule_facts** — **dict**. The schedule facts, passed by the same node from the workflow state.
**clearance_outcome** — **dict**. What **compute_clearance_outcome** returned at the gate, passed by the same node from the workflow state.
**rebooking_steps** — **str**. The passage **read_rebooking_steps** returned to the same node from the category's code.

**PURPOSE**
Write the restriction notice for the withdrawn spots.
**REQUIREMENTS**

- The prompt states the campaign, every withdrawn spot with its channel, broadcast day, time, programmes and breach codes, the code's sentences for the rules they broke, how many spots are cleared and the re-booking steps. A programme is marked only as at or above the threshold, never with its index.
- The agent is told to quote no index, forecast, score, note or assessment, to name no member of staff and no other agency or advertiser, and to invent no contact details.
- The reply carries **SUBJECT**, **WITHDRAWN SPOTS** and **NEXT STEPS**; a **WITHDRAWN SPOTS** that does not name each withdrawn spot exactly once, matched by **config.SPOT_REFERENCE_MENTION_PATTERN**, or names a cleared spot, is refused. The call is made as **config.AGENT_NAME_RESTRICTION_NOTICE**.

**RETURNS**
**dict**. **restriction_notice** and **call_record**.

**FUNCTION SIGNATURE**
`draft_clearance_certificate(campaign_profile, schedule_facts, clearance_outcome, standing_restr`

**PARAMETERS**
**campaign_profile** — **dict**. The campaign's queue row, passed by **draft_clearance_certificate_node** from the workflow state.
**schedule_facts** — **dict**. The schedule facts, passed by the same node from the workflow state.
**clearance_outcome** — **dict**. What **compute_clearance_outcome** returned at the gate, passed by the same node from the workflow state.
**standing_restrictions** — **str**. The passage **read_standing_restrictions** returned to the same node from the category's code.
**restriction_notice** — **dict**. The notice's fields when one was written, and empty when the decision withdrew nothing, passed by the same node from the workflow state.
**PURPOSE**
Write the clearance certificate for the cleared spots.
**REQUIREMENTS**

- The prompt states the campaign, the decision, every cleared spot numbered in transmission order with its channel, broadcast day, time and programmes, the standing restrictions, and the notice's subject where a notice was sent, or that none was.
- The reply carries **CLEARED SPOTS**, **STANDING RESTRICTIONS** and **RESTRICTION NOTICE**; a **CLEARED SPOTS** that does not name each cleared spot exactly once, matched by **config.SPOT_REFERENCE_MENTION_PATTERN**, or names a withdrawn spot, is refused.
- The call is made as **config.AGENT_NAME_CLEARANCE_CERTIFICATE**.

**RETURNS**
**dict**. **clearance_certificate** and **call_record**.

**MODULE: agentic/nodes.py**
The ten workflow steps and the router out of the gate. Each node takes the state and returns only the keys it wrote. A node that calls an agent records the call against the run, in the same transaction as its other writes, as soon as the agent function returns; a reply the agent function refuses stops the run before its call is recorded.

**MODULE-LEVEL RESOURCES**
Everything below is opened once, when the module is first imported, from the paths in config, and every node and the service use it.

| Name | Type | What it holds |
|---|---|---|
| amendment_model, exposure_model, issue_model | **sklearn.pipeline.Pipeline** | The saved models, loaded by **load_model** |
| amendment_feature_names, exposure_feature_names | **list[str]** | Their saved schemas, loaded by **load_feature_schema** |
| open_campaigns, booked_spots, programme_schedule, open_campaign_notes | **pandas.DataFrame** | The four live-queue files, read by **load_raw_dataset** |
| knowledge_collection | **chromadb.api.models.Collection.Collection** | The knowledge base, opened by **open_vector_index** |
| programme_forecasts | **dict[str, float]** | Every programme of the booking week against its forecast, from **forecast_programme_audiences** |

**FUNCTION SIGNATURE**
`load_campaign_context_node(state)`

**PARAMETERS**
**state** — **ClearanceAssessmentState**. The workflow's own state, a typed dictionary handed to the node by LangGraph and checkpointed after every node. This node reads **campaign_reference**, **executive_request**, **executive_id** and **thread_id**.
**PURPOSE**

Open the run and gather the campaign's facts.

**REQUIREMENTS**

- The campaign's facts, spots and notes are fetched, and its amendment facts and issue facts established, before any agent runs.
- The run identifier is **config.RUN_ID_PREFIX** and the assessment identifier **config.ASSESSMENT_ID_PREFIX**, each followed by **config.IDENTIFIER_HEX_LENGTH** random upper-case hexadecimal characters; the run is written to **agent_runs** with status **config.AGENT_RUN_STATUS_RUNNING**.

**RETURNS**

**dict**. **run_id**, **assessment_id**, **campaign_profile**, **campaign_spots**, **campaign_notes**, **amendment_facts** and **issue_facts**.

**FUNCTION SIGNATURE**

```
plan_assessment_node(state)
```

**PARAMETERS**

**state** — **ClearanceAssessmentState**. The workflow's own state. This node reads **campaign_profile**, **executive_request** and **run_id**.

**PURPOSE**

Ask the planning agent what has to be established.

**REQUIREMENTS**

- The agent is called once, and the call is recorded against the run.

**RETURNS**

**dict**. **assessment_plan**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**

```
assess_copy_risk_node(state)
```

**PARAMETERS**

**state** — **ClearanceAssessmentState**. The workflow's own state. This node reads **amendment_facts**, **campaign_profile**, **assessment_plan** and **run_id**.

**PURPOSE**

Ask the copy risk agent to explain the score.

**REQUIREMENTS**

- The agent is called once, and the call is recorded against the run.

**RETURNS**

**dict**. **copy_assessment**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**

```
identify_issues_node(state)
```

**PARAMETERS**

**state** — **ClearanceAssessmentState**. The workflow's own state. This node reads **issue_facts**, **campaign_notes**, **assessment_plan**, **copy_assessment** and **run_id**.

**PURPOSE**

Ask the issue agent how much weight the notes deserve.

**REQUIREMENTS**

- The agent is given the executive's checks from the guide of the issue the notes name; it is called once, and the call is recorded against the run.

**RETURNS**

**dict**. **issue_analysis**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**

```
check_spot_schedule_node(state)
```

**PARAMETERS**

**state** — **ClearanceAssessmentState**. The workflow's own state. This node reads **campaign_profile**, **campaign_spots**, **assessment_plan** and **run_id**.

**PURPOSE**

Test every spot against its category's own code, then ask the schedule agent to explain the results.

**REQUIREMENTS**

- Every spot is placed, tested against its category's code from **read_category_code**, and summarised before the agent is called.
- The agent is called once, and the call is recorded against the run.

**RETURNS**

**dict**. **schedule_facts**, **schedule_advice**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**

```
compose_clearance_assessment_node(state)
```

**PARAMETERS**

**state** — **ClearanceAssessmentState**. The workflow's own state. This node reads **campaign_profile**, **executive_request**, **amendment_facts**, **issue_facts**, **schedule_facts**, **copy_assessment**, **issue_analysis**, **schedule_advice**, **assessment_plan**, **assessment_id**, **run_id** and **llm_call_count**.

**PURPOSE**

Work out the exposure, have the assessment written, and record it for the Clearance Controller.

**REQUIREMENTS**

- The exposure is worked out before the writer is called, and the writer is called once.
- In one transaction, the call is recorded, the assessment is written to **clearance_assessments** with every column known at the gate and status **config.CAMPAIGN_STATUS_PENDING**, and the run moves to **config.AGENT_RUN_STATUS_AWAITING_DECISION** with its call count including this call.

**RETURNS**

**dict**. **llm_call_count** as **1**.

**FUNCTION SIGNATURE**

```
request_controller_decision_node(state)
```

**PARAMETERS**

**state** — **ClearanceAssessmentState**. The workflow's own state. This node reads **assessment_id** and **schedule_facts**.

**PURPOSE**

Stop the run until a Clearance Controller decides, then work out what the decision clears.

**REQUIREMENTS**

- The run is paused with the assessment identifier, and resumes with the controller's decision record.
- On resuming, the outcome is worked out by **compute_clearance_outcome**, and the notice and the certificate start empty. No language model is called here.

**RETURNS**

**dict**. **controller_decision**, **clearance_outcome**, **restriction_notice** and **clearance_certificate**.

**FUNCTION SIGNATURE**

```
route_controller_decision(state)
```

**PARAMETERS**

**state** — **ClearanceAssessmentState**. The workflow's own state. This router reads **controller_decision**.

**PURPOSE**

Send the run down the path its decision calls for.

**REQUIREMENTS**

- The value returned is the decision itself, one of **config.DECISIONS**, and the router changes nothing.

**RETURNS**

**str**. The decision.

**FUNCTION SIGNATURE**

```
draft_restriction_notice_node(state)
```

**PARAMETERS**

**state** — **ClearanceAssessmentState**. The workflow's own state. This node reads **campaign_profile**, **schedule_facts**, **clearance_outcome** and **run_id**.

**PURPOSE**

Have the restriction notice written.

**REQUIREMENTS**

- The agent is given the re-booking steps from the category's code; it is called once, and the call is recorded against the run.

**RETURNS**

**dict**. **restriction_notice**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**

```
draft_clearance_certificate_node(state)
```

**PARAMETERS**

**state** — **ClearanceAssessmentState**. The workflow's own state. This node reads **campaign_profile**, **schedule_facts**, **clearance_outcome**, **restriction_notice** and **run_id**.

**PURPOSE**

Have the clearance certificate written.

**REQUIREMENTS**

- The agent is given the standing restrictions from the category's code; it is called once, and the call is recorded against the run.

**RETURNS**

**dict**. **clearance_certificate**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**

```
record_clearance_outcome_node(state)
```

**PARAMETERS**

**state** — **ClearanceAssessmentState**. The workflow's own state. This node reads **controller_decision**, **clearance_outcome**, **restriction_notice**, **clearance_certificate**, **assessment_id**, **run_id** and **llm_call_count**.

**PURPOSE**

Close the campaign with its decision, what it cleared and withdrew, and the documents sent.

**REQUIREMENTS**

- In one transaction the decision is written to **controller_decisions**, the assessment is closed and the run moves to **config.AGENT_RUN_STATUS_COMPLETE** with its call count.
- The assessment takes the status **config.CAMPAIGN_STATUS_BY_DECISION** gives the decision, the spots cleared and withdrawn with their counts, and the notice and the certificate each written as its labelled fields joined by blank lines, or nothing where none was drafted.
- No language model is called here.

**RETURNS**
**dict**. Empty: the run is over.

**MODULE: agentic/graph.py**
The order the nodes run in, the gate, the branch and the checkpoint.
**FUNCTION SIGNATURE**
`build_workflow(checkpoint_path)`

**PARAMETERS**
**checkpoint_path** — **str**. The file paused runs are written to, passed by the service module when it loads, from **config.CHECKPOINT_DATABASE_FILE**.
**PURPOSE**
Assemble the ten nodes into a workflow that can be run, paused and resumed.
**REQUIREMENTS**

- The ten nodes are registered under exactly the names above, and the first six run in the order shown, followed by the gate.
- After the gate there is one conditional branch, **route_controller_decision**, and each of the three values in **config.DECISIONS** leads to its own next node; every path ends at **record_clearance_outcome**, which is joined to the workflow's **END**, and every node is reachable from the start.
- On the path that clears with restrictions, the notice is written before the certificate.
- The compiled workflow is given a checkpointer that writes to **checkpoint_path**, whose connection the web framework may use from any thread, so a run paused at the gate can be resumed in another process, after a restart; building the workflow starts nothing and calls no model.

**RETURNS**
**langgraph.graph.state.CompiledStateGraph**. The compiled workflow, ready to be run and resumed.

**========== THE REST SERVICE ==========**
Every fact the dashboard shows comes from this service; the dashboard holds no business rule and reads no file.

**MODULE: api/schemas.py**
Every request and reply shape, declared once. A request failing its conditions is refused with status **422** before any work, and a reply carries exactly the fields listed. Every facts dictionary holds only **str**, **int**, **float**, **bool**, **None** and lists or dictionaries of them, never a NumPy value or a NaN. Reproduce field names exactly.
**MODULE-LEVEL TYPES**

- **CategoryFilter** — **typing.Literal**. The values the queue may be filtered by, from **config.QUEUE_CATEGORY_OPTIONS**: **config.QUEUE_CATEGORY_ALL** for the whole queue, or one of the five categories.
- **CampaignStatusValue** — **typing.Literal**. The values an assessment's status may take, from **config.CAMPAIGN_STATUS_OPTIONS**.
- **DecisionValue** — **typing.Literal**. The decisions a controller may send, from **config.DECISIONS**.
- **ExecutiveRequestText**, **DecisionReasonText** and **QuestionText** — **typing.Annotated** text types, stripped of surrounding whitespace and at least **config.EXECUTIVE_REQUEST_MINIMUM_CHARACTERS**, **config.DECISION_REASON_MINIMUM_CHARACTERS** and **config.CHAT_QUESTION_MINIMUM_CHARACTERS** characters long.

**REQUEST SHAPE**
`SignInRequest`
**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| username | **str** | The account signing in |
| password | **str** | The password as typed |

**VALIDATION**

- Both fields are required.

**RESPONSE SHAPE**
`SignedInUserResponse`
**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| username | **str** | The account that signed in |
| display_name | **str** | The person's name as the dashboard shows it |
| role | **str** | The role that account holds |
| assigned_agencies | **list[str]** | The agency codes a liaison is assigned; empty for the desk's roles |
| session_token | **str** | The token every later request carries in **config.SESSION_TOKEN_HEADER** |
| pages | **list[str]** | The pages this role is offered, from **config.ROLE_PAGE_ACCESS** |
| areas | **list[str]** | The areas this role reaches, from **config.ROLE_AREA_ACCESS** |

#### VALIDATION

None beyond the field types above.

#### RESPONSE SHAPE

`SignedOutResponse`

#### FIELDS

| Field | Type | What it carries |
|---|---|---|
| username | **str** | The account whose session ended |
| signed_out | **bool** | That the session is over |

#### VALIDATION

None beyond the field types above.

#### RESPONSE SHAPE

`QueueRow`

#### FIELDS

| Field | Type | What it carries |
|---|---|---|
| queue_position | **int** | The campaign's place in the queue, from **1** |
| campaign_reference | **str** | The campaign |
| agency_name, advertiser_name, copy_title, product_category | **str** | The campaign's own facts |
| spots_booked, spots_breaching | **int** | Its spot counts |
| first_transmission | **dict** | Its first spot, as **summarise_spot_checks** gives it |
| exposure | **float** | Its exposure, in thousands |
| amendment_score | **float** | Its amendment score |
| amendment_band | **str** | The score's band |

#### VALIDATION

None beyond the field types above.

#### RESPONSE SHAPE

`CampaignDetailResponse`

#### FIELDS

| Field | Type | What it carries |
|---|---|---|
| queue_position | **int** | The campaign's place in the queue |
| profile | **dict** | The campaign's own facts, as **fetch_campaign_profile** returns them, a missing value as **None** |
| campaign_notes | **list[dict]** | Its notes, as **fetch_campaign_notes** returns them |
| amendment | **dict** | Its entry from **build_amendment_facts**: **amendment_score** and **amendment_band** |
| issue | **dict** | Its issue facts, as **build_issue_facts** returns them |
| code_rules | **dict** | Its category's rules, as **read_category_code** returns them |
| schedules | **list[dict]** | The schedules of every channel and broadcast day its spots are booked on |
| spot_results | **list[dict]** | Every spot placed and tested, as **check_campaign_spots** returns them |
| summary | **dict** | As **summarise_spot_checks** returns it |
| exposure | **float** | Its exposure, in thousands |

**VALIDATION**
None beyond the field types above.

**REQUEST SHAPE**
`AssessmentRequest`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| campaign_reference | **str** | The campaign to assess |
| executive_request | **ExecutiveRequestText** | What the person opening the assessment wants looked at |

**VALIDATION**
- **executive_request**, stripped, is at least **config.EXECUTIVE_REQUEST_MINIMUM_CHARACTERS** characters long.

**REQUEST SHAPE**
`DecisionRequest`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| decision | **DecisionValue** | One of **config.DECISIONS** |
| decision_reason | **DecisionReasonText** | The controller's reason, in their own words |

**VALIDATION**
- **decision** is one of **config.DECISIONS**, and **decision_reason**, stripped, is at least **config.DECISION_REASON_MINIMUM_CHARACTERS** characters long.

**RESPONSE SHAPE**
`ClearanceRecordResponse`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| every column of **clearance_assessments** | as in the table | The assessment as stored |
| thread_id, executive_id | **str** | From its run |
| llm_call_count | **int** | The calls its run has recorded |
| decision, decided_by, decided_by_role, decision_reason | **str \| None** | From its decision, **None** until decided |
| decided_at | **datetime.datetime \| None** | When it was decided, **None** until then |

**VALIDATION**
None beyond the field types above.

**RESPONSE SHAPE**
`CallBudgetResponse`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| assessment_id | **str** | The assessment |
| run_id | **str** | Its run |
| call_count | **int** | The calls the run has recorded |
| expected_call_count | **int** | **config.LLM_CALLS_BEFORE_DECISION** until decided, then **config.LLM_CALLS_BY_DECISION** for the decision taken |
| total_latency_milliseconds | **int** | The recorded calls' latencies added together |
| calls | **list[dict]** | Every recorded call, in the order made, with every column of **llm_calls** |

**VALIDATION**
None beyond the field types above.

**REQUEST SHAPE**
`ChatRequest`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| question | **QuestionText** | The question asked |

**VALIDATION**

- **question**, stripped, is at least **config.CHAT_QUESTION_MINIMUM_CHARACTERS** characters long.

**RESPONSE SHAPE**

`ChatResponse`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| question_type | **str** | One of **config.CHAT_TYPE_OPTIONS** |
| campaign_reference | **str \| None** | The campaign the question named, or **None** |
| answer | **str** | The answer given |
| source_documents | **list[str]** | The documents the answer was given from, empty for a refusal or a campaign status answer |
| was_refused | **bool** | Whether the question was refused |

**VALIDATION**

None beyond the field types above.

**RESPONSE SHAPE**

`ChatTurnResponse`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| every column of **chat_turns** | as in the table | One stored turn |

**VALIDATION**

None beyond the field types above.

**MODULE: api/routes.py**

**PURPOSE**

Serve the eleven endpoints in one application, all but signing in and out behind the sign-in and access guard, with one failure handler.

**MODULE-LEVEL OBJECTS**

- **assessment_graph** — **langgraph.graph.state.CompiledStateGraph**. The workflow, built once by **build_workflow** from **config.CHECKPOINT_DATABASE_FILE**.
- **category_codes** — **dict[str, dict]**. Every category in **config.CATEGORY_CODE_DOCUMENTS** against its rules, read once by **read_category_code**.
- **campaign_profiles** — **dict[str, dict]**. Every live campaign's profile, keyed by its reference.
- **amendment_facts_by_reference** — **dict[str, dict]**. Every live campaign's amendment facts, keyed by its reference.
- **campaign_spots_by_reference** — **dict[str, list]**. Every live campaign's spots on the broadcast day, keyed by its reference.
- **booked_schedules** — **list[dict]**. The schedules of every channel and broadcast day any live spot is booked on, fetched once.
- **spot_results_by_reference**, **summaries_by_reference** and **exposures_by_reference** — **dict**. Every live campaign's checked spots, summary and exposure, keyed by its reference.
- **ranked_campaigns** — **list[dict]**. The queue entries in the order **rank_clearance_queue** gives them.
- **clearance_queue** — **list[dict]**. The ranked entries, each with its **queue_position** from **1**.
- **queue_rows_by_reference** — **dict[str, dict]**. Each queue row, keyed by its reference.
- **assessment_start_locks** — **dict[str, threading.Lock]**. One lock per campaign: two simultaneous starts of one campaign open one run.
- **decision_lock** — **threading.Lock**. Held while a decision is checked and recorded: two simultaneous decisions record one.
- **public_router** — **fastapi.APIRouter**. Signing in and signing out, reachable without a token.
- **guarded_router** — **fastapi.APIRouter**. Every other endpoint, behind the guard.
- **SessionTokenHeader** — a **typing.Annotated** alias for **str | None**. The optional header **config.SESSION_TOKEN_HEADER**; a request without it is answered **401**, not **422**.
- **SignedInUser** — a **typing.Annotated** alias for **dict**. The endpoint parameter type that receives the account **authorize_request** returned.
- **service_application** — **fastapi.FastAPI**. The application, titled **config.API_TITLE**, holding both routers, the guarded one behind **authorize_request**, and **report_failed_request** as its failure handler.

**REQUIREMENTS**

- The models, the live queue and the knowledge base are taken from the workflow's nodes module, not loaded again. Every campaign's facts above are worked out once, at start-up, with the same tools the workflow uses.
- No endpoint carries a default parameter value except the optional session token header. Every endpoint answers **200** when it succeeds, except **POST /assessments**, which answers **201**.
- Only **POST /assessments**, **POST /clearances/{assessment_id}/decision** and **POST /chat/ask** may cost language model calls.

**sign_in_user**
**ENDPOINT**
**POST /auth/signin**
**FUNCTION SIGNATURE**
`sign_in_user(sign_in_request)`

**PARAMETERS**
**sign_in_request** — **SignInRequest**. The username and password posted, read from the request body by FastAPI.
**PURPOSE**
Exchange a username and password for a session token.
**REQUIREMENTS**

- An unknown username and a wrong password are answered identically, with **401** and **config.SIGN_IN_FAILED_MESSAGE**; the password is checked against the stored form with the account's own salt, compared in constant time.
- A correct password issues a fresh token made from **config.SESSION_TOKEN_BYTES** random bytes and stores it against the account, replacing any token it held.

**RETURNS**
**dict**. A **SignedInUserResponse**.

**sign_out_user**
**ENDPOINT**
**POST /auth/signout**
**FUNCTION SIGNATURE**
`sign_out_user(session_token)`

**PARAMETERS**
**session_token** — **str | None**. The token to end, declared as **SessionTokenHeader** and taken by FastAPI from the header **config.SESSION_TOKEN_HEADER**, and **None** when the header is absent.
**PURPOSE**
End a session.
**REQUIREMENTS**

- The account's token is cleared and cannot be used again; a request with no token, or a token no account holds, is answered **401** with **config.SIGN_IN_REQUIRED_MESSAGE**.

**RETURNS**
**dict**. A **SignedOutResponse**.

**list_clearance_queue**
**ENDPOINT**
**GET /queue**
**FUNCTION SIGNATURE**
`list_clearance_queue(product_category)`

**PARAMETERS**
**product_category** — **str**. One of **config.QUEUE_CATEGORY_OPTIONS**, declared as **CategoryFilter** and taken by FastAPI from the query string.
**PURPOSE**
Give the desk its ranked list of open campaigns.
**REQUIREMENTS**

- **config.QUEUE_CATEGORY_ALL** gives every live campaign once; a category gives exactly the rows of that category, in the same order and with the same positions.

**RETURNS**
**list[dict]**. One **QueueRow** per campaign.

**get_campaign_detail**
**ENDPOINT**
**GET /queue/{campaign_reference}**
**FUNCTION SIGNATURE**
`get_campaign_detail(campaign_reference)`

**PARAMETERS**
**campaign_reference** — **str**. The campaign, taken by FastAPI from the path.
**PURPOSE**
Show one campaign with every fact the assessment works from.
**REQUIREMENTS**

- A campaign not in the queue is answered **404** with **config.CAMPAIGN_NOT_IN_QUEUE_MESSAGE** filled with the reference.
- The facts come from the same tools, schedules and codes the workflow uses; no language model call is made.

**RETURNS**
**dict**. A **CampaignDetailResponse**.

**start_clearance_assessment**
**ENDPOINT**
**POST /assessments**
**FUNCTION SIGNATURE**

```
start_clearance_assessment(assessment_request, signed_in_user, request)
```

**PARAMETERS**
**assessment_request** — **AssessmentRequest**. The campaign and what is to be looked at, read from the request body by FastAPI.
**signed_in_user** — **dict**. The account of whoever is opening the assessment, declared as **SignedInUser** and supplied by FastAPI from **authorize_request**.
**request** — **fastapi.Request**. The request itself, supplied by FastAPI, through which **report_failed_request** learns the thread of a run that fails.
**PURPOSE**
Run the five assessing agents on one campaign and leave it waiting for a Clearance Controller.
**REQUIREMENTS**

- A campaign not in the queue is answered **404** with **config.CAMPAIGN_NOT_IN_QUEUE_MESSAGE**, and a campaign that already has an assessment, whatever its status, **409** with **config.CAMPAIGN_ALREADY_ASSESSED_MESSAGE**, each filled with the reference; neither opens a run.
- The run is given its own thread, recorded on **request** for the failure handler before the workflow starts; the workflow runs with the caller's username as **executive_id** and stops at the gate after **config.LLM_CALLS_BEFORE_DECISION** calls.

**RETURNS**
**dict**. A **ClearanceRecordResponse** for the assessment written, with status **config.CAMPAIGN_STATUS_PENDING**.

**list_clearance_records**
**ENDPOINT**
**GET /clearances**
**FUNCTION SIGNATURE**

```
list_clearance_records(status_value)
```

**PARAMETERS**
**status_value** — **list[str]**. One or more of **config.CAMPAIGN_STATUS_OPTIONS**, declared as a list of **CampaignStatusValue** and taken by FastAPI from the query string, where it may be repeated.
**PURPOSE**
List assessments by status, for the decision queue and the audit trail.
**REQUIREMENTS**

- Every assessment whose status is one of the values given is returned, newest first, and no other.

**RETURNS**
**list[dict]**. One **ClearanceRecordResponse** per assessment.

**submit_controller_decision**
**ENDPOINT**
**POST /clearances/{assessment_id}/decision**
**FUNCTION SIGNATURE**

```
submit_controller_decision(assessment_id, decision_request, signed_in_user)
```

**PARAMETERS**
**assessment_id** — **str**. The assessment being decided, taken by FastAPI from the path.
**decision_request** — **DecisionRequest**. The decision and the reason, read from the request body by FastAPI.
**signed_in_user** — **dict**. The Clearance Controller deciding, declared as **SignedInUser** and supplied by FastAPI from **authorize_request**.
**PURPOSE**
Record a Clearance Controller's decision and finish the run down the path it calls for.
**REQUIREMENTS**

- An unknown assessment is answered **404** with **config.ASSESSMENT_NOT_FOUND_MESSAGE**, and a decision by the person who opened it **403** with **config.OWN_ASSESSMENT_REFUSED_MESSAGE**.
- An assessment already decided is answered **409** with **config.ALREADY_DECIDED_MESSAGE**, and a decision the spots do not leave open **409** with **config.DECISION_NOT_OPEN_MESSAGE** filled with the decision sent and the assessment's **spots_breaching**, **spots_booked** and **spots_compliant**; every refusal is settled under **decision_lock**, before the run resumes.
- The run resumes with **decision**, **decided_by** as the controller's display name, **decided_by_role** and **decision_reason**.
- A decision that failed while its documents were being drafted can be finished only by the same decision, person and reason; anything else is answered **409** with **config.DECISION_IN_PROGRESS_MESSAGE**, filled with the decision already sent and the display name of the Clearance Controller who sent it.

**RETURNS**
**dict**. A **ClearanceRecordResponse** for the assessment, as decided.

**get_call_budget**
**ENDPOINT**
**GET /clearances/{assessment_id}/calls**
**FUNCTION SIGNATURE**

```
get_call_budget(assessment_id)
```

**PARAMETERS**
**assessment_id** — **str**. The assessment, taken by FastAPI from the path.
**PURPOSE**
Show the run's language model calls against the expected count.

**REQUIREMENTS**

- An unknown assessment is answered **404** with **config.ASSESSMENT_NOT_FOUND_MESSAGE**.

**RETURNS**

**dict**. A **CallBudgetResponse**.

**ask_clearance_assistant**

**ENDPOINT**

**POST /chat/ask**

**FUNCTION SIGNATURE**

```
ask_clearance_assistant(chat_request, signed_in_user)
```

**PARAMETERS**

**chat_request** — **ChatRequest**. The question, read from the request body by FastAPI.

**signed_in_user** — **dict**. The account asking, declared as **SignedInUser** and supplied by FastAPI from **authorize_request**.

**PURPOSE**

Answer one question from the documents, or about one campaign from its record, and remember the exchange.

**REQUIREMENTS**

- Each question costs exactly one language model call, made through **answer_from_documents** with the asker's own recalled conversation; a **config.CHAT_TYPE_CAMPAIGN_STATUS** answer is the one **answer_campaign_status** gives the person asking.
- The turn is stored in **chat_turns** with the asker, their role, the question, its type, the campaign named, the answer, the source documents joined by a comma and a space, the closest distance and whether it was refused.

**RETURNS**

**dict**. A **ChatResponse**.

**list_own_chat_turns**

**ENDPOINT**

**GET /chat/turns**

**FUNCTION SIGNATURE**

```
list_own_chat_turns(signed_in_user, turn_limit)
```

**PARAMETERS**

**signed_in_user** — **dict**. The account asking, declared as **SignedInUser** and supplied by FastAPI from **authorize_request**.

**turn_limit** — **int**. How many turns to return, taken by FastAPI from the query string, between **1** and **config.CHAT_AUDIT_MAX_ROWS**.

**PURPOSE**

Show a person their own latest chat turns.

**REQUIREMENTS**

- Only the asker's own turns are returned, newest first, at most **turn_limit** of them.

**RETURNS**

**list[dict]**. One **ChatTurnResponse** per turn.

**list_recent_chat_turns**

**ENDPOINT**

**GET /audit/chat-turns**

**FUNCTION SIGNATURE**

```
list_recent_chat_turns(turn_limit)
```

**PARAMETERS**

**turn_limit** — **int**. How many turns to return, taken by FastAPI from the query string, between **1** and **config.CHAT_AUDIT_MAX_ROWS**.

**PURPOSE**

List everyone's latest chat turns.

**REQUIREMENTS**

- The latest turns of everyone are returned, newest first, at most **turn_limit** of them.

**RETURNS**

**list[dict]**. One **ChatTurnResponse** per turn.

**report_failed_request**

**FUNCTION SIGNATURE**

```
report_failed_request(request, failure)
```

**PARAMETERS**

**request** — **fastapi.Request**. The request that failed, supplied by FastAPI.

**failure** — **Exception**. What went wrong: a **ValueError** from the project's own code, or an error raised by the language model client or by its HTTP connection.

**PURPOSE**

Turn every failure into one plain reply, and close the run a failed request opened.

**REQUIREMENTS**

- It is registered for **ValueError**, for the language model client's API error and for the HTTP client's errors.
- When the failed request had opened a run, that run moves to **config.AGENT_RUN_STATUS_FAILED** with the calls it recorded.

- The reply is **502** with a **detail** naming the method, the path and the failure.

**RETURNS**
**fastapi.responses.JSONResponse**. The failure reply.

**========== ACCESS CONTROL ==========**
Everyone signs in, and the service, not the dashboard, decides what each role may reach.

**MODULE: access/password_security.py**
Turns a password and its salt into the form the database stores.

**FUNCTION SIGNATURE**
`hash_password(plain_password, password_salt)`

**PARAMETERS**
**plain_password** — **str**. The password as typed, passed by **sign_in_user** from the sign-in body, or by **run_database_setup.py** from **config.SEED_USER_ACCOUNTS**.
**password_salt** — **str**. The account's own salt, read by **sign_in_user** from its **user_accounts** row, or made by **run_database_setup.py** from **config.PASSWORD_SALT_BYTES** random bytes when the account is seeded.
**PURPOSE**
Turn a password into the form the database stores.
**REQUIREMENTS**

- The stored form is a key derived with **config.PASSWORD_HASH_ALGORITHM** over **config.PASSWORD_HASH_ROUNDS** rounds, salted with **password_salt**, and returned as hexadecimal text.
- The same password and salt always give the same result, and a different salt gives a different result.

**RETURNS**
**str**. The stored form.

**MODULE: access/permissions.py**
The sign-in and access guard every protected endpoint sits behind.

**FUNCTION SIGNATURE**
`authorize_request(request, session_token)`

**PARAMETERS**
**request** — **fastapi.Request**. The incoming request, supplied by FastAPI and read for the route it matched and its method.
**session_token** — **str | None**. The token sent under **config.SESSION_TOKEN_HEADER**, taken by FastAPI from the header with a default of **None** when the header is absent.
**PURPOSE**
Decide, for every guarded request, who is asking and whether their role reaches that area.
**REQUIREMENTS**

- A request with no token, or a token no account holds, is refused **401** with **config.SIGN_IN_REQUIRED_MESSAGE**.
- The area is decided from the path of the route the request matched, by **config.AREA_BY_PATH_START**, except that a **POST** whose route ends in **config.DECISION_PATH_ENDING** belongs to **config.AREA_DECISION**.
- A role that does not reach that area in **config.ROLE_AREA_ACCESS** is refused **403** with **config.AREA_REFUSED_MESSAGE**, naming the role and the area, before the endpoint runs and whether or not what it asks for exists.

**RETURNS**
**dict**. The signed-in person's account record.

**========== THE CLEARANCE CHAT ==========**
One question, one language model call, answered only from the documents the chat may read. The assistant classifies the question, answers the kinds it may answer and refuses the rest. A question about one campaign is answered by the service from the record, by the asker's role and, for a liaison, their agencies.

| Kind | What it covers | Answered from |
|---|---|---|
| **config.CHAT_TYPE_CODE_RULES** | What a category's code allows or forbids: its restricted hours, the live sport exception, the index threshold, programmes made for children, the daily limit, the separation, its standing restrictions or how a withdrawn spot is re-booked | The five codes, each read whole, and the passages found |
| **config.CHAT_TYPE_SCHEDULING_RULES** | How spots are placed and checked: the broadcast day and clock, centre and end breaks, in and adjacent, how a campaign is reviewed, who decides it and what each decision means | The documents the chat may read |
| **config.CHAT_TYPE_AGENCY_ADVICE** | What an agency can do about its copy, its claims and their evidence, or its bookings | The documents the chat may read |
| **config.CHAT_TYPE_CAMPAIGN_STATUS** | Where one named campaign has got to, or anything about one campaign's spots, scores or decision | The campaign's record, by the service |
| **config.CHAT_TYPE_OUT_OF_SCOPE** | Anything the documents do not answer | Nothing: it is refused |

**MODULE: chat/conversation_memory.py**
Recalls the last few exchanges of a person's own conversation.

**FUNCTION SIGNATURE**
```
recall_conversation(session, username, question_text)
```
**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **ask_clearance_assistant** from **session_factory**.
**username** — **str**. Whose conversation to recall, taken by **ask_clearance_assistant** from the signed-in account.
**question_text** — **str**. The question just asked, taken by **ask_clearance_assistant** from the request body.
**PURPOSE**
Recall the person's recent exchanges as history and as search text.
**REQUIREMENTS**
- At most **config.CHAT_HISTORY_TURN_LIMIT** of that person's own most recent turns are recalled, and nobody else's.
- **history_text** shows them oldest first, each exchange with its question and its answer, a campaign status answer replaced by **config.CHAT_STATUS_ANSWER_IN_HISTORY**; with no earlier turns it says this is the first question.
- **search_text** is the question asked just before, when there is one, followed by the new question.

**RETURNS**
**dict**. **history_text** and **search_text**.

**MODULE: chat/clearance_assistant.py**
Settles what kind of question was asked and answers it from the documents the chat may read, in one language model call.

**FUNCTION SIGNATURE**
```
answer_from_documents(collection, question_text, conversation)
```
**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index** at start-up in the workflow's nodes module, passed by **ask_clearance_assistant**.
**question_text** — **str**. The question as written, taken by **ask_clearance_assistant** from the request body.
**conversation** — **dict**. **history_text** and **search_text**, as **recall_conversation** returned them to **ask_clearance_assistant**.
**PURPOSE**
Answer a question from the documents the chat may read in one language model call, refusing when they do not ground an answer.
**REQUIREMENTS**
- The prompt gives the five codes, each read whole, the passages **search_documents** finds for the question and for **search_text**, each once, and the conversation; it tells the model that a question leaving out its subject continues the last exchange, and that a category's rule is given with the category named.
- The prompt names the kinds in **config.CHAT_TYPE_OPTIONS** and tells the model to answer only from what it is given, to quote a figure only from its category's own code, and that it knows nothing about any campaign; the call is made as **config.AGENT_NAME_CLEARANCE_ASSISTANT**.
- The reply carries **QUESTION TYPE**, **CAMPAIGN REFERENCE**, as **config.CAMPAIGN_REFERENCE_FORM**, and **ANSWER**, with **config.CHAT_NO_VALUE_REPLY** where one does not apply. A type or reference that is not a single line, a type not among the five, a reference other than **config.CHAT_NO_VALUE_REPLY** that **config.CAMPAIGN_REFERENCE_PATTERN** does not match, and an answerable type with no answer, are refused with a **ValueError**, not answered as **was_refused**.
- An answer is given only when the type is one of **config.CHAT_ANSWERED_TYPES** and the nearest passage is within **config.CHAT_MAX_RELEVANCE_DISTANCE**; otherwise **was_refused** is true, the type becomes **config.CHAT_TYPE_OUT_OF_SCOPE** and the answer **config.CHAT_REFUSAL_MESSAGE**, except that a

**config.CHAT_TYPE_CAMPAIGN_STATUS** question keeps its type and the model's own **ANSWER** and is not marked refused.

- **campaign_reference** is the reference in upper case, or **None**; **source_documents** names once each the documents of the passages offered, the five codes included for a **config.CHAT_TYPE_CODE_RULES** answer, and is empty unless an answer was given; **closest_distance** is the nearest passage's distance.

**RETURNS**
**dict**. **question_type**, **campaign_reference**, **answer**, **source_documents**, **closest_distance** and **was_refused**.

**MODULE: chat/campaign_status_answer.py**
Answers a question about one named campaign from its record, in the way the asker's role and assignments allow.

**FUNCTION SIGNATURE**
```
answer_campaign_status(session, open_campaigns, signed_in_user, campaign_reference)
```
**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **ask_clearance_assistant** from **session_factory**.
**open_campaigns** — **pandas.DataFrame**. The live campaigns, read once at start-up by the workflow's nodes module and passed by **ask_clearance_assistant**, used to tell a campaign in the queue from one that is not, and to find its agency.
**signed_in_user** — **dict**. The asker's account record, as **authorize_request** returned it, passed by **ask_clearance_assistant**; its **role** and **assigned_agencies** are read.
**campaign_reference** — **str | None**. The reference **answer_from_documents** read from the model's **CAMPAIGN REFERENCE** field, or **None**, passed by **ask_clearance_assistant**.
**PURPOSE**
Answer a question about one named campaign from its record, across the ownership line.
**REQUIREMENTS**

- The answer follows **THE OWNERSHIP LINE**: **config.CHAT_CAMPAIGN_NOT_FOUND_MESSAGE** for no reference, a reference not in the queue, or another agency's campaign asked about by a liaison; **config.CHAT_NOT_ASSESSED_TEMPLATE** for a campaign nobody has assessed; otherwise **config.CHAT_STATUS_TEMPLATE**, then once decided **config.CHAT_DECIDED_TEMPLATE**, filled with the decision and the counts of spots cleared and withdrawn, and **config.CHAT_DOCUMENTS_SENT** for the status, joined by single spaces.
- A role in **config.CHAT_RECORD_ROLES** also hears, after the status line and before the decision line, the assessment's identifier and the username that started it, the copy, clock number, agency, category and code, the amendment score and band, the issue as recorded and its agreement, the spot counts and breaches by rule, each breaching spot with its programmes and indexes, the first transmission, the exposure, the decisions open and the recommendation.
- Between the decision line and the **config.CHAT_DOCUMENTS_SENT** line, such a role also hears who decided, in which role, their reason as written, and the references of the spots cleared and withdrawn.
- A liaison never hears any of those details, and no language model call is made.

**RETURNS**
**str**. The answer.

**========== THE DASHBOARD ==========**
Five pages over the service, named by **config.PAGE_QUEUE**, **config.PAGE_CAMPAIGN**, **config.PAGE_DECISION**, **config.PAGE_AUDIT** and **config.PAGE_CHAT**. Every campaign fact on a page comes from the service. A page checks only the service's input limits, a decision on one's own assessment and whether a campaign is already assessed. Each page is a function that draws itself and returns nothing.
A band, a decision, a status and a question type are badged with **config.AMENDMENT_BAND_BADGE_COLOURS**, **config.DECISION_BADGE_COLOURS**, **config.STATUS_BADGE_COLOURS** and **config.CHAT_TYPE_BADGE_COLOURS**, and the broadcast-day chart is drawn with **config.PROGRAMME_KIND_COLOURS**, **config.SPOT_RESULT_COLOURS**, **config.SPOT_RESULT_SHAPES**, **config.CHART_SURFACE_COLOUR**, **config.CHART_LABEL_COLOUR**, **config.CHART_AXIS_COLOUR**, **config.CHART_GRID_COLOUR**, **config.STRIP_ROW_HEIGHT** and **config.STRIP_LABEL_WIDTH**; beyond a neutral grey, no page writes a colour of its own. Times are shown in **config.DASHBOARD_TIME_FORMAT**. The theme and the port are pre-loaded in **.streamlit/config.toml**.

**MODULE: dashboard/api_client.py**
Every request the dashboard sends passes through this module; no page holds an address, a timeout or a session token. The one exception is **run_dashboard.py**, which checks that something is listening at **config.API_BASE_URL** before any page is drawn.

**FUNCTION SIGNATURE**
```
call_service(method, path, query, payload, waiting_message)
```
**PARAMETERS**
**method** — **str**. The HTTP method, written by the page making the call, or by **run_dashboard.py** to sign in and to sign out, for example **GET** or **POST**.
**path** — **str**. The path beneath **config.API_BASE_URL**, written by the same caller, for example **/queue** or **/auth/signin**.
**query** — **dict | None**. Query parameters built by the calling page from what the reader chose, or **None** where the endpoint takes none.

**payload** — **dict | None**. The request body built by the same caller from what the reader typed, or **None** where the endpoint takes none.
**waiting_message** — **str | None**. What the page tells the reader while a call that makes language model calls runs, written by the calling page, or **None** for a call that returns at once.
**PURPOSE**
Make one call to the service and return its reply.
**REQUIREMENTS**

- The address is built from **config.API_BASE_URL**, the call is given **config.API_REQUEST_TIMEOUT_SECONDS**, and the token travels under **config.SESSION_TOKEN_HEADER** whenever there is a session; while the request runs, a spinner shows **waiting_message** with the time elapsed.
- A **401** while signed in ends the dashboard's session and returns the reader to the sign-in form with **config.SESSION_ENDED_MESSAGE**.
- Any other reply with status **400** or more is shown as the service's own **detail**, or as the status and the text when the reply is not JSON, and the page stops there.

**RETURNS**
**dict | list**. The service's reply read from JSON.

**MODULE: dashboard/clearance_queue.py**
The clearance queue page.

**FUNCTION SIGNATURE**
`render_clearance_queue_page()`
**PARAMETERS**
None. The page reads from the service through **call_service** and from the browser session.
**PURPOSE**
Show the ranked queue and let the reader pick a campaign.
**REQUIREMENTS**

- Counts along the top, from the whole queue, give the campaigns, those with a breaching spot, those whose every spot breaches, the spots booked and the largest exposure.
- The reader filters by one of **config.QUEUE_CATEGORY_OPTIONS**, and the service does the filtering and the ordering. The table shows each campaign's position, reference, advertiser, category, spots and breaching spots, first spot, exposure, amendment score and band.
- Selecting a row remembers that campaign for the detail page.

**RETURNS**
**None**. The page draws itself.

**MODULE: dashboard/campaign_detail.py**
The campaign detail page and the form that opens an assessment.

**FUNCTION SIGNATURE**
`render_campaign_detail_page()`
**PARAMETERS**
None. The page reads from the service through **call_service** and from the browser session.
**PURPOSE**
Show one campaign's facts and start its assessment.
**REQUIREMENTS**

- The campaign is chosen in queue order, opening on the one the queue page remembered, and everything shown comes from its detail and its records.
- The headline shows the category, the amendment score and band, the decisions open, the queue position and any status, with the spots booked, compliant and breaching, the exposure and the first spot.
- The broadcast days are drawn channel by channel on the broadcast clock, ticks every **config.BROADCAST_CLOCK_TICK_MINUTES** minutes, programmes as bars of kind **config.PROGRAMME_KIND_LIVE_SPORT**, **config.PROGRAMME_KIND_CHILDREN** or **config.PROGRAMME_KIND_OTHER**, and spots as points marked **config.SPOT_RESULT_COMPLIANT** or **config.SPOT_RESULT_BREACHING**.
- The five rules are shown as read beside the code's own sentences, then every spot in transmission order with its break, programmes and result, then the copy, its claims, its history and its notes with how each was read.
- The assessment form takes a request of at least **config.EXECUTIVE_REQUEST_MINIMUM_CHARACTERS**; starting it calls the service once with a waiting message and shows the assessment waiting for a Clearance Controller, with its recommendation. A campaign already assessed shows its assessment's identifier, who started it, when and its status instead.

**RETURNS**
**None**. The page draws itself.

**MODULE: dashboard/decision_queue.py**
The decision queue page and the decision form.

**FUNCTION SIGNATURE**
`render_decision_queue_page()`
**PARAMETERS**
None. The page reads from the service through **call_service** and from the browser session.
**PURPOSE**
Show the assessments waiting at the gate and let a Clearance Controller decide one.
**REQUIREMENTS**

- Only assessments at **config.CAMPAIGN_STATUS_PENDING** are listed, and the chosen one is shown in full: its recommendation, band, category and issue, its spot counts and exposure, the assessment text, every breaching spot with its programmes and codes, the code it was read from, who opened it, what they asked and the plan.
- Where the signed-in person opened the assessment, the page shows **config.OWN_ASSESSMENT_REFUSED_MESSAGE** in place of the form.
- The form offers exactly the decisions open, with what each costs in further model calls, and requires a reason of at least **config.DECISION_REASON_MINIMUM_CHARACTERS**; recording it calls the service once with a waiting message, and the page reports the status reached and the spots cleared and withdrawn.

**RETURNS**
**None**. The page draws itself.

**MODULE: dashboard/audit_trail.py**
The audit trail page: assessed campaigns, the record behind each one, the documents sent, the model calls and the chat log.

**FUNCTION SIGNATURE**
`render_audit_trail_page()`
**PARAMETERS**
None. The page reads from the service through **call_service** and from the browser session.
**PURPOSE**
Show the assessments, the record behind each and the chat log.
**REQUIREMENTS**

- Assessments are listed by the statuses the reader chooses from **config.CAMPAIGN_STATUS_OPTIONS**, **config.DECIDED_CAMPAIGN_STATUSES** chosen at first, with a count for each status chosen.
- Selecting one shows its status and recommendation, who started it and when, the decision with who took it, their role, when and why, the spots cleared and withdrawn, its model calls against the calls expected with the total latency and every call, and the assessment, the restriction notice and the clearance certificate.
- A second tab shows the most recent **config.CHAT_AUDIT_MAX_ROWS** chat turns from everyone, newest first, with who asked, their role, each question's type, the campaign it named, whether it was refused and its sources.

**RETURNS**
**None**. The page draws itself.

**MODULE: dashboard/clearance_chat.py**
The clearance chat page.

**FUNCTION SIGNATURE**
`render_clearance_chat_page()`
**PARAMETERS**
None. The page reads from the service through **call_service** and from the browser session.
**PURPOSE**
Ask the clearance assistant a question and show the conversation.
**REQUIREMENTS**

- The conversation is the signed-in person's own turns, oldest at the top, at most **config.CHAT_PAGE_TURN_LIMIT** of them, each answer with its type, marked where it was refused, and with the documents it drew on.
- The page states the kinds of question the assistant answers and that it remembers the last **config.CHAT_HISTORY_TURN_LIMIT** exchanges; a role outside **config.CHAT_RECORD_ROLES** is told it hears about the campaigns of its own agencies only, naming them.
- A question shorter than **config.CHAT_QUESTION_MINIMUM_CHARACTERS** is not sent; asking calls the service once, with a waiting message saying the documents are being searched in one language model call.

**RETURNS**
**None**. The page draws itself.

**========== THE SIX ENTRY POINTS ==========**
The first four entry points build the system and run once, in order; the last two start the service and the dashboard. Each prints its progress.

**MODULE: run_database_setup.py**

Creates the database, its tables and the accounts people sign in with. It must be safe to run again on a machine that already has a database.

**FUNCTION SIGNATURE**
`main()`

**PARAMETERS**
None.
**PURPOSE**
Create the database, its tables and the seed accounts.
**REQUIREMENTS**

- The folder **config.DATABASE_DIRECTORY** and every table exist when the run finishes, and a second run neither fails nor duplicates anything; an account already present is left exactly as it is.
- Each account in **config.SEED_USER_ACCOUNTS** can sign in afterwards with the username, password, role and assigned agencies given there, stored with its own random salt of **config.PASSWORD_SALT_BYTES** bytes, written as hexadecimal, and the hashed form of its password.
- The console names **config.APPLICATION_DATABASE_FILE**, the tables, each account as it is created, and how many accounts were created and how many were already present.

**RETURNS**
**None**.
**MAIN BLOCK**

- Running the file performs the setup once; importing the module performs none of it.

**MODULE: run_data_pipeline.py**
Cleans all five raw extracts and writes the quality report. Its five cleaning runs are listed, in order, in the module-level **DATASET_JOBS**.

**FUNCTION SIGNATURE**
`main()`

**PARAMETERS**
None.
**PURPOSE**
Clean the five extracts and write the quality report.
**REQUIREMENTS**

- All five jobs run in order, and each extract is profiled as it arrived and as cleaned.
- Each extract's profile is its general and detail profiles merged into one flat dictionary, filed under the raw file's name without **.csv**, and the report is written to **config.DATA_QUALITY_REPORT_FILE**.
- Every row removed is accounted for by **duplicate_rows**, **rows_missing_target** and, where the extract has one, **rows_with_impossible_target** or **rows_with_too_few_words**.
- One console line per extract says how many rows were received, kept and removed, and a closing line names the report.

**RETURNS**
**None**.
**MAIN BLOCK**

- Running the file cleans all five extracts and writes the report; importing the module cleans nothing.

**MODULE: run_model_training.py**
Trains and saves all three models and reports whether each reaches the minimum config sets for it.

**FUNCTION SIGNATURE**
`main()`

**PARAMETERS**
None.
**PURPOSE**
Produce the three saved models, their schemas and their scores.
**REQUIREMENTS**

- The amendment and exposure models are fitted on their cleaned training extracts and scored on their cleaned held-out extracts, on inputs from **build_feature_frame**, the audiences with their slot minutes; the amendment answer is encoded by **config.SUBMISSION_TARGET_ENCODING**.
- Each schema saved is exactly the columns its model was fitted on, in order.
- The cleaned notes are split once, **config.NOTE_HELD_OUT_SHARE** held out, stratified by issue, with **config.RANDOM_SEED**, and the issue model is fitted on the training part only.
- The three models and two schemas are saved to the paths in config, and **config.MODEL_METRICS_FILE** records under **amendment**, **exposure** and **issue** the **algorithm**, the **training_rows** and **held_out_rows**, the **scores**, the **minimum_required**, whether it **meets_minimum** and the **scikit_learn_version**.
- The console gives each model's held-out score, training score and gap, and closes by saying either that every model reaches its minimum or which do not; a model below its minimum is reported, not raised.

**RETURNS**

**None**.
**MAIN BLOCK**
- Running the file trains, saves and scores all three models; importing the module trains nothing.

**MODULE: run_knowledge_indexing.py**
Builds the knowledge base and reads every category's rules back from its own code.

**FUNCTION SIGNATURE**
`main()`
**PARAMETERS**
None.
**PURPOSE**
Build the searchable store and read every code back from it.
**REQUIREMENTS**
- The store at **config.VECTOR_STORE_DIRECTORY** holds every passage of the sixteen documents, and is rebuilt from scratch on every run; the console says how many documents were read, how many sections were found, how many passages were stored and where the store is.
- Every category's rules, read back from the reopened store, are printed with the code they came from: its restricted hours or that its spots may go out at any time, its exception where it has one, its index threshold, its children's rule, its daily limit and its separation.
- A code that cannot be read stops the run with its refusal rather than reporting success.

**RETURNS**
**None**.
**MAIN BLOCK**
- Running the file builds the store and reads it back; importing the module builds nothing.

**MODULE: run_api_service.py**
Starts the REST service after checking that everything it needs is on disk. The module defines one module-level constant, **REQUIRED_BEFORE_STARTING**, a **list[tuple[str, str]]** pairing each path the service needs with what produces it: the three models and two schemas, the four live-queue files, the knowledge base and the database.

**FUNCTION SIGNATURE**
`main()`
**PARAMETERS**
None.
**PURPOSE**
Start the service, or name what is missing.
**REQUIREMENTS**
- Every path in **REQUIRED_BEFORE_STARTING** is checked before anything is imported from the service; where something is missing, the console says the service cannot start and names each missing path and what produces it.
- Otherwise the service starts on **config.API_HOST** and **config.API_PORT**, **8081**, and the console says where the interactive documentation is.

**RETURNS**
**int**. Nought when the service ran, **1** when it could not start.
**MAIN BLOCK**
- Running the file starts the service and makes **main**'s result the process's exit status; importing the module starts nothing.

**MODULE: run_dashboard.py**
The dashboard: signing in, the navigation, and the five pages. The module defines one module-level constant, **PAGE_RENDERERS**, a **dict[str, tuple]** pairing each page name in config with the function that draws it and the icon it is shown with.

**FUNCTION SIGNATURE**
`main()`
**PARAMETERS**
None.
**PURPOSE**
Draw the dashboard with only the pages the signed-in role is offered.
**REQUIREMENTS**
- The page is titled **config.DASHBOARD_TITLE**, and a service that is not answering at **config.API_BASE_URL** is reported as such, naming the address and how to start it.
- Before anyone signs in, the page shows only the sign-in form, under **config.DASHBOARD_TITLE** and **config.DASHBOARD_SUBTITLE**, and a refused sign-in shows the service's own message; after signing in, the person's name, role and any assigned agencies are shown, with a way to sign out that ends the session at the service.
- The pages offered are exactly the ones the service returned for that role, in that order, each drawn by the function **PAGE_RENDERERS** pairs with it.

**RETURNS**

**None**. The dashboard draws itself.

**MAIN BLOCK**

- Running the file through Streamlit draws the dashboard on port **8080**; importing the module draws nothing.

**OUTPUT**

Running the six entry points in order produces the following inside **Project/**.

**FILES CREATED**

| Path | Created by | Contents |
|---|---|---|
| **database/breakcheck.db** | **run_database_setup.py** | The six tables the application writes to, and the fourteen seed accounts |
| **processed/campaign_submissions_train_clean.csv** | **run_data_pipeline.py** | Cleaned amendment training data |
| **processed/campaign_submissions_eval_clean.csv** | **run_data_pipeline.py** | Cleaned amendment held-out data |
| **processed/programme_audiences_train_clean.csv** | **run_data_pipeline.py** | Cleaned exposure training data, every programme on its broadcast day |
| **processed/programme_audiences_eval_clean.csv** | **run_data_pipeline.py** | Cleaned exposure held-out data |
| **processed/clearance_notes_clean.csv** | **run_data_pipeline.py** | Cleaned clearance notes |
| **output/data_quality_report.json** | **run_data_pipeline.py** | How dirty each raw extract was, and what cleaning did to it |
| **output/models/amendment_model.pkl** | **run_model_training.py** | The saved amendment model |
| **output/models/exposure_model.pkl** | **run_model_training.py** | The saved exposure model |
| **output/models/issue_model.pkl** | **run_model_training.py** | The saved issue model, text handling included |
| **output/models/amendment_features.json** | **run_model_training.py** | The columns the amendment model expects, in order |
| **output/models/exposure_features.json** | **run_model_training.py** | The columns the exposure model expects, in order |
| **output/model_metrics.json** | **run_model_training.py** | Every score of every saved model |
| **database/chroma_store/** | **run_knowledge_indexing.py** | The searchable knowledge base |
| **database/checkpoint.db** | **run_api_service.py** | Every run's workflow state, checkpointed after each node |

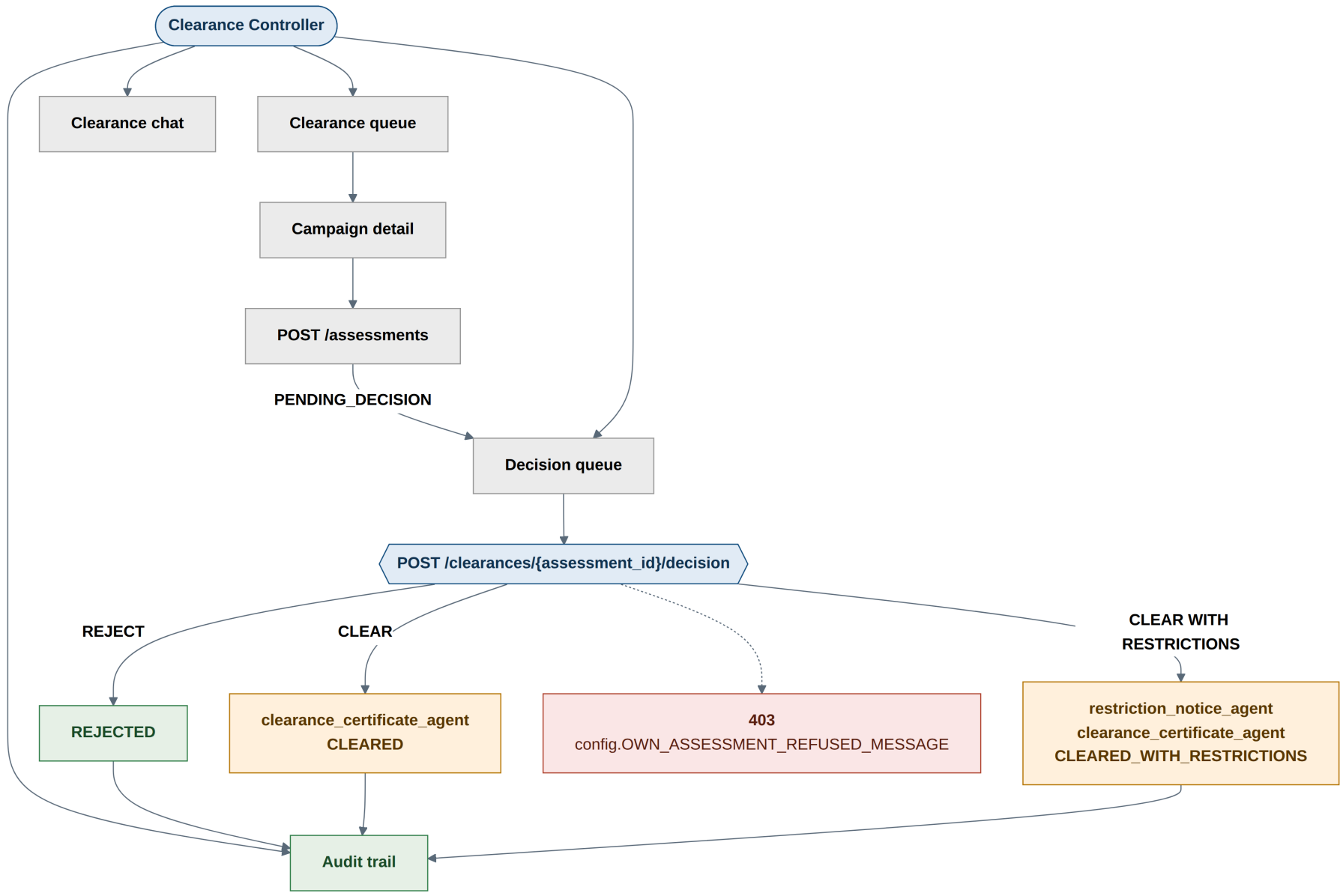
**INFORMATION EACH COMMAND MUST REPORT**

The marking suite reads no console output.

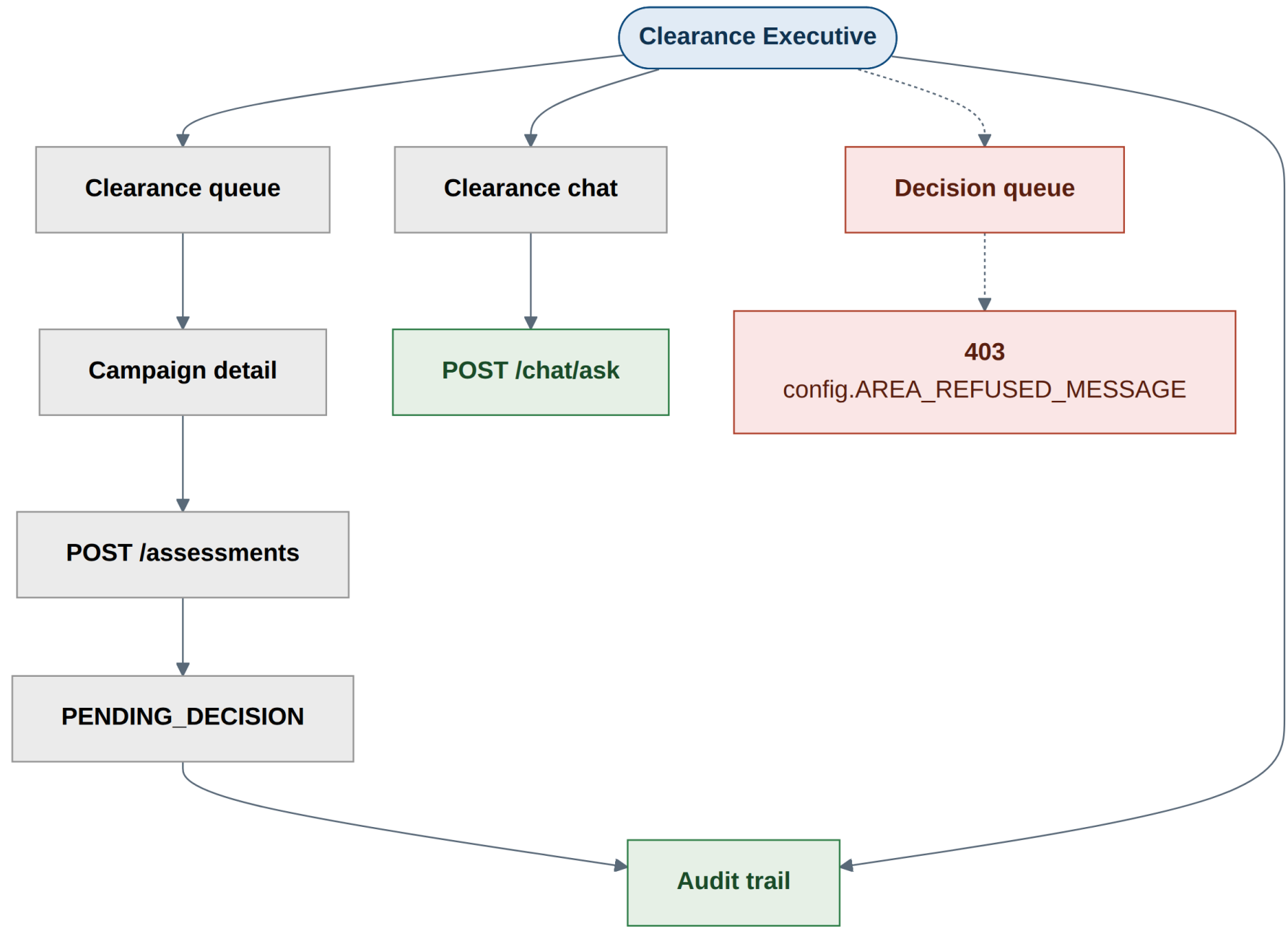
| Command | What it must make clear |
|---|---|
| **run_database_setup.py** | Where the database was created, which tables exist, and which accounts can sign in |
| **run_data_pipeline.py** | For each of the five extracts, how many rows arrived and how many survived cleaning, and where the quality report was written |
| **run_model_training.py** | For each of the three models, the score on data it did not learn from, the score on data it did, the gap between them, and whether it clears the minimum config sets for it |
| **run_knowledge_indexing.py** | How many documents were read, how many sections were found and how many passages were stored, and every category's five rules read back from its own code |
| **run_api_service.py** | That the service is listening on port **8081**, and where its interactive documentation can be opened |
| **run_dashboard.py** | That the dashboard is listening on port **8080**, and the address at which it can be opened |

**FLOW DIAGRAM:**

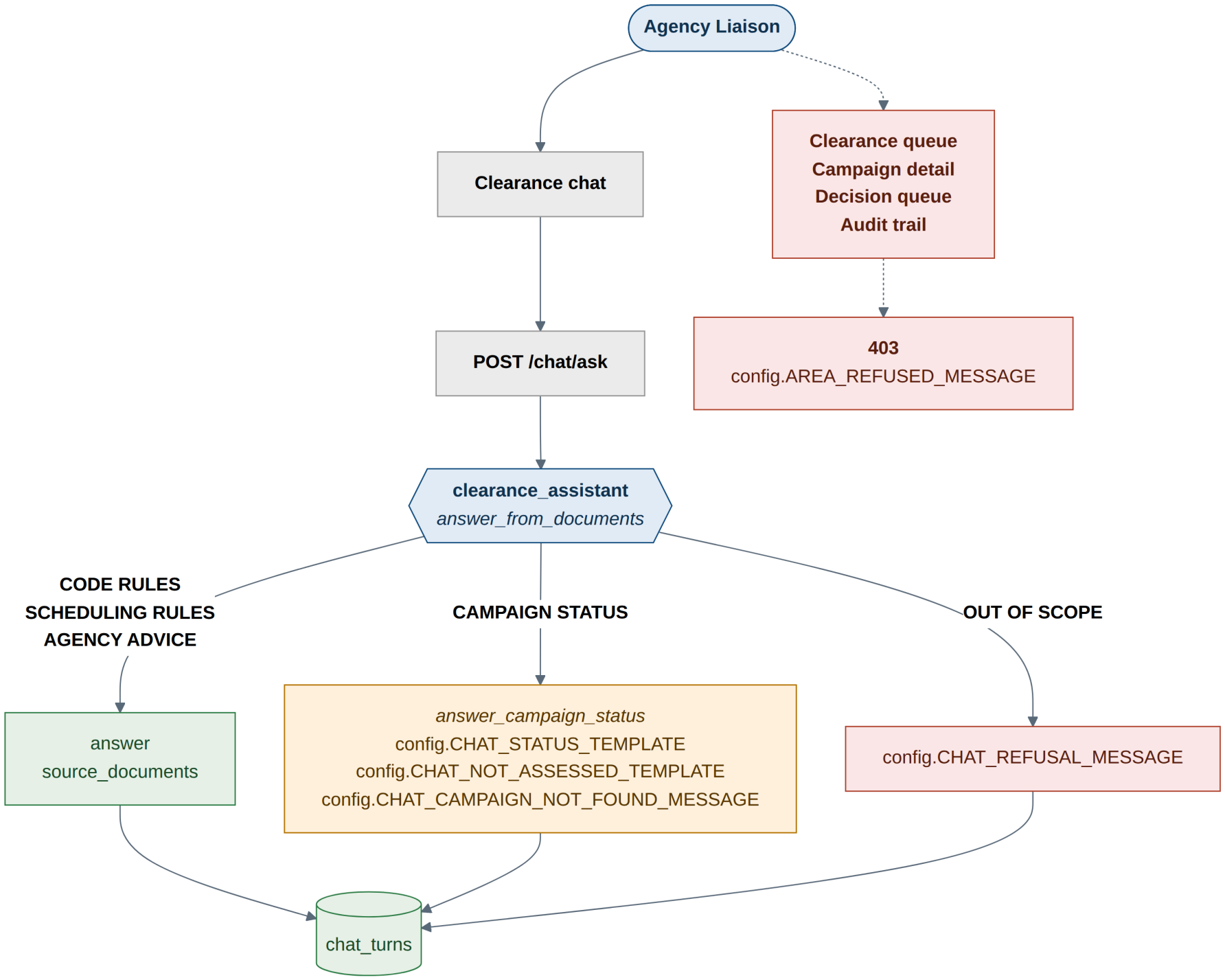
## Role Clearance Officer:



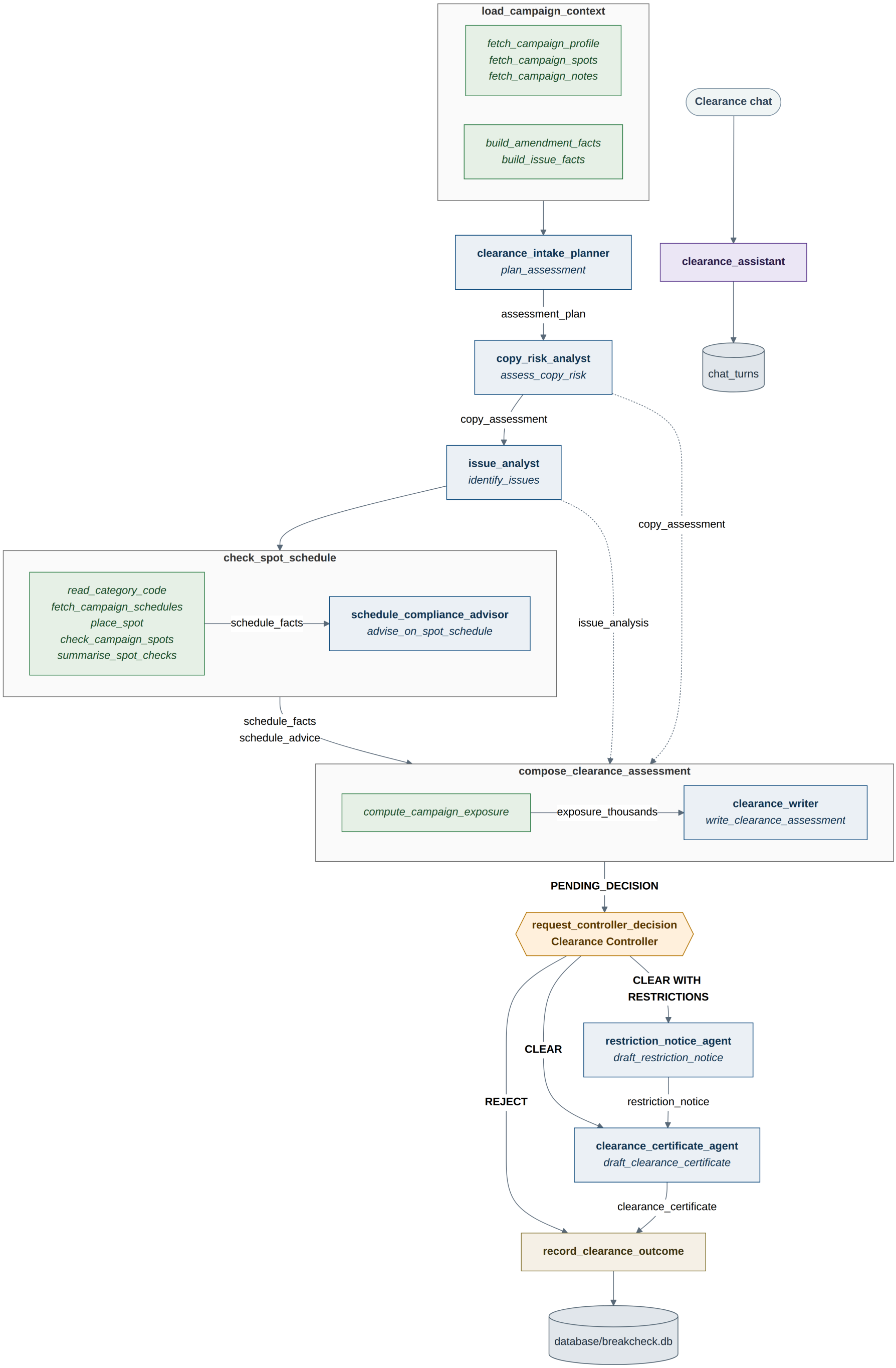

## Role Clearance Executive Officer:




## Role Agency Liaison Officer:




**Agent to Agent Communication Flow:**




**EXPECTED OUTPUT**
One correct solution's output. Scores and the passage count depend on the models and the chunking; the row, document and section counts are fixed. The rules are shown in outline.

**ENTRY POINT 1 — run_database_setup.py**

```
database: database/breakcheck.db
  tables: agent_runs, chat_turns, clearance_assessments, controller_decisions, llm_calls, user_
  account created  priya    Priya Raman     Clearance Controller
  account created  owen     Owen Hughes     Clearance Controller
  ...
  account created  tom      Tom Fletcher    Clearance Executive
  ...
  account created  lucy     Lucy Carter     Agency Liaison, agencies QSC, HWK, PVT, FWL
  ...
  account created  nadia    Nadia Haddad    Agency Liaison, agencies LNT, GRF, ASH, NLT
  accounts created: 14, already present: 0
```

Run a second time it creates nothing and reports **accounts created: 0, already present: 14**.

**ENTRY POINT 2 — run_data_pipeline.py**

```
campaign_submissions_train        30706 received ->  29697 kept  (1009 removed)
campaign_submissions_eval          8514 received ->   8224 kept  (290 removed)
programme_audiences_train         30877 received ->  29817 kept  (1060 removed)
programme_audiences_eval           7004 received ->   6770 kept  (234 removed)
clearance_notes                   34727 received ->  33045 kept  (1682 removed)

quality report written to output/data_quality_report.json
```

**ENTRY POINT 3 — run_model_training.py**

```
training three models
  amendment  held-out ROC AUC N.NNNN  (train N.NNNN, gap +N.NNNN)
  exposure   held-out R2 N.NNNN  MAE N.NN thousand  (train N.NNNN, gap +N.NNNN)
  issue      held-out macro F1 N.NNNN  (train N.NNNN, gap +N.NNNN)

metrics written to output/model_metrics.json
every model reaches the minimum config sets for it
```

A model below its minimum is named instead, and the run still finishes.

**ENTRY POINT 4 — run_knowledge_indexing.py**

```
building the clearance knowledge index
    documents read:  16
    sections found:  150
    chunks stored:   NNN
    store location:  database/chroma_store

rules read from each category's own code
  Alcohol                Code-Alcohol.pdf
      restricted hours: HH:MM to HH:MM, or none, spots at any time of the broadcast day
      under-18 index threshold NNN, children's programmes <rule>, at most N spots a channel a b
  Betting and gaming     Code-Betting-And-Gaming.pdf
      restricted hours: HH:MM to HH:MM
      live sport exception: centre breaks more than NN minutes after the start and more than NN
      under-18 index threshold NNN, children's programmes <rule>, at most N spots a channel a b
  Less healthy food      Code-Less-Healthy-Food.pdf
      ...
  Consumer credit        Code-Consumer-Credit.pdf
      ...
  Medicines and health   Code-Medicines-And-Health.pdf
      ...
```

A live sport exception line appears only for a code that states one.

**ENTRY POINT 5 — run_api_service.py**

```
Starting BreakCheck AI API on port 8081. Interactive documentation: http://localhost:8081/docs
INFO:     Started server process [NNNN]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8081 (Press CTRL+C to quit)
```

With the saved amendment model missing, it names what is missing and stops:

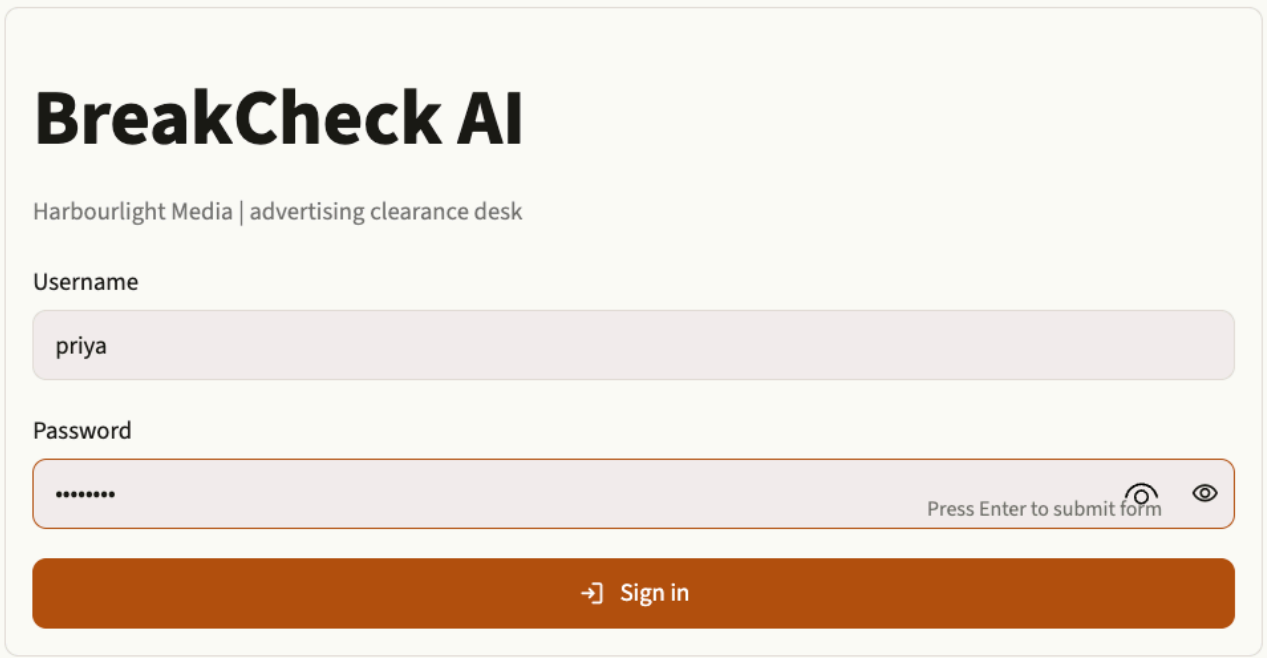
```
The service cannot start.
  output/models/amendment_model.pkl is missing; it comes from run_model_training.py
```

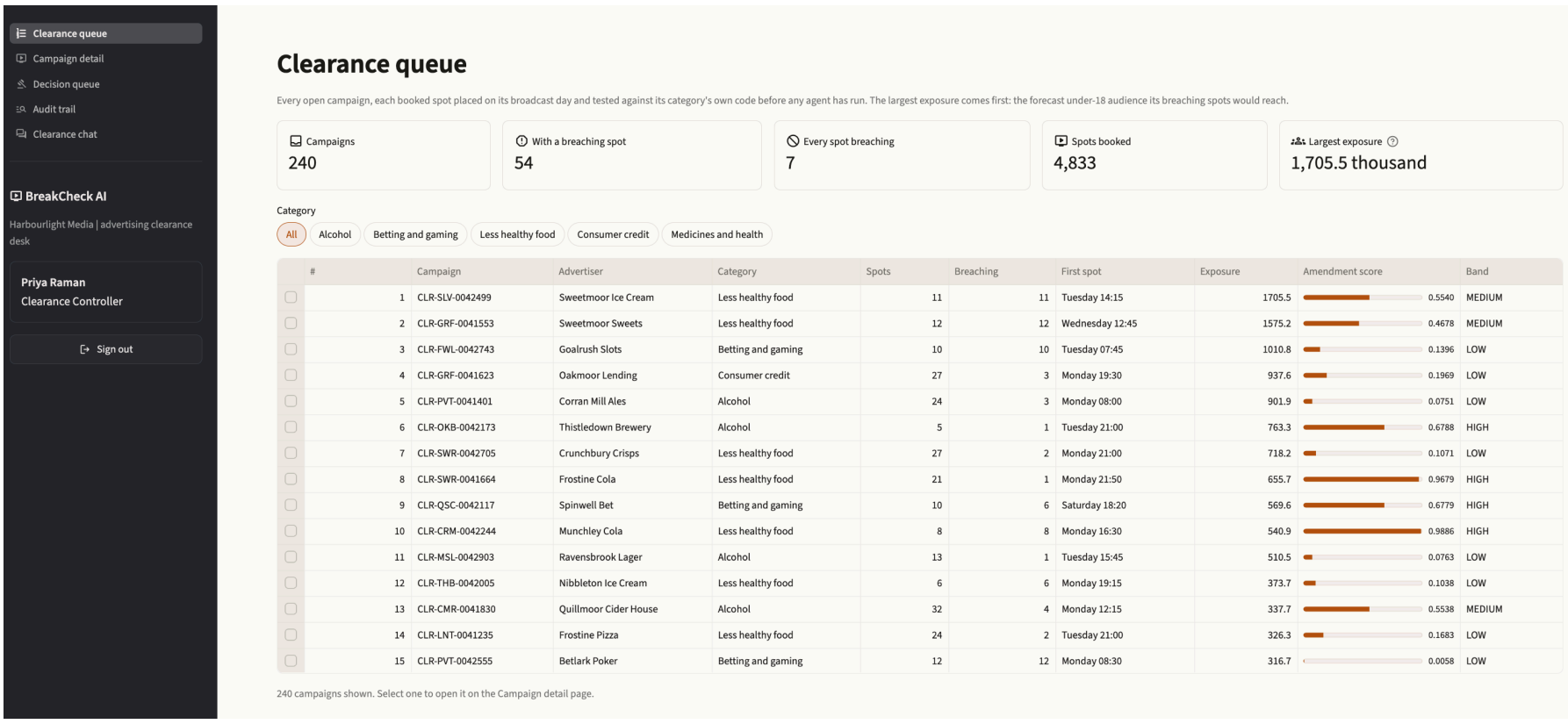
**ENTRY POINT 6 — run_dashboard.py**

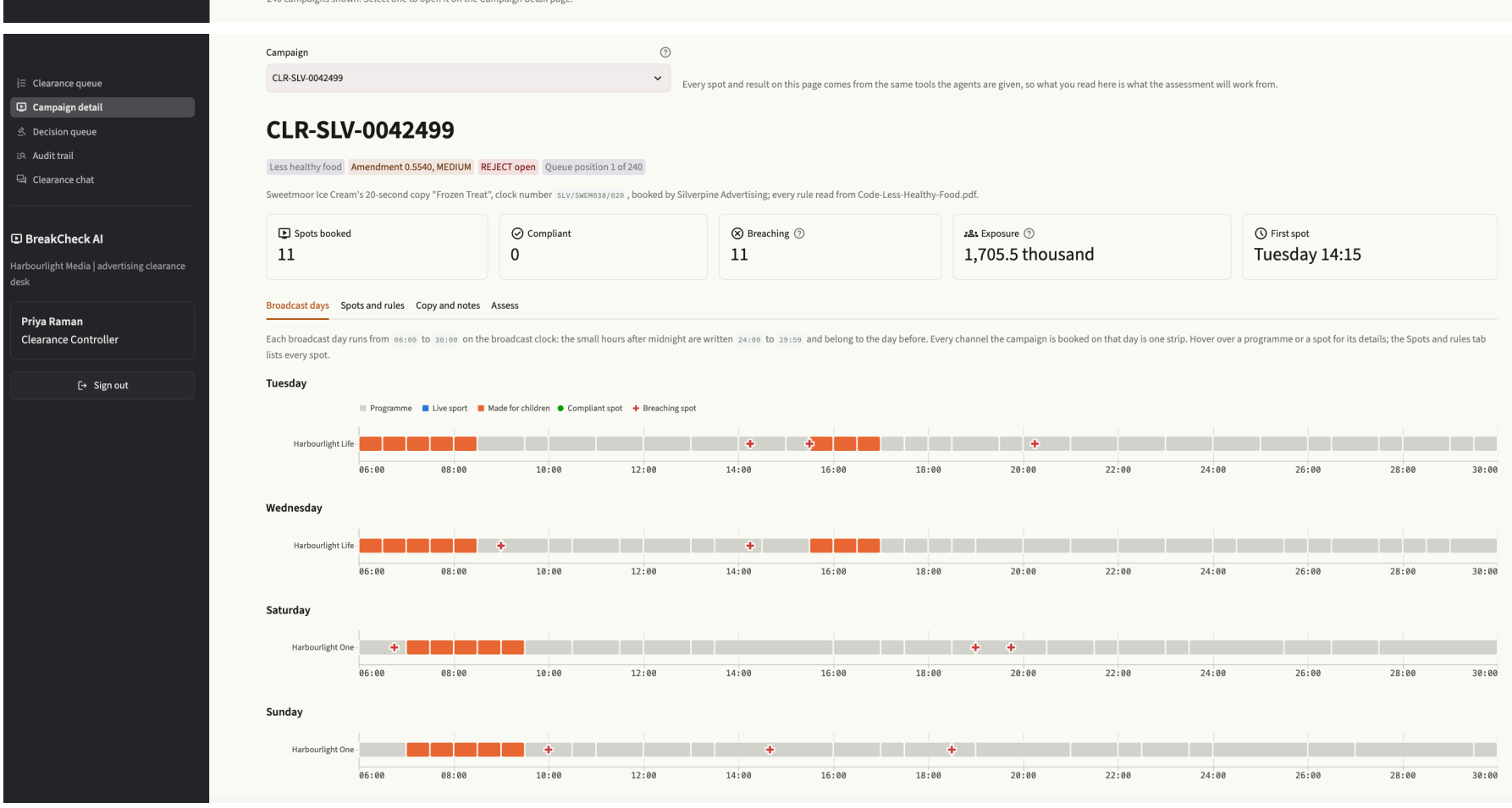
```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8080
```

**DASHBOARD OUTPUT:**



Clearance queue
Campaign detail
Decision queue
Audit trail
Clearance chat

BreakCheck AI

Harbourlight Media | advertising clearance desk

Priya Raman
Clearance Controller

Sign out

Campaign

CLR-LNT-0041235

Every spot and result on this page comes from the same tools the agents are given, so what you read here is what the assessment will work from.

# CLR-LNT-0041235

Less healthy food | Amendment 0.1683, LOW | CLEAR WITH RESTRICTIONS open | REJECT open | Queue position 14 of 240

Frostine Pizza's 60-second copy "Family Feast", clock number LNT/FRON066/060 , booked by Lantern Works; every rule read from Code-Less-Healthy-Food.pdf.

| Spots booked | Compliant | Breaching | Exposure | First spot |
|---|---|---|---|---|
| 24 | 22 | 2 | 326.3 thousand | Tuesday 21:00 |

Broadcast days | Spots and rules | Copy and notes | Assess

**The five rules, as Code-Less-Healthy-Food.pdf states them**

| Rule | Read as | The code's own sentence |
|---|---|---|
| WINDOW | 05:30 to 21:00 | Spots for this category may not be transmitted between 05:30 and 21:00. |
| INDEX | index 120 or more | Spots may not be transmitted in or adjacent to a programme whose under-18 audience index is 120 or more. |
| CHILDREN | in or adjacent | Spots may not be transmitted in or adjacent to a programme made for children. |
| FREQUENCY | 8 a channel a broadcast day | No more than 8 spots of one campaign may be transmitted on one channel in one broadcast day. |
| SEPARATION | 10 minutes apart | Two spots of one campaign on one channel must be at least 10 minutes apart. |

**Every spot, in transmission order** · 22 compliant, 2 breaching

| Spot | Channel | Day | Time | Break | In or between | Result |
|---|---|---|---|---|---|---|
| SPT-0041235-02 | Harbourlight Life | Tuesday | 21:00 | end | Holiday Reps (115) and Dream Home Makeover (45) | compliant |
| SPT-0041235-06 | Harbourlight Drama | Tuesday | 21:00 | end | The Doctors' Wing (64) and Detective Rowe (43) | compliant |
| SPT-0041235-12 | Harbourlight Life | Tuesday | 21:00 | end | Holiday Reps (115) and Dream Home Makeover (45) | SEPARATION |
| SPT-0041235-09 | Harbourlight One | Tuesday | 21:15 | centre | The Quayside Murders (38) | compliant |
| SPT-0041235-10 | Harbourlight Drama | Tuesday | 21:45 | centre | Detective Rowe (43) | compliant |
| SPT-0041235-04 | Harbourlight One | Tuesday | 23:45 | centre | Flatmates (73) | compliant |
| SPT-0041235-16 | Harbourlight Life | Tuesday | 23:45 | centre | Blind Date Club (74) | compliant |
| SPT-0041235-05 | Harbourlight Sport | Tuesday | 24:20 | centre | Classic Matches (28) | compliant |
| SPT-0041235-23 | Harbourlight Drama | Tuesday | 24:35 | centre | Late Film (27) | compliant |
| SPT-0041235-14 | Harbourlight One | Thursday | 21:15 | centre | The Quayside Murders (44) | compliant |

Clearance queue
Campaign detail
Decision queue
Audit trail
Clearance chat

BreakCheck AI

Harbourlight Media | advertising clearance desk

Priya Raman
Clearance Controller

Sign out

Campaign

CLR-LNT-0041235

Every spot and result on this page comes from the same tools the agents are given, so what you read here is what the assessment will work from.

CLR-LNT-0041235

Less healthy food | Amendment 0.1683, LOW | CLEAR WITH RESTRICTIONS open | REJECT open | Queue position 14 of 240

Frostine Pizza's 60-second copy "Family Feast", clock number LNT/FRON066/060 , booked by Lantern Works; every rule read from Code-Less-Healthy-Food.pdf.

Spots booked 24 | Compliant 22 | Breaching 2 | Exposure 326.3 thousand | First spot Tuesday 21:00

Broadcast days | Spots and rules | Copy and notes | Assess

**The copy**

Less healthy food, Fast food
60 seconds, 6 supers, legal text on screen 15 seconds
Voiceover Yes · humour Yes
Lead performer 18 to 24 · celebrity No · sports personality No
Child-appeal elements 1

**Claims and offers**

Objective claims 1
Price Yes · health No · environmental No · comparative No
Offer terms on screen: Yes
Bonus or free bet No
Representative APR on screen: not applicable outside consumer credit

**Booking and history**

24 spots on 4 channels
4% before 21:00, 0% in live sport
First spot 6 days after submission
Advertiser: 3 of 6 campaigns amended in the past year · new advertiser No
Agency: 34.3% of its copy amended

The amendment model scores the copy **0.1683**, band LOW. The issue model names **Price or offer claim**, weighted agreement 80% across the 4 notes it read, a legal review counting twice.

Note 1 · Executive first read · read as Price or offer claim

Frostine Pizza 60-second copy - Offer terms for the meal deal are shown but in small print below the price; check legibility. Raised on the call with the agency this morning.

Note 2 · Agency reply · read as Price or offer claim

Reply from Lantern Works: The agency has sent the full promotion terms for us to check against the ad. Their client has been told.

Note 3 · Legal review · read as Price or offer claim

Legal: a promotional price for the pizza must be available for the whole period and any limits shown with the offer. Happy to look again once we have it.

Note 4 · Executive update · read as Health claim

Spoke to Lantern Works, response expected tomorrow.

Clearance queue
Campaign detail
Decision queue
Audit trail
Clearance chat

BreakCheck AI

Harbourlight Media | advertising clearance desk

Priya Raman
Clearance Controller

Sign out

Campaign

CLR-LNT-0041235

Every spot and result on this page comes from the same tools the agents are given, so what you read here is what the assessment will work from.

CLR-LNT-0041235

Less healthy food | Amendment 0.1683, LOW | CLEAR WITH RESTRICTIONS open | REJECT open | Queue position 14 of 240

Frostine Pizza's 60-second copy "Family Feast", clock number LNT/FRON066/060 , booked by Lantern Works; every rule read from Code-Less-Healthy-Food.pdf.

Spots booked 24 | Compliant 22 | Breaching 2 | Exposure 326.3 thousand | First spot Tuesday 21:00

Broadcast days | Spots and rules | Copy and notes | Assess

Say what you want looked at. Five agents run in turn, one language model call each, and the campaign then waits for a Clearance Controller who did not start the assessment. No spot is cleared and nothing is sent to the agency.

What do you want looked at?

Need more info.

Start the assessment

ASM-EC0F996148BB is waiting for a Clearance Controller, recommending CLEAR WITH RESTRICTIONS.

This campaign for Frostine Pizza's "Family Feast" presents a low risk of requiring changes, with an amendment score of 0.1683. However, the advertiser's history of 50% of previous campaigns needing amendments, coupled with the inclusion of child-appeal elements and objective/price claims, warrants careful scrutiny. The primary issue identified is "Price or offer claim," with strong weighted agreement from the notes. While 22 spots are compliant, two breach the code: SPT-0041235-12 for SEPARATION and SPT-0041235-08 for WINDOW and INDEX. Given that some spots breach the code and some comply, the appropriate decision is CLEAR WITH RESTRICTIONS.

Clearance queue
Campaign detail
Decision queue
Audit trail
Clearance chat

BreakCheck AI

Harbourlight Media | advertising clearance desk

Priya Raman
Clearance Controller

Sign out

# Decision queue

2 waiting for a decision

CLR-LNT-0041235 · ASM-EC0F996148BB · recommends CLEAR WITH RESTRICTIONS

CLR-LNT-0041235 · ASM-EC0F996148BB · recommends CLEAR WITH RESTRICTIONS
CLR-QSC-0042117 · ASM-4050EA4A0F94 · recommends CLEAR WITH RESTRICTIONS

Recommended: CLEAR WITH RESTRICTIONS | Amendment 0.1683 | Less healthy food | Price or offer claim

| Spots booked | Compliant | Breaching | Exposure |
|---|---|---|---|
| 24 | 22 | 2 | 326.3 thousand |

This campaign for Frostine Pizza's "Family Feast" presents a low risk of requiring changes, with an amendment score of 0.1683. However, the advertiser's history of 50% of previous campaigns needing amendments, coupled with the inclusion of child-appeal elements and objective/price claims, warrants careful scrutiny. The primary issue identified is "Price or offer claim," with strong weighted agreement from the notes. While 22 spots are compliant, two breach the code: SPT-0041235-12 for SEPARATION and SPT-0041235-08 for WINDOW and INDEX. Given that some spots breach the code and some comply, the appropriate decision is CLEAR WITH RESTRICTIONS.

| Breaching spot | Channel | Day | Time | Codes | In or between |
|---|---|---|---|---|---|
| SPT-0041235-12 | Harbourlight Life | Tuesday | 21:00 | SEPARATION | Holiday Reps (115) and Dream Home Makeover (45) |
| SPT-0041235-08 | Harbourlight Drama | Saturday | 19:15 | WINDOW, INDEX | Westfield High (147) |

Read from Code-Less-Healthy-Food.pdf · first spot Tuesday 21:00 · notes agree 80% · assessed by priya

› What the executive asked, and what the planner decided to check

Clearance queue
Campaign detail
Decision queue
Audit trail
Clearance chat

BreakCheck AI

Harbourlight Media | advertising clearance desk

Priya Raman
Clearance Controller

Sign out

# Audit trail

Assessed campaigns | Clearance chat log

Status

PENDING_DECISION | CLEARED | CLEARED_WITH_RESTRICTIONS | REJECTED

| Cleared | Cleared with restrictions | Rejected | Pending decision |
|---|---|---|---|
| 1 | 0 | 1 | 2 |

| | Campaign | Status | Decided | By | Cleared | Withdrawn | Model calls |
|---|---|---|---|---|---|---|---|
| ☐ | CLR-LNT-0041235 | PENDING_DECISION | None | None | None | None | 5 |
| ☐ | CLR-MSL-0042822 | CLEARED | CLEAR | Priya Raman | 33 | 0 | 6 |
| ☐ | CLR-QSC-0042117 | PENDING_DECISION | None | None | None | None | 5 |
| ☐ | CLR-FWL-0042743 | REJECTED | REJECT | Owen Hughes | 0 | 10 | 5 |

Select a row to open its record: the decision, the documents sent to the agency and the language model calls the run recorded, one for each reply that was accepted.

Clearance queue
Campaign detail
Decision queue
Audit trail
Clearance chat

BreakCheck AI

Harbourlight Media | advertising clearance desk

Priya Raman
Clearance Controller

Sign out

# Clearance chat

What is the daily limit and the separation for alcohol spots?

CODE RULES

No more than 6 spots of one campaign may be transmitted on one channel in one broadcast day. Two spots of one campaign on one channel must be at least 20 minutes apart. Both rules are applied per campaign, per channel and per broadcast day, to the spots in the order they are transmitted.

From Code-Alcohol.pdf, Code-Betting-And-Gaming.pdf, Code-Less-Healthy-Food.pdf, Code-Consumer-Credit.pdf, Code-Medicines-And-Health.pdf, Scheduling-And-Audience-Rules.pdf

Can a betting advert run in a live football match before 9pm?

CODE RULES

Spots for betting and gaming may not be transmitted between 05:30 and 21:00. However, this restriction does not apply to a centre break within live sport coverage that begins more than 10 minutes after the coverage starts and more than 10 minutes before it ends. An end break is never within live sport coverage.

From Code-Alcohol.pdf, Code-Betting-And-Gaming.pdf, Code-Less-Healthy-Food.pdf, Code-Consumer-Credit.pdf, Code-Medicines-And-Health.pdf, Guide-Gambling-Inducements.pdf, Scheduling-And-Audience-Rules.pdf

And for less healthy food?

CODE RULES

Spots for less healthy food and drink may not be transmitted between 05:30 and 21:00. These restricted hours apply to every break, including a break within live sport coverage.

From Code-Alcohol.pdf, Code-Betting-And-Gaming.pdf, Code-Less-Healthy-Food.pdf, Code-Consumer-Credit.pdf, Code-Medicines-And-Health.pdf, Guide-Health-Claims.pdf

**What the assistant answers**

CODE RULES
SCHEDULING RULES
AGENCY ADVICE
CAMPAIGN STATUS
OUT OF SCOPE

Answers come only from Harbourlight Media's codes, policies and guides, never from the casebook of past clearances, and each lists the documents it was given to answer from. All five codes are read in full for every question. The assistant remembers your last 3 exchanges.

And for less healthy food?

CODE RULES

Spots for less healthy food and drink may not be transmitted between 05:30 and 21:00. These restricted hours apply to every break, including a break within live sport coverage.

From Code-Alcohol.pdf, Code-Betting-And-Gaming.pdf, Code-Less-Healthy-Food.pdf, Code-Consumer-Credit.pdf, Code-Medicines-And-Health.pdf, Guide-Health-Claims.pdf

What hours are consumer credit adverts kept out of?

CODE RULES

Spots for consumer credit may not be transmitted between 06:00 and 19:00. These restricted hours apply to every break, including a break within live sport coverage.

From Code-Alcohol.pdf, Code-Betting-And-Gaming.pdf, Code-Less-Healthy-Food.pdf, Code-Consumer-Credit.pdf, Code-Medicines-And-Health.pdf

What is the under-18 index threshold for medicines and health?

CODE RULES

Spots may not be transmitted in or adjacent to a programme whose under-18 audience index is 150 or more. The index is the forecast under-18 audience index that Harbourlight's audience planning team publishes for each programme before the booking week begins, as the programme schedule shows it. A spot in an end break is tested against both programmes, and breaks this rule if either of them is indexed at or above the figure in this section.

From Code-Alcohol.pdf, Code-Betting-And-Gaming.pdf, Code-Less-Healthy-Food.pdf, Code-Consumer-Credit.pdf, Code-Medicines-And-Health.pdf, Scheduling-And-Audience-Rules.pdf

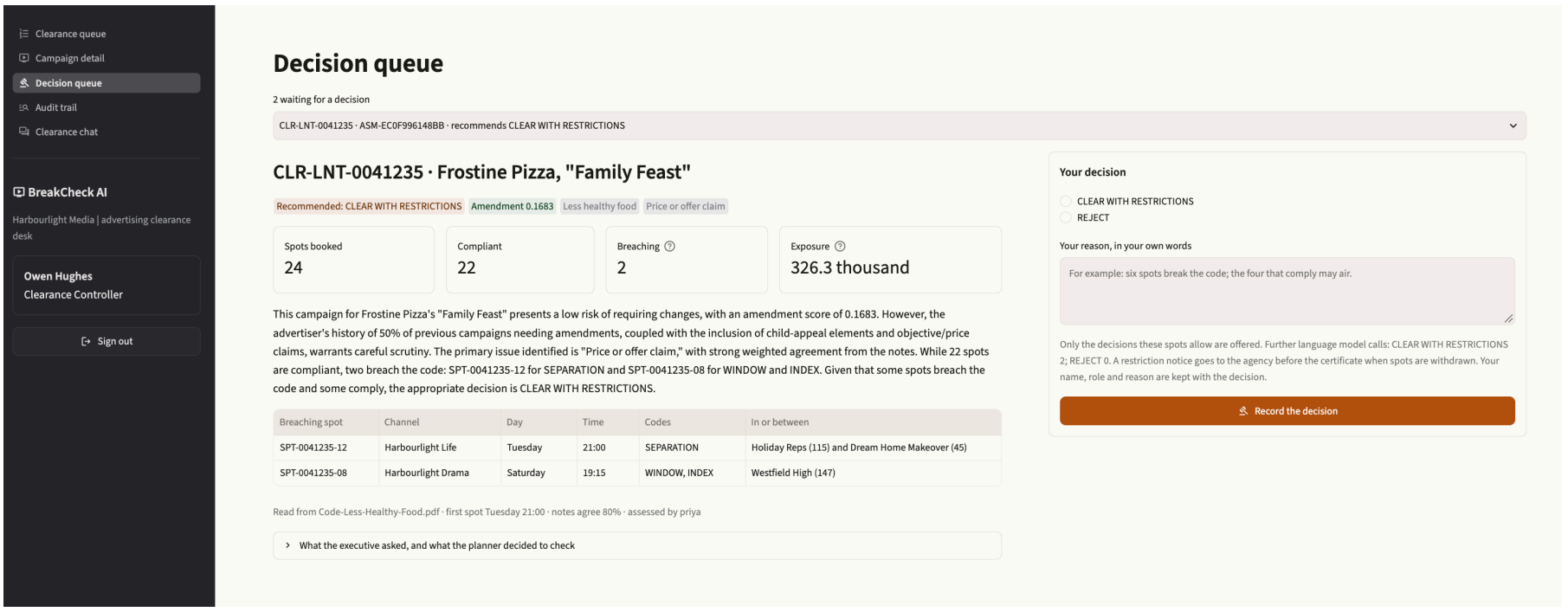
And what is its daily limit?

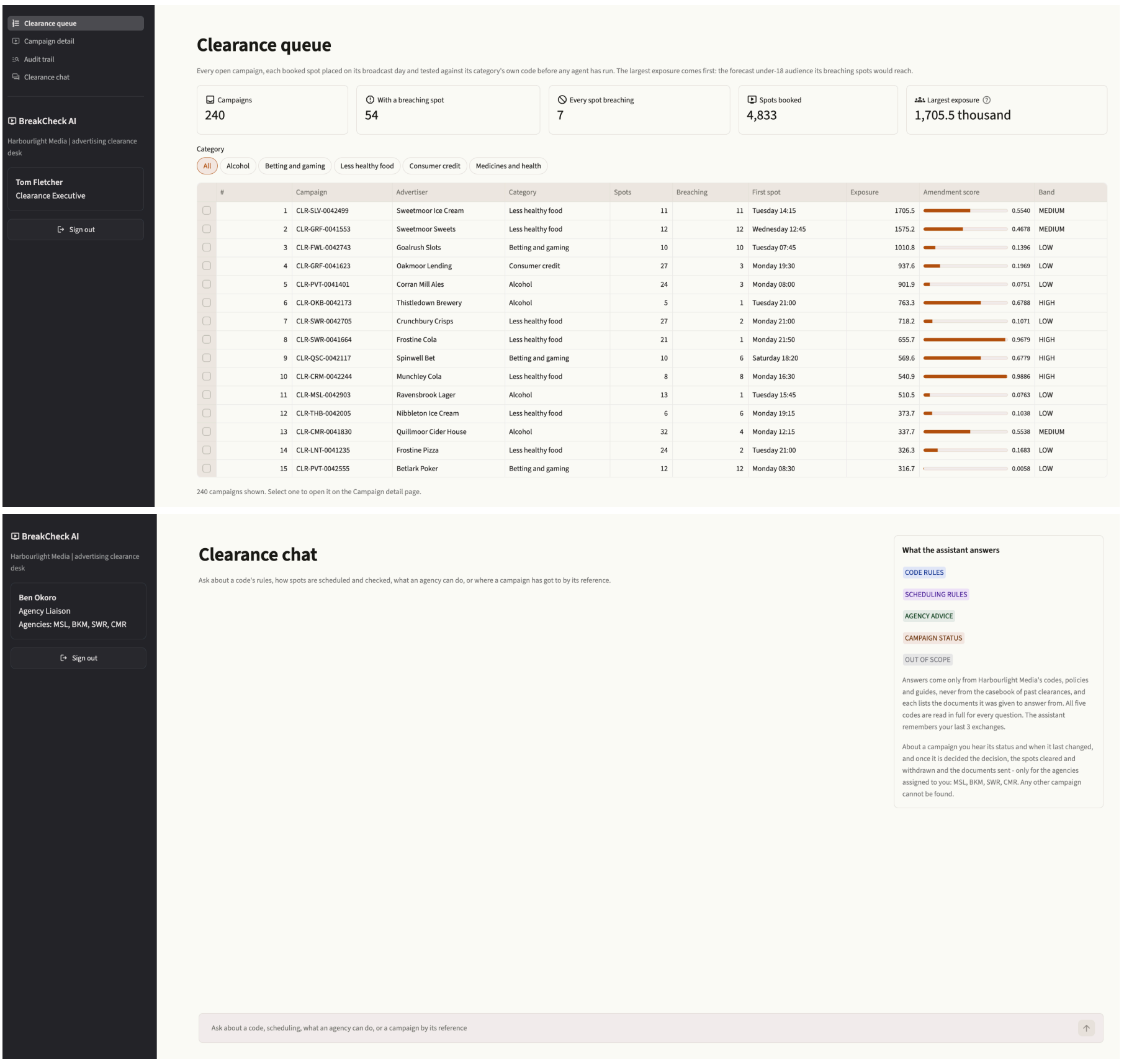
CODE RULES

No more than 8 spots of one campaign may be transmitted on one channel in one broadcast day. Two spots of one campaign on one channel must be at least 10 minutes apart. Both rules are applied per campaign, per channel and per broadcast day, to the spots in the order they are transmitted.

From Code-Alcohol.pdf, Code-Betting-And-Gaming.pdf, Code-Less-Healthy-Food.pdf, Code-Consumer-Credit.pdf, Code-Medicines-And-Health.pdf, Scheduling-And-Audience-Rules.pdf, Advertising-Clearance-

Ask about a code, scheduling, what an agency can do, or a campaign by its reference







**IMPLEMENTATION STEPS**

Every command is run from the Project folder.

**STEP 0. INSTALL AND CONFIGURE**

**cd Project**

**pip install -r requirements.txt**

**STEP 1. CREATE THE DATABASE**

**python3 run_database_setup.py**

**STEP 2. CLEAN THE RAW DATA**

**python3 run_data_pipeline.py**

**STEP 3. TRAIN THE MODELS**

**python3 run_model_training.py**

**STEP 4. BUILD THE KNOWLEDGE BASE**
**python3 run_knowledge_indexing.py**

**STEP 5. START THE REST SERVICE**
Leave this running and use a second terminal for anything else.
**python3 run_api_service.py**

**STEP 6. START THE DASHBOARD**
In a third terminal, with the service still running:
**cd Project**
**python3 -m streamlit run run_dashboard.py**